

Contents

Visual Layer Documentation	15
Guides	16
Getting Started	16
Introduction to Visual Layer	17
Browser Requirements	17
Key features	17
Use cases for using Visual Layer	17
Quick Start	19
Setting Up	19
Creating a Dataset	19
Exploring Your Dataset	19
Deploy Visual Layer	20
Visual Layer Highly Secured Cloud	21
On-Premises	22
Definitions & Entities	23
Annotations	23
Bounding Box	23
Cluster	23
Cluster View	23
Dataset	23
Dataset Analysis	23
Dataset Exploration	23
Dataset Exploration View	23
Dataset Inventory	24
Duplicates	24
Filtering	24
Enrichment	24
Image	24
Image View	24
Issue	24
Issue Type	24
Metadata	24
Mislabel	24

Object	25
Object View	25
Outliers	25
Saved View(s)	25
Selected Item(s)	25
Semantic Search	25
Similarity Results	25
Similarity Search	25
Similarity Vertex	25
User Tags	25
Video	26
Video Frame	26
Understanding Clusters	27
Cluster Previews	27
Example Views	27
Clustering Granularity	27
Finding Similar Items	28
Integrations	28
Amazon S3	29
Step 1: Configure access permissions for the S3 bucket	29
AWS access control requirements	29
Creating an IAM policy	29
Step 2: Create the IAM Role in AWS	30
Step 3: Grant the IAM User Permissions to Access Bucket Objects	30
Step 4: Send us the Role ARN value	31
Step 5: Create a Dataset	31
Public S3 Bucket Setup	32
Prerequisites	32
Step 1: Disable block public access settings	32
Step 2: Add the public bucket policy	32
Step 3: Create a dataset in Visual Layer	33
Related articles	33
Google Cloud Bucket	34
Prerequisites	34
Method 1: Filesystem Mounting with GCS FUSE (Recommended)	34
Step 1: Install Google Cloud SDK	34
Step 2: Authenticate with Google Cloud	34
Step 3: Install GCS FUSE	35
Step 4: Create Mount Point	35
Step 5: Authenticate for GCS FUSE	35
Step 6: Mount the Bucket	36
Step 7: Verify the Mount	36
Method 2: Command-Line Access	36
Step 1: Install Google Cloud SDK	36
Step 2: Authenticate with Google Cloud	36
Step 3: Access Bucket Contents	37
Integration with Visual Layer	37
Running Visual Layer Profiler	37

Configuration Options	38
Need Help?	38
Creating & Updating Datasets	38
Upload Data	39
1. Create a New Dataset	39
Name Your Dataset	39
Select Data Source	40
Preview Your Data	41
2. Add Annotations (Optional)	42
3. Upload and Index	42
4. Delete a Dataset	42
Custom Metadata	44
Pre-processing	44
Uploading the Metadata	44
Preparing the JSON File	44
How to Get media_ids:	45
Automation Option	45
Step 1: Declare a New Metadata Field	46
Request Body	46
For string, float, datetime, link	46
For enum	46
Response Body:	47
Step 2: Upload Metadata File	48
Upload Format	48
For string, float, datetime, link	48
For enum - when is_multi = false	48
For enum - when is_multi = true	48
Step 3: Check Upload Status	50
Response Example:	50
Error Response Example	50
Step 4: Retrieve Metadata	51
List Declared Metadata Fields	51
Get Metadata for an Image	51
Step 5: Search and Filter Using Custom Metadata	52
Supported filter types:	52
Notes	52
Custom Metadata Upload Script	53
Features	53
How the Script Works	53
Prerequisites	53
Install Dependencies	54
CSV Requirements	54
Required Column	54

Optional Metadata Columns	54
How to Get file_fullpath Values	54
Supported Field Types	54
String (--string-fields)	55
Float (--float-fields)	55
Datetime (--datetime-fields)	55
Enum (--enum-fields)	55
Multi-Enum (--multi-enum-fields)	55
Link (--link-fields)	56
Usage	56
Basic Example	56
Command-Line Parameters	57
Example Use Cases	57
Authentication (Cloud Only)	59
Get Your API Credentials	59
Authentication Example	59
Validation and Error Handling	60
Pre-Upload Validation Checks	60
Error Messages	60
Script Output	60
Successful Upload Example	60
Tips and Best Practices	61
Test with Sample Data First	62
Use Datetime Flexibility	62
Multi-Enum Format Options	62
Handle Empty Values	62
Use Descriptive Field Names	62
Troubleshooting	62
“No field types specified”	62
“Failed to export dataset”	62
“Validation failed with N errors”	63
“Enum field exceeds 600 unique values limit”	63
“Field already exists”	63
Getting Help	63
Expected Result in Visual Layer	63
Related Resources	63
Custom Metadata Upload Script Code	64
Key Features	64
Installation Requirements	64
Usage	64
CSV Format	65
Complete Script Code	65
DICOM Converter Script	83
Complete DICOM-to-Visual Layer Workflow	83
Step 1: Convert DICOM Files	83
Step 2: Create Visual Layer Dataset	83
Step 3: Export Dataset Metadata	83
Step 4: Upload DICOM Metadata	84

Step 5: Search and Filter by DICOM Metadata	84
Why Custom Metadata Matters	84
DICOM Converter Script	84
Prerequisites	84
Installation	84
Script Code	85
Basic Usage	85
DICOM Metadata Upload Script	85
Script Features	85
Script Code	85
Troubleshooting	85
Adapting This Pattern	86
Related Resources	86
DICOM Converter Script Code	87
Installation Requirements	87
Usage	87
Complete Script Code	87
DICOM Metadata Upload Script Code	100
Installation Requirements	100
Usage	100
Complete Script Code	100
Annotations Overview	116
What Are Annotations?	116
Common annotation types:	116
Why Are Annotations Important?	116
Supported Formats	116
How to Import Annotations	116
Preparing Annotation File	118
Tabular Formats: Parquet / CSV	119
Image Annotations	119
Example Format:	119
Example for Multiple Labels:	119
Object Annotations	119
Example Format:	119
JSON Annotations (COCO Format)	121
Example JSON Format:	121
Summary of Bounding Box Formats	121
Bounding Box Visual Layer Format	123
Overview	123
CSV Format Structure	123
Example Annotation	123
1.COCO (Common Objects in Context) Format	123
2.PASCAL VOC (Visual Object Classes) Format	124
3.YOLO (You Only Look Once) Format	124
4.TFRecord / TensorFlow Object Detection API Format	124
5.LabelMe Format	125

Converting to Visual Layer format	126
Converting YOLO Annotations to Visual Layer Format	127
YOLO Annotation Format	127
Conversion Process	127
Python Conversion Script	128
Script Explanation	129
Converting Segmentation Masks to Visual Layer Format	131
Segmentation Mask Annotation Format	131
Conversion Process	132
Python Conversion Script	132
Script Explanation	135
Usage	135
Creating Annotation File with Class Labels	136
Reusing Caption Data from Previous Datasets	137
Introduction	137
Overview	137
Using the Caption Extraction Script	137
Prerequisites	137
What the Script Does	137
Script Location	137
Installation	138
Step-by-Step Workflow	138
Step 1: Create Initial Dataset (with Captioning)	138
Step 3: Run the Extraction Script	138
Step 4: Copy to New Dataset Directory	139
Step 5: Create New Dataset (Fast!)	139
Understanding Relative Paths	139
Why Relative Paths?	139
Examples	139
How the Script Handles Paths	140
Complete Example	141
Initial State: Dataset 1 (with captioning)	141
Processing with Script	141
Dataset 2 (without captioning - fast!)	141
Troubleshooting	142
Images Not Found	142
Checking Parquet Contents	142
Captions Not Being Used	142
Script Errors	142
Related Documentation	143
Caption Extraction Script	144
Installation	144
Quick Usage	144
Script Code	144
Curating Datasets	147
User Tags	148
Tag Multiple Items	148
View and Remove Tags	148

Tag Individual Images	148
Using Exported VL Metadata	150
Step 1: Extracting Data Using Visual Layer's API	150
Simplified Python API	150
Usage Example	152
Expected Output	153
Step 2: Parsing Exported Metadata	153
Loading the Exported Metadata	154
View the Dataset Level Information	154
Get Image Level Details	154
Step 3: Analyzing Your Data	155
Filter By Uniqueness Score	155
Get Duplicate Images	155
Get Mislabeled	155
Step 4: Converting Video Frame Data to CSV	156
Example Video Frame JSON Structure	156
Converting Video Frame Data to CSV	157
Step 5: Converting Object Detection Data to CSV	158
Example Object Detection JSON Structure	158
Converting Object Detection Data to CSV	159
Expected CSV Output	160
Step 6: Analyzing Video Frame Similarity	160
Creating a Video Frame Data Table	160
Creating a Video Similarity Aggregation Table	162
Alternative Approach Using Clustering	163
Export Results to CSV	164
Expected Output	165
Step 7: Analyzing Mislabel Issues	165
Understanding Mislabel Structure	165
Comprehensive Mislabel Analysis	165
Mislabel Statistics and Analysis	167
High-Confidence Corrections Analysis	167
Summary Statistics and Export	168
Expected Output	168
Next Steps	169
Managing Datasets	169
Save Views	170
Why Use Views	170
Save a Query as a View	170
Explore a Saved View	171
Share a View	171
Delete a View	171
Share Dataset	172
Share a Dataset	172
Accept an Invite	172
Manage Sharing Access	173
Export Dataset	174
Export a Dataset	174

Export Options	174
Entire Dataset	174
Matching Filters	174
Current Selection	174
JSON Export Format	175
1. info	175
2. media_items	175
Metadata Items	175
Exploring Datasets	177
Exploration Overview	178
Semantic Search	179
What Is Semantic Search?	179
How It Works	179
Example Queries	179
Caption Search	180
Caption Search Operators	180
OR Operator	180
NOT Operator	180
Exact Match	181
Combining Operators	181
Visual Search	182
Overview	182
Similarity Search on a Cluster	182
Similarity Search on an Image or Object	182
Search from the Dataset Page	182
Search from the Cluster Details Page	182
Search by Object	183
Similarity Search Using a Region of Interest	183
Similarity Search by Image Upload	183
Filter Dataset	184
Filter Menu	184
Available filter types include:	184
Query Modal	184
Query Panel	184
Filter Logic & Behavior	184
Editing & Removing Filters	185
Unlabeled	186
Why Unlabeled Data Matters	186
How to Find Unlabeled Data	186
Steps to Apply the Filter	186
What Happens Next	186
What You Can Do with Unlabeled Data	186
Duplicates	187
What Are Duplicate Images and Objects?	187
Common Causes of Duplicates	187
Why It Matters	187
How to Prevent and Fix Duplicates	187
How to Detect Duplicates	188

Outliers	189
Common Causes of Outliers	189
Why It Matters	190
How to Detect Outliers in Visual Layer	190
Class Outlier	191
What Are Class Outliers?	191
Why It Matters	191
How to Detect	191
Select Unique Items	192
How It Works	192
How to Apply the Filter	192
Uniqueness in Exported Data	192
Overview	193
Dark	194
What Are Dark Images and Objects?	194
Common Causes of Darkness in Visual Data	194
Why It Matters	194
How to Detect and Remove Dark Images	194
Blurry	196
What Are blurry Images and Objects?	196
Common Causes of Blurriness	196
Why It Matters	196
How to Detect and Remove Blurry Images	196
Identifying Mislabeled Images and Objects	197
Why It Matters	197
Mislabeled Images vs. Mislabeled Objects	197
How to Fix It	198
Related Articles	198
Bright	199
What Are Bright Images and Objects?	199
Common Causes of Brightness Issues	199
Why It Matters	199
How to Detect and Remove Bright Images	199
Dataset Enrichment	200
Enrich Your Datasets with AI Models	201
Verify Dataset is Ready	201
Step 1: Select Models	201
Step 2: Enrich the Dataset	201
Explore Model Catalog	203
Available Enrichment Models	203
Coming Soon	204
Want Early Access?	204
Analyze Enriched Dataset	205
Common use cases:	205
Filter by Issues	205
Use Captions & Tags	205
Export Enriched Data	205
Included in the export:	205

Changelog	206
Release Notes & Product Updates	207
New Features	207
New Embedding Model	207
New Filter	207
Enhancements & Fixes	208
New Features & Enhancements	208
Improved UI Navigation	208
Optimized Filter Logic	208
Streamlined Insights	208
Class Outlier Detection	209
Better Documentation to Support You	209
Fixed Issues	209
API Reference	210
API Documentation	210
API Introduction	211
Cloud API vs On-Premises API	211
Authentication	212
Step 1: Get Your API Key and Secret	212
Request	212
Example Response	212
Step 2: Generate JWT Token for Authentication	212
Example Python Code	213
Step 3: Test Your JWT Token	213
Test Using curl	213
Test Using Python	214
Expected Success Response	214
Common Error Responses	215
Create a Dataset from a Public S3 Bucket	216
Prerequisites	216
Make the API Request	216
Request	216
Required Form Parameters	216
Example	216
Create a Dataset from a Local Folder for On-Prem Installations	217
API Overview	217
Endpoint Format	217
Parameters	217
Example Request	217
Example Response	217
Error Handling	218
Example Error Response	218
Notes & Considerations	218
Retrieve Dataset Status	219
API Request	219
Example Request	219
Example Output	219

Dataset Status Values	219
Share a Dataset with Another User	221
API Request	221
Form Parameters	221
Example Request	221
Possible Responses	221
Duplicate Retrieval	222
Step 1: Retrieve Internal Media ID	222
Example	222
Step 2: Retrieve Duplicates Using Media ID	222
Example	222
Visual Search	223
Upload an Image for Search	223
Request Parameters:	223
Visual Layer API - Dataset Exploration	223
Query Parameters	223
relevance_score	223
Sample Response	224
Example:	224
Notes:	224
Semantic Search	226
Overview	226
Basic Caption Search	226
Query Parameters	226
Example (cURL)	226
Sample Response	226
Response Concepts	227
Advanced Filtering	227
Retrieve Dataset Metadata	227
Python Client Example	228
Example Usage	229
Caption Search	230
Overview	230
Example	230
Authentication	230
Textual Caption Search	230
Endpoint	230
Required Parameters	230
Example (cURL)	231
Response Example	231
Response Breakdown	231
Filter and Refine	232
Add Similarity Threshold	232
Filter by Tags and Labels	232
Python Client	232
Example Usage	233
Advanced Techniques	233
Phrase Match	233

Semantic Exclusions	233
Best Practices	233
Pagination Helper	234
Error Handling	234
Limitations	234
Exporting a Dataset via curl	236
Initiate Export	236
Sample Response	236
Poll Export Status	236
Download ZIP	236
Unzip and Explore	237
Full Automation Script	237
Filter Exported Data by Uniqueness	238
Copy Filtered Images	238
Summary	238
Running Visual Layer Within an Iframe	239
Embedding Visual Layer Search with Seamless Authentication	239
Step 1: Basic Iframe Integration	239
Step 2: Cross-Domain Authentication Setup	239
DNS Configuration	239
Authentication Cookie	239
Cookie Encryption	240
Visual Layer Server Setup	240
Full Example Flow	240
Self-Hosting	242
Installation and Configuration	242
System Requirements	243
System Configuration	243
Dependencies	243
NVIDIA GPU Setup (GPU machines only)	243
Advised Tools for Troubleshooting/Maintenance	244
Required Permissions and Access	244
Set Up the Environment	245
Log into Docker Image Repo	245
Run the installer	245
Air-Gapped Environment Installation	246
Prerequisites	246
Step 1: Docker Authentication	246
Step 2: Required Docker Images	246
Step 3: Get Installation Script	247
Step 4: Run Installation	247
For GPU-enabled Systems:	247
For CPU-only Systems:	247
Step 5: Verification	247
Troubleshooting	247
Configure Settings	248
S3 Configuration	249

□ Configuration Flags	249
SERVE_ORIGINAL_IMAGE: true	249
PREPROCESS_IMAGE_TRANSFER: "keep_original_filename"	249
CDN_FULLPATH: "/image"	249
SOURCE_USE_FULLPATH: true	249
REMOTE_SAVE: "s3://mybucket/visual-layer-intermediate/"	250
USE_S5CMD_WHEN_AVAILABLE: true	250
SKIP_IMAGE_THUMBNAIL_GENERATION: true	250
SAVE_THUMBNAILS_METHOD: "crops"	250
EXPORT_ENTITIES_RESULT_BUCKET: "mybucket"	250
OBJECT_STORE_URL: "http://minio:9000"	250
□ AWS Credentials Setup	251
□ File Format	251
□ File Location	251
□ Security Best Practices	251
□ Custom SSL Certificate (Optional)	251
□ What it does:	251
□ Usage	251
□ Folder Structure Summary	252
□ Need Help?	252
Usage	252
Basic Operations	253
Connect to the Application	253
Start the System	253
Stop the System	253
Creating Datasets	253
Accessing Logs & Support	253
The VL Profiler Script: run_profiler.sh	254
Usage	254
Parameters	254
Dataset Path (-p, <i>required</i>)	254
Dataset Name (-n, <i>required</i>)	254
Execution Mode (-e)	254
Reduce Disk Space (-r)	255
Help (-h)	255
Examples	255
Execution Modes	255
Compose Mode (Default)	255
Local Mode	256
Data Source Support	256
Local Paths	256
Remote Paths	256
Supported File Formats	256
Error Handling	257
Dependencies	257
Output	257
Compose Mode	257
Local Mode	257

Performance Considerations	257
Reduced Disk Space Mode (-r)	257
Hardware Configuration	258
Integration	258
Troubleshooting	258
Administration	258
User Management: ADFS (OIDC) Integration Guide	259
OIDC Authentication Flow	259
Information Required from Your Side	259
1. Well-Known Configuration URL	259
2. Client ID and Client Secret	259
3. Allowed Scopes	259
4. Example Tokens	260
Information We Will Provide	260
1. Redirect URI	260
Required Changes in Your OpenShift Settings	260
Update or Add the Following Settings:	260
Frontend (FE) Settings:	260
Permissions Management	261
Additional Notes	261
Troubleshooting	262
Resolving Common Errors	262

Visual Layer Documentation

Guides

Getting Started

Introduction to Visual Layer

Welcome to the Visual Layer documentation! This guide introduces you to Visual Layer, a powerful platform designed to revolutionize how you manage and analyze visual data. By addressing challenges like disorganization, scattered storage, and poor data quality, Visual Layer empowers teams to work more efficiently, improve model performance, and reduce costs. Its scalable and collaborative tools make it indispensable for handling datasets of any size, enabling you to streamline workflows and unlock actionable insights.

Visual Layer provides a centralized platform to curate, manage, and analyze visual data at scale. By leveraging advanced filtering, visualization tools, and organizational features, it transforms how teams interact with complex datasets. Whether you're looking to save time, enhance collaboration, or improve the quality of your data, Visual Layer seamlessly integrates into your workflows to drive innovation and success.

Browser Requirements

Chrome browser is recommended for the best Visual Layer experience. While other browsers like Firefox may work for basic functionality, they are not fully supported and may have compatibility issues with advanced features.

[Embedded content]

Key features

At its core, Visual Layer is built to simplify the complexities of working with large-scale datasets. It provides tools for:

- **Dataset exploration:** Explore and identify patterns and outliers by going over clusters of similar images, objects, and video frames
- **Semantic & Visual search:** Quickly find relevant data using natural language queries or a reference image/object.
- **Data organization:** Tag relevant data or save filtering queries as Views for future use
- **Data quality improvement:** Identify, review and address critical issues like duplicates, annotation errors and outliers
- **Data enrichment:** Enhance your data by generating metadata using advanced, cutting-edge models

Use cases for using Visual Layer

Visual Layer is versatile and can be applied across various industries and scenarios. Here are some common use cases:

- **AI models development:** Prepare and refine datasets for training models by identifying and removing duplicates or outliers, ensuring data quality and representativeness
- **Manufacturing & logistics:** Optimize workflows and manufacturers to harness the full potential of your visual data for a competitive edge
- **Defense & security:** Manage and analyze video feeds for event detection and categorization

- **Media & entertainment:** Organize large media libraries and quickly locate specific assets using powerful filtering tools
- **Retail & E-commerce:** Analyze product imagery for consistency, quality control, and trend identification
- **Healthcare & research:** Explore medical imaging datasets to uncover patterns or anomalies efficiently

Ready to dive in? Check out our **Quick Start guide** to unlock the full potential of your visual data.

Quick Start

Setting Up

To get started with Visual Layer:

1. Visit the Login page
 2. Sign in with your Google or GitHub account.
 3. Access the Dataset Inventory — the homepage where your datasets live.
 4. Browse public datasets or create your own.
-

Creating a Dataset

1. Click “New Dataset” on the Dataset Inventory page.
2. Give your dataset a name - or keep the suggested one.
3. Upload your images or videos. You can also upload annotations at this stage (optional).
4. Visual Layer will begin indexing your data. You’ll receive an email once it’s ready to explore.

[Video content]

□ For more details, check out the full guide: [Creating a Dataset](#)

Exploring Your Dataset

Once your dataset is ready to explore:

1. Click your dataset card to open the Dataset Exploration View.
2. Begin filtering, searching, tagging, and identifying key patterns using Visual Layer’s built-in tools.
3. Try visual and text-based search, inspect clusters, and identify potential issues like mislabels, duplicates, or outliers.

[Video content]

□ For more details, check out the [Exploring Datasets](#) page

Deploy Visual Layer

Visual Layer Highly Secured Cloud

Visual Layer Cloud is a fully managed platform hosted on **AWS Cloud Services**, offering:

- **Secure** multi-tenant infrastructure
- **Fast onboarding**– Start with a trial and seamlessly scale to production
- **Cost-efficient pricing**
- **Latest features and enhancements**– Delivered through automatic updates

This option is ideal for teams looking to to **get started quickly** and for organizations with **strict governance requirements**, such as data residency and advanced security policies.

On-Premises

Visual Layer's Self-Managed deployment lets you run the platform entirely on your own infrastructure.

- **Self-hosted**– Runs fully within your private environment
- **Packaged solution**– Comes as a fully self-contained, easy-to-deploy package
- **Full data control**– No external dependencies. Your data stays within your infrastructure

This option is ideal for teams that require **full control over their data and environments**.

Definitions & Entities

This glossary provides definitions and explanations for key terms and entities relevant to Visual Layer. Use it as a quick reference to understand the platform's core concepts and features.

Annotations

Descriptive metadata or markers added to images or objects, such as bounding boxes or class labels, to enhance dataset usability.

Bounding Box

A shape annotation drawn around an object in an image to define its position and size for object detection tasks.

Cluster

A set of data entities grouped by a common characteristic. Images are grouped by a sophisticated similarity algorithm.

Cluster View

A structured interface that visually presents groups of similar images, allowing users to explore related data efficiently.

Dataset

A collection of visual data, such as images or videos, that users upload to Visual Layer for organization, exploration, and analysis.

Dataset Analysis

A process in which the images in a given dataset are tested for possible issues from the known issue types and are grouped together for easy visualization. The result of the analysis is zero or more issues and zero or more Clusters.

Dataset Exploration

The process of browsing, filtering, and searching through dataset contents, typically for data selection or inspection.

Dataset Exploration View

The primary view for exploring and analyzing a dataset. It provides access to images, objects, clusters, metadata filters, and search tools.

Dataset Inventory

An overview of all available datasets in your Visual Layer environment.

Duplicates

Identical or near-identical images or videos within a dataset. Identifying and removing duplicates improves data quality and prevents redundancy.

Filtering

The process of reducing a set of items by applying conditions to metadata properties. Filters can be applied to the entire dataset, a cluster, or a similar images.

Enrichment

The process of adding metadata or attributes to dataset items to enhance their value. This includes annotations, tags, or external data sources.

Image

A visual file (e.g., PNG, JPG, JPEG, GIF, TIFF, HEIC, BMP, WEBP) representing an object or scene. It can be an artifact, such as a photograph or other two-dimensional picture, that resembles a subject.

Image View

Displays clusters of visually similar images across your dataset, enabling efficient visual browsing.

Issue

A concrete instance of an issue type, typically linked to one or more images (e.g., train/test leakage, duplicates, or data drift).

Issue Type

A predefined category of problem identified during dataset analysis. It generates issue instances based on specific patterns or anomalies.

Metadata

Information associated with each visual item, such as filename, dimensions, timestamps, and user-defined tags. Crucial for organizing and filtering datasets.

Mislabel

An incorrect or inconsistent label applied to a dataset item. Detecting and correcting mislabels ensures dataset accuracy.

Object

A region of interest within an image, usually representing an instance of a specific class (e.g., a face, vehicle, or product).

Object View

Displays clusters of annotated or enriched objects grouped by visual similarity. This view focuses on object-level insights rather than whole images.

Outliers

Data points that differ significantly from the rest of the dataset. Identifying outliers can improve data quality and uncover unique patterns.

Saved View(s)

Custom configurations of filters and sorting applied to a dataset. Saved Views allow users to revisit specific subsets of data and share them with collaborators.

Selected Item(s)

The image, object, or cluster a user selects to view in detail or perform an action on.

Semantic Search

A search technique based on meaning and context rather than exact keyword matches, enabling more intuitive dataset queries.

Similarity Results

Images or clusters that are visually similar to a selected item (the “vertex”).

Similarity Search

The process of finding visually similar images or clusters for a given query item.

Similarity Vertex

The reference item (image, object, or cluster) used as the starting point for similarity search.

User Tags

Custom labels applied by users to annotate or categorize dataset items. Tags support better organization, search, and collaboration.

Video

A media file (e.g., MP4, AVI, MOV, MKV, WEBM, MPG, MPEG) containing a sequence of moving images. Videos are treated as datasets and can be analyzed frame-by-frame.

Video Frame

An individual still image extracted from a video sequence. Each frame can be analyzed or annotated independently.

```
{/* Filter Menu  
Query Modal  
Query Tab  
Query Panel */}
```

Understanding Clusters

Managing large collections of images and videos can be time-consuming. To simplify exploration, Visual Layer automatically groups similar **images** and **Core Frames** (extracted from videos) into **Clusters**. Clusters are structured based on the selected view—either as full images in **Image View**, or as annotated regions in **Object View**.

Clusters help you:

- **Explore** and **filter** vast amounts of visual data
- **Tag** and **sort** content based on similarity
- **Identify patterns** across groups of related images or objects

Cluster Previews

Clusters are displayed as preview cards to help you quickly understand the structure of your data. Each cluster preview includes:

- **Number of items** – The total number of items that match your current filters
- **Prominent labels** – The most frequent labels within the cluster
- **Representing images** – A few key examples (images or Core Frames) that visually summarize the cluster

[Image: https://files.readme.io/5e0d818-Screenshot_2024-05-21_at_10.47.33.png]

Example Views

Clusters in **Image View**:

[Image: https://files.readme.io/3d225ad-Screenshot_2024-05-29_at_17.29.51.png]

Clusters in **Object View** (objects are cropped):

[Image: https://files.readme.io/711dd8a-Screenshot_2024-05-29_at_17.35.27.png]

Clustering Granularity

The **Granularity knob**, located in the bottom-right corner of the screen, lets you adjust the similarity threshold for clustering.

[Image: https://files.readme.io/8b5976c48d84e6affe81677bfe61cd8aba53e86c8426efad270310738846750f-Scene-1_copy.gif]

- Move the knob to the **left** for looser clusters — broader grouping with lower similarity thresholds (*default setting*)

- Move the knob to the **right** for stricter clusters — tighter grouping with higher similarity thresholds

Finding Similar Items

When hovering over a cluster preview, you can search for visually similar images or objects depending on the current view.

For more details, check out the Visual Search page.

[Image: https://files.readme.io/dc4dd7ba2b5a177b242a5d10947d72d52a94e272736c4976bbbaa736ae0e1497-Scene-1_copy_3.gif]

Integrations

Amazon S3

Step 1: Configure access permissions for the S3 bucket

AWS access control requirements

VisualLayer requires the following permissions on an S3 bucket to be able to access files in the bucket:

- s3:GetBucket
- s3:GetObject
- s3:GetObjectVersion
- s3:ListBucket

Creating an IAM policy

The following step-by-step instructions describe how to configure access permissions for VisualLayer in your AWS Management Console so that you can use an S3 bucket to load and unload data:

1. Log into the AWS Management Console
2. From the home dashboard, search for and select **IAM**
3. From the left-hand navigation pane, select **Account settings**
4. From the left-hand navigation pane, select **Policies**
5. Select **Create Policy**
6. For **Policy editor**, select **JSON**
7. Add a policy document that will allow VisualLayer to access the S3 bucket. Copy and paste the text into the policy editor:

□ Make sure to replace <bucket> with your actual bucket name

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:GetObjectVersion"
      ],
      "Resource": "arn:aws:s3:::<bucket>/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetBucket*"
      ]
    }
  ]
}
```

```

    ],
    "Resource": "arn:aws:s3:::<bucket>"
  }
]
}

```

Step 2: Create the IAM Role in AWS

To configure access permissions for VisualLayer in the AWS Management Console, do the following:

1. From the left-hand navigation pane in the Identity and Access Management (IAM) Dashboard, select **Roles**
2. Select **Create role**
3. Select **AWS account** as the trusted entity type
4. In the **Account ID** field, enter your own AWS account ID temporarily. Later, you modify the trust relationship and grant access to VisualLayer
5. Select the **Require external ID** option. An external ID is used to grant access to your AWS resources (such as S3 buckets) to a third party like VisualLayer. Enter a placeholder ID such as 0000
6. In a later step, you will modify the trust relationship for your IAM role and specify the external ID for your storage integration and click **Next**
7. Select the policy you created in Step 1 and click **Next**
8. Enter a name and description for the role, then select **Create role**. You have now created an IAM policy for a bucket, created an IAM role, and attached the policy to the role
9. On the role summary page, locate and record the **Role ARN** value, you will be asked to send it to us in step 4

Step 3: Grant the IAM User Permissions to Access Bucket Objects

The following step-by-step instructions describe how to configure IAM access permissions for VisualLayer in your AWS Management Console so that you can use a S3 bucket:

1. Log in to the AWS Management Console
2. Select **IAM**
3. From the left-hand navigation pane, select **Roles**
4. Select the role you created in Step 2
5. Select the **Trust Relationships** tab
6. Select **Edit Trust Policy** and insert the following:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::027730031917:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {}
    }
  ]
}
```

7. Select **Update Policy** to save your changes

Step 4: Send us the Role ARN value

1. Go to the summary page of the IAM Role you created on step 2
2. Locate and record the Role ARN value, and send it to support@visual-layer.com with the title "Private S3 bucket"
3. Wait for us to review and confirm this was received and configured

Step 5: Create a Dataset

1. Create a new Dataset
2. Select Public ☐ S3 Bucket ☐ as the data source
3. Insert the URL to your private AWS S3 bucket

Public S3 Bucket Setup

This guide explains how to configure an Amazon S3 bucket for public access so you can create datasets in Visual Layer cloud. Making a bucket public allows anyone with the bucket URL to access its contents.

Note: Making a bucket public means anyone with the URL can access the data. Only use this approach for non-sensitive data that you intend to share publicly.

Prerequisites

Before configuring your bucket, ensure you have the following:

- An AWS account with permissions to modify S3 bucket settings
- An existing S3 bucket containing the data you want to use
- Visual Layer cloud account

Step 1: Disable block public access settings

AWS blocks public access to S3 buckets by default. You need to disable this protection before applying a public bucket policy.

1. Log into the AWS Management Console.
2. Navigate to the S3 service.
3. Select the bucket you want to make public.
4. Select the **Permissions** tab.
5. In the **Block public access (bucket settings)** section, select **Edit**.
6. Uncheck **Block all public access**.
7. Select **Save changes**.
8. Confirm by typing `confirm` in the confirmation dialog.

Step 2: Add the public bucket policy

After disabling the block public access settings, you need to add a bucket policy that grants public read access.

1. In the **Permissions** tab of your bucket, scroll down to **Bucket policy**.
2. Select **Edit**.
3. Add the following policy, replacing `<bucket_name>` with your actual bucket name:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```



```

        "Sid": "PublicReadGetObject",
        "Effect": "Allow",
        "Principal": "*",
        "Action": "s3:GetObject",
        "Resource": "arn:aws:s3:::<bucket_name>/*"
    },
    {
        "Sid": "PublicListBucket",
        "Effect": "Allow",
        "Principal": "*",
        "Action": "s3:ListBucket",
        "Resource": "arn:aws:s3:::<bucket_name>"
    }
]
}

```

4. Select **Save changes**.

Your bucket is now configured for public access.

Step 3: Create a dataset in Visual Layer

Once your bucket is publicly accessible, you can create a dataset in Visual Layer using the bucket URL.

1. Create a new dataset in Visual Layer.
2. Select **Public S3 Bucket** as the data source.
3. Enter your S3 bucket URL in the format: `s3://your-bucket-name` or `https://your-bucket-name.s3.amazonaws.com`.

Visual Layer will import the data from your public bucket.

Related articles

- Amazon S3 (Private Buckets)
- Create a Dataset from a Public S3 Bucket (API)
- Create/Delete Datasets

Google Cloud Bucket

This guide explains how to access Google Cloud Storage (GCS) buckets from on-premise machines using two different methods: filesystem mounting with GCS FUSE and command-line access with Google Cloud SDK.

❑ **Recommended Approach:** We recommend using **Method 1 (Filesystem Mounting)** as it provides seamless integration with Visual Layer's profiler and allows you to work with your data as if it were stored locally.

Prerequisites

- Linux-based system (Ubuntu/Debian recommended)
- Internet connectivity
- Google Cloud account with access to the target bucket
- Appropriate permissions on the GCS bucket

Method 1: Filesystem Mounting with GCS FUSE (Recommended)

This is the recommended approach for Visual Layer integration as it provides direct access to your GCS bucket without needing to copy files locally.

GCS FUSE allows you to mount a GCS bucket as a local filesystem, making it appear as a regular folder on your machine.

Step 1: Install Google Cloud SDK

Update your system and install required dependencies:

```
sudo apt-get update && sudo apt-get install -y curl unzip python3
```

Download and install the Google Cloud CLI:

```
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-465.0.0-linux-x86_64.tar.gz
tar -xzf google-cloud-cli-465.0.0-linux-x86_64.tar.gz
./google-cloud-sdk/install.sh
```

Restart your shell to apply the changes:

```
exec -l $SHELL
```

Step 2: Authenticate with Google Cloud

Choose one of the following authentication methods:

Option A: User Authentication

Authenticate using your Google account:

```
gcloud auth login user@example.com
```

Option B: Service Account Authentication

For automated processes or production environments, use a service account:

1. Create and download a service account key file from the Google Cloud Console
2. Set the environment variable:

```
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your/service-account-key.json"
```

3. Authenticate using the service account:

```
gcloud auth activate-service-account --key-file="/path/to/your/service-account-key.json"
```

This command will: 1. Open a browser window (or provide a link to copy/paste) 2. Prompt you to sign in with your Google account 3. Generate an authentication code 4. Return the code to paste back into the terminal

Step 3: Install GCS FUSE

Add the Google Cloud packages repository:

```
curl -fsSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | gpg --dearmor | sudo  
echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] http://packages.cloud.google.com/apt cloud-gcp-archive main" > /etc/apt/sources.list.d/cloud-gcp-archive.list
```

Update package list and install GCS FUSE:

```
sudo apt-get update  
sudo apt-get install -y gcsfuse
```

Step 4: Create Mount Point

Create a directory where you want to mount the bucket:

```
mkdir -p ~/gcs_mount
```

Step 5: Authenticate for GCS FUSE

Choose the authentication method that matches what you used in Step 2:

For User Authentication:

```
gcloud auth application-default login
```

For Service Account Authentication:

If you're using a service account, the `GOOGLE_APPLICATION_CREDENTIALS` environment variable from Step 2 will be automatically used by GCS FUSE.

Step 6: Mount the Bucket

Mount your GCS bucket to the local filesystem:

```
gcsfuse your-bucket-name ~/gcs_mount
```

Example:

```
gcsfuse my-example-bucket ~/gcs_mount
```

Step 7: Verify the Mount

Verify that the bucket is successfully mounted:

```
ls ~/gcs_mount
```

You should see the contents of your bucket.

Method 2: Command-Line Access

Use this method when you need to copy specific files locally before running Visual Layer profiler, or for one-time data transfers.

Step 1: Install Google Cloud SDK

If you haven't already installed it for Method 1, update your system and install required dependencies:

```
sudo apt-get update && sudo apt-get install -y curl unzip python3
```

Download and install the Google Cloud CLI:

```
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-465.0.0-linux-x86_64.tar.gz  
tar -xzf google-cloud-cli-465.0.0-linux-x86_64.tar.gz  
./google-cloud-sdk/install.sh
```

Restart your shell to apply the changes:

```
exec -l $SHELL
```

Step 2: Authenticate with Google Cloud

Choose one of the following authentication methods:

Option A: User Authentication

Authenticate using your Google account:

```
gcloud auth login user@example.com
```

Option B: Service Account Authentication

For automated processes or production environments, use a service account:

1. Create and download a service account key file from the Google Cloud Console
2. Set the environment variable:

```
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your/service-account-key.json"
```

3. Authenticate using the service account:

```
gcloud auth activate-service-account --key-file="/path/to/your/service-account-key.json"
```

Step 3: Access Bucket Contents

Once authenticated, you can list and access bucket contents using gsutil:

```
gsutil ls gs://your-bucket-name/
```

Example operations:

```
# List all files in a bucket
```

```
gsutil ls gs://my-example-bucket/
```

```
# Copy files from bucket to local machine
```

```
gsutil cp gs://my-example-bucket/data.zip ./
```

```
# Copy files from local machine to bucket
```

```
gsutil cp ./local-file.txt gs://my-example-bucket/
```

```
# Sync directories
```

```
gsutil rsync -r ./local-folder gs://my-example-bucket/remote-folder
```

Integration with Visual Layer

Running Visual Layer Profiler

After setting up access to your GCS bucket, you can run Visual Layer's profiler using either method:

Option A: Using Mounted Bucket (Recommended)

With the bucket mounted as a local filesystem, run the profiler directly on the mounted directory:

```
# Ensure bucket is mounted
```

```
gcsfuse my-example-bucket ~/gcs_mount
```

```
# Run Visual Layer profiler on the mounted folder
```

```
./run_profiler.sh --input ~/gcs_mount/dataset-folder/
```

Option B: Using Command-Line Copied Data

First copy the data locally using gsutil, then run the profiler:

```
# Copy data from bucket to local directory
gsutil -m cp -r gs://my-example-bucket/dataset-folder/ ~/local-dataset/

# Run Visual Layer profiler on the local folder
./run_profiler.sh --input ~/local-dataset/
```

Configuration Options

When using GCS buckets with Visual Layer on-premise installations, you can:

1. **Use mounted buckets** for seamless file system access (recommended)
2. **Copy data locally** using gsutil commands for faster processing
3. **Configure bucket URLs** directly in Visual Layer dataset creation

For Visual Layer specific configuration, refer to the S3 Configuration documentation for similar patterns that can be adapted for GCS.

Need Help?

For additional support: - Contact Visual Layer support: support@visual-layer.com - Refer to Google Cloud Storage documentation - Check GCS FUSE documentation for advanced configuration options

Creating & Updating Datasets

Upload Data

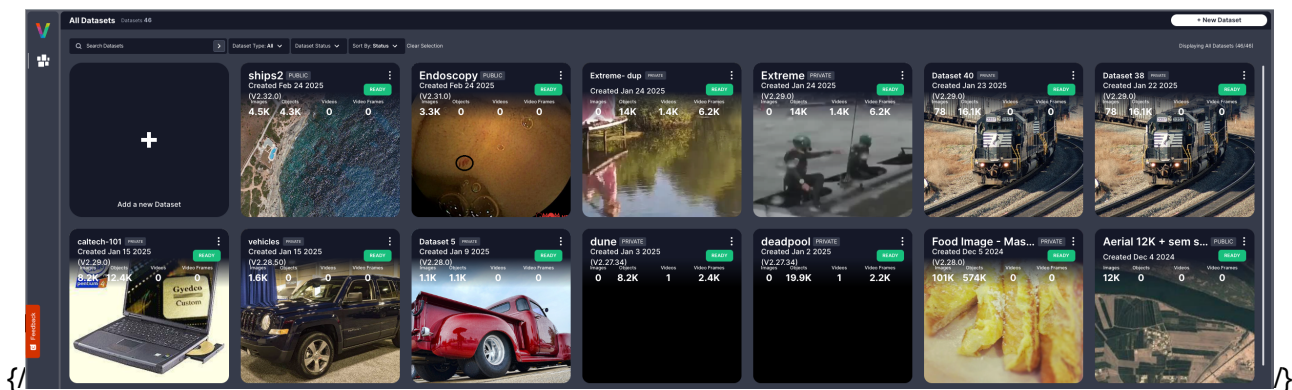
Create datasets in Visual Layer by uploading your own files, folders, or linking to external storage sources. Follow the steps below to configure your dataset, preview your upload, and (optionally) add annotations.

[Video content]

1. Create a New Dataset

To get started:

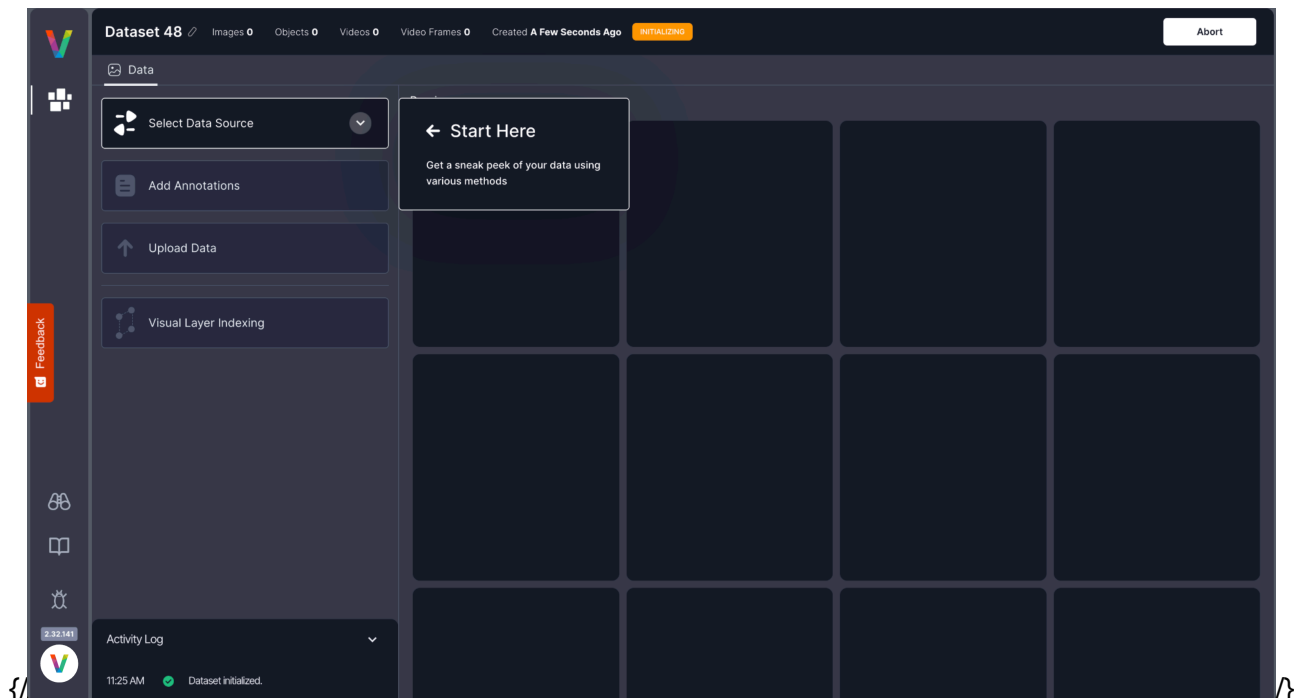
1. Navigate to the **Dataset Inventory** page.
2. Click **+** icon or **New Dataset**.



Name Your Dataset

You can enter a custom name or use the name auto-suggested by Visual Layer.

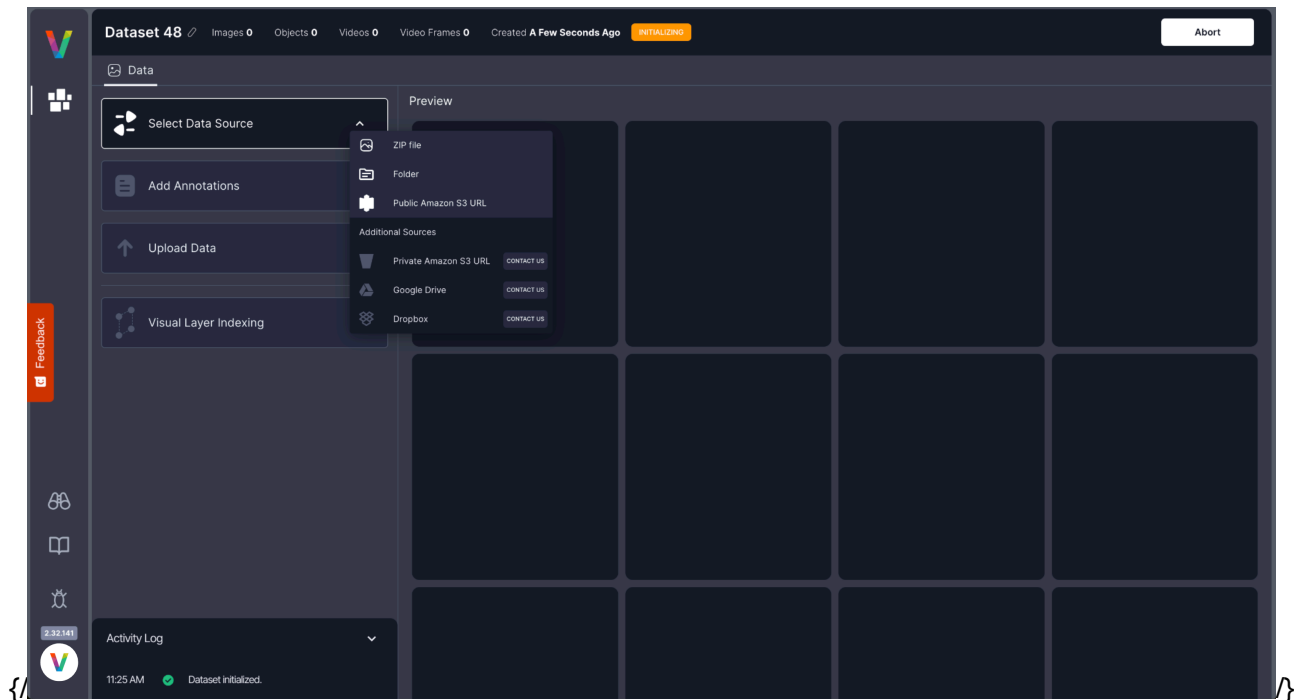
Info: Avoid using duplicate names.



Select Data Source

Choose where to upload your data from. Visual Layer supports the following data source types:

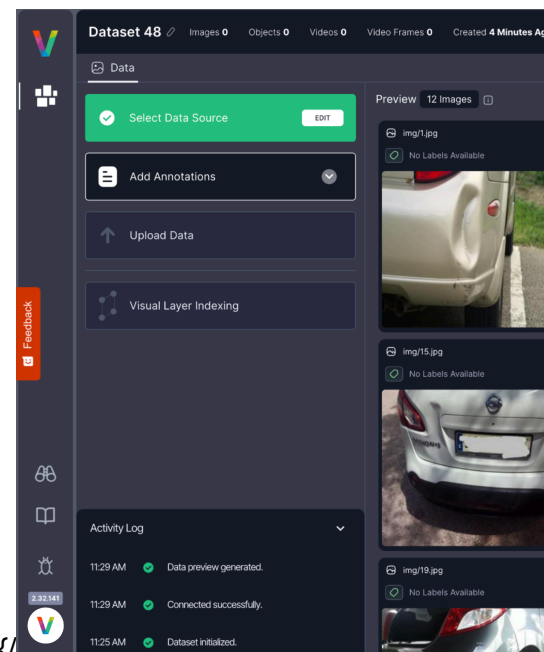
- ZIP file
- Files or Folders
- Public Amazon S3
- Private Amazon S3



Tip: Need another data source? Contact support@visual-layer.com or use the **Contact Us** form.

Preview Your Data

Once uploaded, Visual Layer generates a preview so you can validate your files.



- To replace the selected source, click **Edit** and follow the prompts. {

2. Add Annotations (Optional)

You can upload an annotation file at this stage to enrich your dataset.

1. Upload the annotation file and click **Next**.
2. A new preview will show the dataset with annotations applied.
3. Click **Edit** if you need to replace the file.

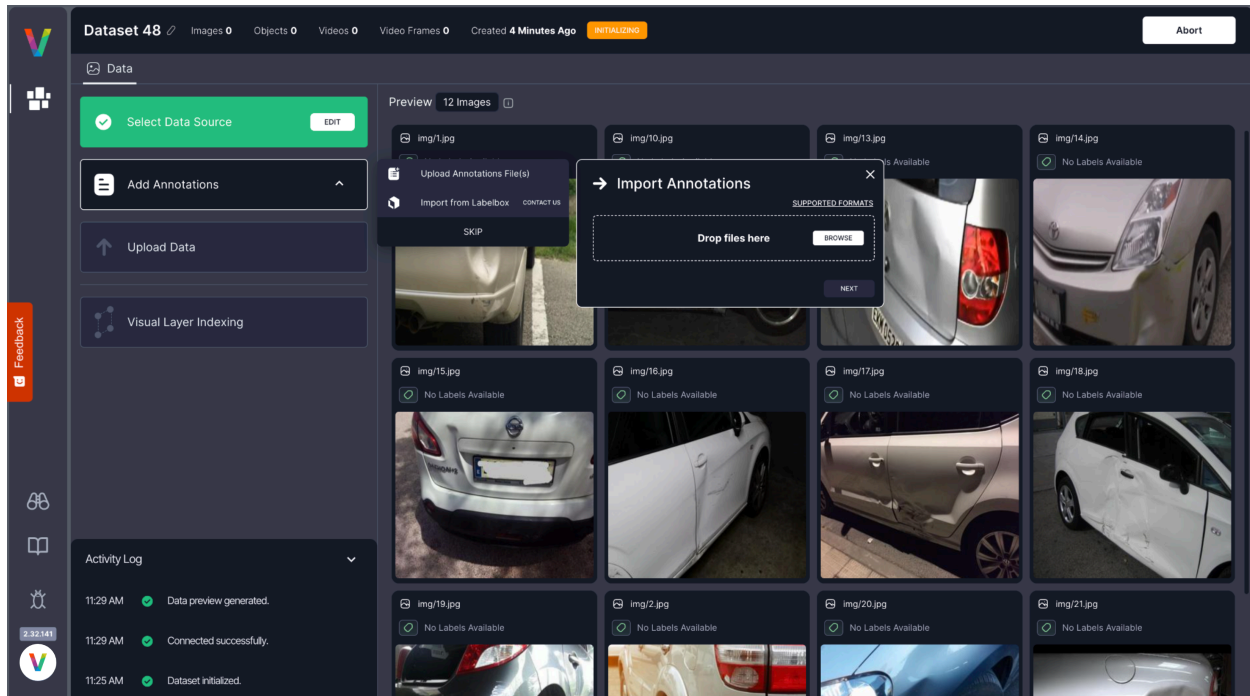


Figure 1: Annotation preview

Skip this step if you don't have annotation files.
For supported formats, see Upload Annotation.

3. Upload and Index

After confirming your data and annotations (if applicable):

- Click **Next** to start the upload and indexing process.
- Uploads are subject to your current plan's limits.
- You'll receive an email when the dataset is ready.

4. Delete a Dataset

To remove an existing dataset:

1. Go to **Dataset Inventory**.

2. Click the menu next to the dataset.
3. Select **Delete**.

Info: Deleting a dataset is irreversible. { /* [Image: [https://files.readme.io/c1f37f6fa067084f90efe3253ca59f0](https://files.readme.io/c1f37f6fa067084f90efe3253ca59f0image.png) */ }

Custom Metadata

How this Helps Add custom metadata like temperature, timestamps, and tools to images via the API. This will help you improve dataset filtering and searchability.

This guide explains how to import custom metadata to your datasets via the API. Each metadata field is user-defined and supports one of several data types.

Custom metadata is associated at the image level and is immutable once uploaded.

To import custom metadata in Visual Layer, you must: - Declare a metadata field (name + type). - Upload a JSON file containing values for that field. > **Tip:** All API requests in the Cloud environment require authentication using a valid API token.

Visual Layer supports these value types:

- string.
- float.
- datetime (UTC format).
- enum (single or multi-select) - each enum option is limited to 20 characters.
- link (URL).

Note: Each upload **task is used to add one metadata field only**. If you want to add multiple fields, you'll need to create a separate task for each - and **make sure each field has a unique field_name**. # Steps

Pre-processing

1. Ensure your dataset is in a READY state.
2. Prepare a JSON file as described below.

Uploading the Metadata

1. Declare a new metadata attribute and receive a task_id.
2. Upload metadata values in a JSON file.
3. Track the ingestion progress via the status endpoint.
4. Metadata becomes available for filtering and retrieval.

Preparing the JSON File

To upload custom metadata, you'll need to associate values with the correct media items in your dataset.

Each entry must include:

- A media_id – a unique identifier generated by Visual Layer for each image
- A value – the metadata value for that image

How to Get media_ids:

1. **Export your dataset** from the Visual Layer platform
2. The exported JSON file will include all image-level metadata, including the media_id assigned to each media item
3. Use these media_ids to match your custom metadata with the corresponding images

Your custom metadata JSON file should follow this structure:

```
[
  { "media_id": "f3e3d00c-1a8e-4a58-85dc-cdea7a8365a0", "value": "Sunny" },
  { "media_id": "b1ac12e7-53dc-4c7c-b93d-1836b4aa4f95", "value": "Cloudy" }
]
```

Automation Option

For users with CSV metadata files, we provide an example automation script that handles the complete upload workflow:

Automation Script Skip the manual JSON creation process with our Python script that automatically maps CSV data to Visual Layer's media_ids and handles the API workflow.

Step 1: Declare a New Metadata Field

Each declaration creates a new upload task for a **single field**. You can repeat the process to add more fields—just ensure each `field_name` is unique within the dataset.

```
POST /api/v1/datasets/{dataset_id}/custom_metadata/tasks
```

Request Body

For `string`, `float`, `datetime`, `link`

```
{
  "field_name": "Your_field_name",
  "field_type": "string" // or - float, datetime, link
}
```

For `enum`

`is_multi = false`

```
{
  "field_name": "priority",
  "field_type": "enum",
  "enum_options": ["Low", "Medium", "High"],
  "is_multi": false
}
```

`is_multi = true`

```
{
  "field_name": "multi_colors",
  "field_type": "enum",
  "enum_options": [
    "red", "green", "blue", "yellow", "purple"
  ],
  "is_multi": true
}
```

The `is_multi` flag determines whether each image can have one or multiple values from the enum options.

- `is_multi: false` (Each image can have only one value.)
- `is_multi: true` (Each image can have multiple values.)

Response Body:

```
{  
  "task_id": "1234e567-e89b-12d3-a456-426614174000",  
  "status": "INIT"  
}
```

Tip: Make sure to save the `task_id`- you'll need it for the next steps in the flow.

Step 2: Upload Metadata File

Use the received `task_id` to upload your metadata file. Each entry must include a valid `media_id` and a `value` matching the declared field type.

```
POST /api/v1/datasets/{dataset_id}/custom_metadata/tasks/{task_id}
```

Upload Format

For string, float, datetime, link

```
[
  { "media_id": "f3e3d00c-1a8e-4a58-85dc-cdea7a8365a0", "value": 23.5 },
  { "media_id": "b1ac12e7-53dc-4c7c-b93d-1836b4aa4f95", "value": 24.0 }
]
```

For enum - when `is_multi = false`

```
[
  {
    "media_id": "2283963d-6c78-442e-9606-352ab1c30e64",
    "value": "yellow"
  }
]
```

For enum - when `is_multi = true`

```
[
  {
    "media_id": "346ba239-5422-475c-aba3-afffc984c13a",
    "value": [
      "black",
      "yellow",
      "blue",
      "pink"
    ]
  }
]
```



```
}  
]
```

Step 3: Check Upload Status

Track progress and get visibility into errors (e.g., type mismatches).

```
GET /api/v1/datasets/{dataset_id}/custom_metadata/tasks/{task_id}/status
```

Response Example:

```
{
  "task_id": "...",
  "status": "COMPLETED",
  "progress": 100.0,
  "inserted_rows": 6535,
  "error_count": 0,
  "sample_errors": []
}
```

Error Response Example

```
{
  "task_id": "...",
  "status": "COMPLETED_WITH_ERRORS",
  "progress": 100.0,
  "inserted_rows": 6535,
  "error_count": 594,
  "sample_errors": [
    { "reason": "Expected String", "row_index": 17 }
  ]
}
```

Note: At this stage, uploaded metadata is immutable - meaning it cannot be edited or replaced once submitted.

If any errors occur during upload, only the problematic rows will fail; all valid rows will proceed successfully.

You can always declare a new `field_name` and reupload the data.

Support for updating existing metadata will be added in the future.

Step 4: Retrieve Metadata

List Declared Metadata Fields

```
GET /api/v1/datasets/{dataset_id}/custom_metadata/schema
```

Returns all declared fields and types.

Get Metadata for an Image

```
GET /api/v1/datasets/{dataset_id}/image/{image_id}/custom_metadata
```

Response:

```
[
  {
    "field_name": "lighting",
    "field_type": "enum",
    "value": "Sunny"
  },
  {
    "field_name": "temperature",
    "field_type": "float",
    "value": 23.5
  }
]
```

Step 5: Search and Filter Using Custom Metadata

Once your custom metadata is uploaded successfully, the declared `field_name` will automatically appear in the **Filters** section of the UI.

You can then filter dataset images using these fields — just like any other built-in filter.

Supported filter types:

- **String** - substring match
- **Float** — numeric comparisons ($>$, $<$, $=$)
- **Datetime** — range queries (BETWEEN `start_date` AND `end_date`)
- **Enum** — exact match (single or multi-select, depending on configuration)
- **Link** - substring or exact match

Tip: Filtering behavior may vary depending on the field type and the selected operator.

Notes

- Only supported at the **image level** (not object level—yet!).
- Metadata is **immutable** and cannot be updated or replaced.
- Each field must have a **unique name** per dataset.

Custom Metadata Upload Script

What This Helps With Automate custom metadata uploads from CSV files to Visual Layer datasets with comprehensive validation, multiple field type support, and detailed error reporting.

This page provides a production-ready Python script that demonstrates how to upload custom metadata from CSV files to Visual Layer. The script validates data before upload, supports all six field types, and provides clear error messages for troubleshooting.

Note: This script demonstrates the complete Visual Layer custom metadata API workflow. You can modify the script to suit your specific needs and data formats.

Working with DICOM Medical Imaging Data? For medical imaging workflows, check out our specialized DICOM converter and upload scripts that handle DICOM-specific field types, date/time formats, and medical metadata standards.

View Complete Script Code Access the full custom metadata upload Python script with complete implementation, ready to copy and use.

Features

The script provides a comprehensive solution for CSV-based metadata uploads:

- **Multiple Field Types** - Supports all six Visual Layer field types: string, float, datetime, enum, multi-enum, and link
- **Comprehensive Validation** - Validates all data before upload with detailed error messages showing row and column information
- **Enum Limit Checking** - Pre-upload validation ensures enum fields don't exceed the 600 unique values limit
- **Flexible Datetime Formats** - Accepts multiple datetime formats and auto-converts to ISO 8601 UTC
- **Clear Progress Tracking** - Detailed status reporting throughout the upload workflow
- **Auto-Detection** - Automatically discovers enum options from your CSV data

How the Script Works

The script follows the complete Visual Layer custom metadata API workflow:

1. **Export dataset** - Calls Visual Layer API to get filename-to-media_id mapping
2. **Read and validate CSV** - Validates all values match field type requirements before upload
3. **Check enum constraints** - Verifies enum fields don't exceed 600 unique values limit
4. **Create custom metadata fields** - Creates each field in Visual Layer via API calls
5. **Upload metadata values** - Uploads validated data for each field using proper JSON format
6. **Monitor upload progress** - Tracks completion status and reports results

Prerequisites

Before using the script, ensure you have:

1. **Visual Layer installation** (cloud or on-premises)
 - **Cloud:** Requires API key and secret (see Authentication Guide)
 - **On-Premises:** No authentication needed
2. **Dataset in READY state** in Visual Layer with images already uploaded
3. **Python environment** with required packages
4. **CSV file** formatted according to requirements below

Install Dependencies

```
pip install requests pandas pyjwt
```

Note: **PyJWT** is required for cloud installations that use JWT authentication. On-premises installations without authentication can omit this package.

CSV Requirements

The CSV file must follow this structure to work with the script.

Required Column

Your CSV must include a `file_fullpath` column that matches the format from Visual Layer's export:

```
file_fullpath,patient_id,score,created_at,status,tags,pdf_url
images/img1.jpg,P001,95.5,2024-01-15,active,"tag1, tag2",https://example.com/doc.pdf
images/img2.jpg,P002,87.2,2024-01-16,pending,"tag3, tag4",https://example.com/doc2.pdf
```

The `file_fullpath` values are used to map your metadata to Visual Layer `media_ids`.

Optional Metadata Columns

You can include any number of metadata columns. Column names become field names in Visual Layer. Specify the type for each column using command-line arguments.

How to Get `file_fullpath` Values

To get the correct `file_fullpath` values for your dataset:

1. Export your dataset from Visual Layer using the `media_id` export endpoint
2. The export returns a CSV with `filename` and `media_id` columns
3. Use the `filename` values as your `file_fullpath` values in your metadata CSV

Supported Field Types

The script supports all six Visual Layer custom metadata field types.

String (--string-fields)

Any text value.

```
file_fullpath,patient_id,notes  
images/img1.jpg,P001,High quality sample
```

Example: patient_id, notes, description

Float (--float-fields)

Decimal numbers.

```
file_fullpath,score,confidence  
images/img1.jpg,95.5,0.98
```

Format: 95.5, 0.98, 123.456

Example: score, confidence, temperature

Datetime (--datetime-fields)

Date and time values in UTC. The script accepts multiple formats and converts to ISO 8601 UTC:

Accepted formats: - 2024-01-15T10:30:00Z (ISO 8601 UTC - preferred) - 2024-01-15 (date only - converts to midnight UTC) - 2024-01-15 10:30:00 (auto-converts to UTC)

Uploaded as: YYYY-MM-DDTHH:MM:SSZ

```
file_fullpath,created_at,scan_date  
images/img1.jpg,2024-01-15T10:30:00Z,2024-01-15
```

Example: created_at, updated_at, scan_date

Enum (--enum-fields)

Single-select categorical data with a maximum of 600 unique values per field.

```
file_fullpath,status,category  
images/img1.jpg,active,approved  
images/img2.jpg,pending,review
```

The script automatically collects unique values from your CSV and validates the count before upload.

Example: status, category, priority

Multi-Enum (--multi-enum-fields)

Multi-select categorical data with a maximum of 600 unique values total across all lists.

Accepts two formats: - Comma-separated: "tag1, tag2, tag3" - JSON array: ["tag1", "tag2", "tag3"]

```
file_fullpath,tags
images/img1.jpg,"tag1, tag2, tag3"
images/img2.jpg,"[""tag4"", ""tag5""]"
```

The script validates the total unique count before upload.

Example: tags, labels, keywords

Link (--link-fields)

URL values that must include a protocol.

```
file_fullpath,pdf_url,documentation_link
images/img1.jpg,https://example.com/doc.pdf,https://docs.example.com
```

Must include protocol: http:// or https://

Example: pdf_url, documentation_link, image_url

Usage

Save the script as `custom_metadata_upload_script.py` and run with field type specifications.

Basic Example

```
# Cloud installation (requires authentication)
python custom_metadata_upload_script.py \
  --csv metadata.csv \
  --dataset-id YOUR-DATASET-ID \
  --base-url https://app.visual-layer.com \
  --api-key YOUR-API-KEY \
  --api-secret YOUR-API-SECRET \
  --string-fields patient_id notes \
  --float-fields score confidence \
  --datetime-fields created_at updated_at \
  --enum-fields status category \
  --multi-enum-fields tags labels \
  --link-fields pdf_url

# On-premises installation (no authentication needed)
python custom_metadata_upload_script.py \
  --csv metadata.csv \
  --dataset-id YOUR-DATASET-ID \
  --base-url http://localhost:8080 \
  --string-fields patient_id notes \
  --float-fields score confidence \
  --datetime-fields created_at updated_at \
  --enum-fields status category \
```



```
--multi-enum-fields tags labels \
--link-fields pdf_url
```

Command-Line Parameters

Parameter	Required	Description	Default
--csv	☐	Path to CSV file with metadata	-
--dataset-id	☐	Visual Layer dataset identifier	-
--base-url	☐	Visual Layer installation URL (use https://app.visual-layer.com for cloud)	http://localhost:8080
--api-key	☐	API key for Visual Layer Cloud authentication	-
--api-secret	☐	API secret for Visual Layer Cloud authentication	-
--string-fields	☐	Space-separated list of string field names	-
--float-fields	☐	Space-separated list of float field names	-
--datetime-fields	☐	Space-separated list of datetime field names	-
--enum-fields	☐	Space-separated list of enum field names	-
--multi-enum-fields	☐	Space-separated list of multi-enum field names	-
--link-fields	☐	Space-separated list of link field names	-
--batch-size	☐	Records per upload batch	50000

Note: Cloud vs On-Premises: - **Cloud** (<https://app.visual-layer.com>): Requires --api-key and --api-secret for JWT authentication - **On-Premises:** No authentication needed, omit --api-key and --api-secret parameters

Tip: At least one field type must be specified. The script will return an error if no field types are provided.

Example Use Cases

The following examples demonstrate common metadata upload scenarios.

Medical Imaging

Cloud example

```
python custom_metadata_upload_script.py \  
  --csv medical_metadata.csv \  
  --dataset-id abc-123 \  
  --base-url https://app.visual-layer.com \  
  --api-key YOUR-API-KEY \  
  --api-secret YOUR-API-SECRET \  
  --string-fields patient_id doctor_notes diagnosis \  
  --float-fields confidence_score tumor_size_mm \  
  --datetime-fields scan_date report_date \  
  --enum-fields scan_type organ priority \  
  --multi-enum-fields findings abnormalities \  
  --link-fields dicom_url report_pdf
```

Note: For on-premises installations, omit `--api-key` and `--api-secret` and change `--base-url` to your local URL (e.g., `http://localhost:8080`).

CSV format:

file_fullpath,patient_id,scan_date,scan_type,confidence_score,findings,report_pdf
scans/chest/scan001.dcm,P12345,2024-01-15,CT,0.95,"nodule, calcification",https://repo

E-commerce Products

Cloud example

```
python custom_metadata_upload_script.py \  
  --csv product_metadata.csv \  
  --dataset-id xyz-789 \  
  --base-url https://app.visual-layer.com \  
  --api-key YOUR-API-KEY \  
  --api-secret YOUR-API-SECRET \  
  --string-fields product_id product_name sku \  
  --float-fields price rating \  
  --datetime-fields created_at last_updated \  
  --enum-fields category status availability \  
  --multi-enum-fields tags colors sizes \  
  --link-fields product_url manual_pdf
```

Document Management

Cloud example

```
python custom_metadata_upload_script.py \  
  --csv document_metadata.csv \  
  --dataset-id doc-456 \  
  --base-url https://app.visual-layer.com \  
  --api-key YOUR-API-KEY \  
  --api-secret YOUR-API-SECRET
```

```
--api-secret YOUR-API-SECRET \  
--string-fields document_id title author \  
--datetime-fields created_at modified_at \  
--enum-fields document_type status department \  
--multi-enum-fields keywords topics \  
--link-fields pdf_url source_url
```

Authentication (Cloud Only)

Visual Layer Cloud requires JWT authentication for all API requests. On-premises installations can skip this section.

On-Premises Installations If you're using an on-premises deployment, you can skip authentication parameters. The script works without `--api-key` and `--api-secret` arguments.

Get Your API Credentials

To use the script with Visual Layer Cloud:

1. **Obtain API Key and Secret** - Follow the instructions in the Authentication Guide to retrieve your API credentials from Visual Layer Cloud
2. **Pass credentials to script** - Use the `--api-key` and `--api-secret` parameters when running the script

The script automatically generates JWT tokens and includes them in API requests when credentials are provided.

Tip: JWT tokens are short-lived (10 minutes). The script generates a fresh token at the start of execution, which is sufficient for most uploads. For very large datasets that take longer than 10 minutes, the upload may fail with authentication errors. Contact Visual Layer support for guidance on long-running uploads.

Authentication Example

```
# Cloud installation with authentication  
python custom_metadata_upload_script.py \  
--csv metadata.csv \  
--dataset-id YOUR-DATASET-ID \  
--base-url https://app.visual-layer.com \  
--api-key YOUR-API-KEY \  
--api-secret YOUR-API-SECRET \  
--string-fields patient_id notes \  
--float-fields score confidence
```

For complete details on obtaining and testing JWT tokens, see the Authentication Guide.

Validation and Error Handling

The script validates all data before uploading to catch issues early.

Pre-Upload Validation Checks

The script performs these validations before any upload:

- Float values are valid numbers
- Datetime values match accepted formats
- Enum fields don't exceed 600 unique values
- Multi-enum fields don't exceed 600 unique values total
- Multi-enum values are properly formatted (comma-separated or JSON array)
- Links include protocol (http:// or https://)
- File paths exist in Visual Layer dataset

Error Messages

The script provides clear error messages showing exactly what's wrong:

- ```
❑ Validation Errors (3 total):
 • Row 5, field 'score': Invalid float value 'abc'. Must be a valid number.
 • Row 8, field 'created_at': Invalid datetime format '01-15-2024'. Expected ISO 8601 format: YYYY-MM-DDTHH:MM:SSZ or YYYY-MM-DD

❑ Enum Constraint Errors (1 total):
 • Field 'status' (enum): Has 750 unique values, exceeds limit of 600
```

Each error includes the row number and field name for easy troubleshooting in your CSV.

## Script Output

The script provides detailed progress information throughout the workflow.

### Successful Upload Example

```
=====
❑ Generic CSV Metadata Upload to Visual Layer
=====
❑ CSV File: metadata.csv
❑ Dataset ID: abc-123
❑ Base URL: http://localhost:8080
=====

❑ Exporting dataset to get media_id mappings...
 ❑ Exported 150 media items

❑ Reading and validating CSV file: metadata.csv
```

```

[] Loaded 150 rows
[] Checking enum field constraints...
 [] status (enum): 3 unique values
 [] tags (multi-enum): 12 unique values
[] Validating 8 metadata fields...

[] Field Configuration:
 • annotated_at: datetime
 • category: enum
 • confidence: float
 • notes: string
 • patient_id: string
 • pdf_url: link
 • status: enum
 • tags: multi-enum

[] Validated 150 rows successfully

[] Creating and uploading 8 custom fields...

[] Processing: annotated_at (datetime)
 [] Creating field: annotated_at (datetime)
 [] Created (task_id: task-123)
 [] Uploading 150 values
 [] Upload completed
 [] Completed (150 rows)

[] Processing: confidence (float)
 [] Creating field: confidence (float)
 [] Created (task_id: task-124)
 [] Uploading 150 values
 [] Upload completed
 [] Completed (150 rows)

...

=====
[] Workflow Completed!
=====
[] Successfully uploaded: 8/8 fields
[] Total rows processed: 150
=====

```

## Tips and Best Practices

Follow these best practices for successful metadata uploads.

## Test with Sample Data First

Start with a small CSV (10-20 rows) to verify: - Field types are correct - Data formats are valid - Mappings work as expected

## Use Datetime Flexibility

The script accepts multiple datetime formats:

```
created_at
2024-01-15 # □ Converts to 2024-01-15T00:00:00Z
2024-01-15T10:30:00Z # □ Already in correct format
2024-01-15 10:30:00 # □ Converts to UTC
```

## Multi-Enum Format Options

Both formats work equally well:

```
tags
"tag1, tag2, tag3" # □ Comma-separated
["tag1", "tag2", "tag3"] # □ JSON array
```

## Handle Empty Values

Empty values are automatically skipped. There is no need to remove rows with missing data.

## Use Descriptive Field Names

Column names become field names in Visual Layer. Make them descriptive and consistent: - □ Good: patient\_id, scan\_date, confidence\_score - □ Avoid: field1, data, x

## Troubleshooting

Common issues and solutions for metadata upload problems.

### “No field types specified”

**Problem:** Script returns error about missing field types.

**Solution:** Add at least one `--*-fields` argument to specify which columns to upload and their types.

### “Failed to export dataset”

**Problem:** Script cannot export dataset to get media\_id mappings.

**Solution:** - Check that the dataset ID is correct - Verify the base URL is accessible - Ensure Visual Layer is running - Confirm the dataset is in READY state

## “Validation failed with N errors”

**Problem:** CSV data doesn't match specified field types.

**Solution:** - Read error messages carefully (shows row and column) - Fix data format issues in CSV - Verify field type assignments match your data - Check that datetime values use accepted formats

## “Enum field exceeds 600 unique values limit”

**Problem:** Enum or multi-enum field has too many unique values.

**Solution:** - Use `string` field type instead of `enum` or `multi-enum` - Reduce the number of unique values in your data - Check unique count with: `cut -d',' -f2 file.csv | sort | uniq | wc -l`

## “Field already exists”

**Problem:** Script reports that a field was created previously.

**Solution:** This is normal behavior, not an error. The script will skip creation and proceed with upload. This occurs when re-running the script or adding data to existing fields.

**Note: Adding vs. Updating Metadata:** - **Adding new fields:** The script will create new fields if they don't exist - **Updating existing fields:** Currently updating existing metadata values is not supported. Contact Visual Layer for assistance with updating existing metadata values

## Getting Help

For advanced use cases, custom metadata deletion, or technical support:

**Contact Visual Layer Support** Reach out to Visual Layer's support team for assistance with complex metadata workflows, custom field requirements, or troubleshooting upload issues.

## Expected Result in Visual Layer

After running the script successfully, your custom metadata fields will be visible in Visual Layer's interface for each image.

[Image: /images/custom-metadata-example.png]

These fields become searchable and filterable, enabling you to query and analyze your dataset based on your custom metadata.

## Related Resources

- Custom Metadata API Guide - Manual API workflow and field type details
- DICOM Converter - Specialized workflow for medical imaging metadata
- Creating Datasets - Guide to creating Visual Layer datasets

## Custom Metadata Upload Script Code

**Custom Metadata Upload Script** This page contains the complete, ready-to-use Python script for uploading custom metadata from CSV files to Visual Layer datasets with comprehensive validation and error handling.

This script demonstrates how to upload custom metadata to Visual Layer from CSV files. The script handles the complete workflow: exports dataset mappings, validates CSV data, creates metadata fields, uploads values, and monitors progress.

**Back to Custom Metadata Upload Documentation** Return to the main custom metadata upload guide for setup instructions, usage examples, and workflow details.

## Key Features

- **CSV-Based Workflow** - Reads metadata from CSV files with `file_fullpath` column
- **Field Type Specification** - Supports all six Visual Layer field types via command-line arguments
- **Comprehensive Validation** - Validates all data before upload with detailed error messages
- **Enum Limit Checking** - Pre-upload validation for 600 unique values constraint
- **Flexible Datetime** - Accepts multiple datetime formats and converts to ISO 8601 UTC
- **Progress Tracking** - Detailed status reporting throughout the workflow

## Installation Requirements

Before using this script, install the required Python packages:

```
For cloud installations (requires PyJWT for authentication)
pip install requests pandas pyjwt
```

```
For on-premises installations (PyJWT optional)
pip install requests pandas
```

## Usage

Save this script as `custom_metadata_upload_script.py` and run:

```
For cloud installations (with authentication)
python custom_metadata_upload_script.py \
 --csv metadata.csv \
 --dataset-id your-dataset-id \
 --base-url https://app.visual-layer.com \
 --api-key YOUR-API-KEY \
 --api-secret YOUR-API-SECRET \
 --string-fields patient_id notes \
 --float-fields score confidence \
 --datetime-fields created_at updated_at \
```



```

--enum-fields status category \
--multi-enum-fields tags labels \
--link-fields pdf_url

For on-premises installations (no authentication)
python custom_metadata_upload_script.py \
 --csv metadata.csv \
 --dataset-id your-dataset-id \
 --base-url http://localhost:8080 \
 --string-fields patient_id notes \
 --float-fields score confidence \
 --datetime-fields created_at updated_at \
 --enum-fields status category \
 --multi-enum-fields tags labels \
 --link-fields pdf_url

```

## CSV Format

The CSV file must include a `file_fullpath` column matching Visual Layer export format:

```

file_fullpath,patient_id,score,created_at,status,tags,pdf_url
images/img1.jpg,P001,95.5,2024-01-15,active,"tag1, tag2",https://example.com/doc.pdf
images/img2.jpg,P002,87.2,2024-01-16,pending,"tag3, tag4",https://example.com/doc2.pdf

```

**Note:** This script is designed for Visual Layer installations (cloud or on-premises). You can modify the validation logic and field detection to suit your specific needs.

## Complete Script Code

```

#!/usr/bin/env python3
"""
Generic CSV-to-Visual-Layer Metadata Upload Script
Reads a CSV file and uploads metadata to Visual Layer with configurable field types.

Requirements:
 pip install requests pandas

Usage:
 python custom_metadata_upload_script.py \
 --csv metadata.csv \
 --dataset-id <dataset-id> \
 --base-url http://localhost:8080 \
 --string-fields patient_id notes \
 --float-fields score confidence \
 --datetime-fields created_at updated_at \
 --enum-fields status category \

```

```

 --multi-enum-fields tags labels \
 --link-fields pdf_url image_url

Documentation:
 https://docs.visual-layer.com/docs/Creating-Datasets/custom-metadata
"""

from typing import Dict, List, Any, Optional, Set
from datetime import datetime, timezone, timedelta
from urllib.parse import urlparse

class ValidationError(Exception):
 """Custom exception for validation errors."""
 pass

class CSVMetadataUploader:
 """Upload custom metadata from CSV to Visual Layer with field type validation."""

 # Maximum unique values for enum fields per Visual Layer documentation
 MAX_ENUM_VALUES = 600

 def __init__(self, dataset_id: str, base_url: str,
 string_fields: List[str] = None,
 float_fields: List[str] = None,
 datetime_fields: List[str] = None,
 enum_fields: List[str] = None,
 multi_enum_fields: List[str] = None,
 link_fields: List[str] = None,
 batch_size: int = 50000,
 api_key: str = None,
 api_secret: str = None):
 """
 Initialize uploader with field type specifications.

 Args:
 dataset_id: Visual Layer dataset ID
 base_url: Base URL for Visual Layer API
 string_fields: List of field names to treat as strings
 float_fields: List of field names to treat as floats
 datetime_fields: List of field names to treat as datetimes (ISO 8601 UTC)
 enum_fields: List of field names to treat as enums (single-select)
 multi_enum_fields: List of field names to treat as multi-enums
 link_fields: List of field names to treat as links (URLs)
 batch_size: Number of records per batch (default: 50000)

```

```

 api_key: API key for Visual Layer Cloud authentication (optional)
 api_secret: API secret for Visual Layer Cloud authentication (optional)
 """
 self.dataset_id = dataset_id
 self.raw_base_url = base_url.rstrip('/')
 self.batch_size = batch_size

 # Build field type mappings
 self.field_types = {}
 for field in (string_fields or []):
 self.field_types[field] = 'string'
 for field in (float_fields or []):
 self.field_types[field] = 'float'
 for field in (datetime_fields or []):
 self.field_types[field] = 'datetime'
 for field in (enum_fields or []):
 self.field_types[field] = 'enum'
 for field in (multi_enum_fields or []):
 self.field_types[field] = 'multi-enum'
 for field in (link_fields or []):
 self.field_types[field] = 'link'

 # Add /api/v1/datasets to base URL if not present
 if not base_url.endswith('/api/v1/datasets'):
 self.base_url = f"{base_url.rstrip('/')}/api/v1/datasets"
 else:
 self.base_url = base_url

 self.session = requests.Session()
 self._temp_files = []

 # Add authentication headers if provided
 if api_key and api_secret:
 jwt_token = self._generate_jwt(api_key, api_secret)
 self.session.headers.update({
 'Authorization': f'Bearer {jwt_token}'
 })

def _generate_jwt(self, api_key: str, api_secret: str) -> str:
 """Generate JWT token for Visual Layer API authentication."""
 jwt_algorithm = "HS256"
 jwt_header = {
 'alg': jwt_algorithm,
 'typ': 'JWT',
 'kid': api_key,
 }

```

```

now = datetime.now(tz=timezone.utc)
expiration = now + timedelta(minutes=10)

payload = {
 'sub': api_key,
 'iat': int(now.timestamp()),
 'exp': int(expiration.timestamp()),
 'iss': 'sdk'
}

return jwt.encode(payload=payload, key=api_secret, algorithm=jwt_algorithm, he

def export_dataset(self) -> Dict[str, str]:
 """Export dataset and return mapping of filename -> media_id."""
 print("[] Exporting dataset to get media_id mappings...")

 url = f"{self.raw_base_url}/api/v1/dataset/{self.dataset_id}/export_media_id"

 try:
 response = self.session.get(url)
 if response.status_code == 200:
 csv_content = response.text
 csv_reader = csv.DictReader(csv_content.splitlines())

 mapping = {}
 for row in csv_reader:
 filename = row.get('filename', '')
 media_id = row.get('media_id', '')
 if media_id and filename:
 mapping[filename] = media_id

 print(f"[] Exported {len(mapping)} media items")
 return mapping
 else:
 print(f"[] Failed to export dataset: {response.status_code} - {resp
 return {}
 except Exception as e:
 print(f"[] Export failed: {str(e)}")
 return {}

def validate_float(self, value: str, field_name: str, row_num: int) -> float:
 """Validate and convert float value."""
 try:
 return float(value)
 except (ValueError, TypeError):
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Invalid float value '{value}'."

```

```

 f"Must be a valid number."
)

def validate_datetime(self, value: str, field_name: str, row_num: int) -> str:
 """
 Validate datetime format (ISO 8601 UTC).

 Accepts formats:
 - YYYY-MM-DDTHH:MM:SSZ (e.g., 2024-01-15T10:30:00Z)
 - YYYY-MM-DD (will be converted to YYYY-MM-DDT00:00:00Z)
 """
 if not value or not value.strip():
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Empty datetime value"
)

 value = value.strip()

 # Try to parse common datetime formats
 formats_to_try = [
 '%Y-%m-%dT%H:%M:%SZ', # 2024-01-15T10:30:00Z
 '%Y-%m-%dT%H:%M:%S', # 2024-01-15T10:30:00
 '%Y-%m-%d %H:%M:%S', # 2024-01-15 10:30:00
 '%Y-%m-%d', # 2024-01-15
]

 for fmt in formats_to_try:
 try:
 dt = datetime.strptime(value, fmt)
 # Always return ISO 8601 UTC format
 return dt.strftime('%Y-%m-%dT%H:%M:%SZ')
 except ValueError:
 continue

 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Invalid datetime format '{value}'.\n"
 f"Expected ISO 8601 format: YYYY-MM-DDTHH:MM:SSZ or YYYY-MM-DD"
)

def validate_enum(self, value: str, field_name: str, row_num: int) -> str:
 """Validate enum value."""
 if not value or not value.strip():
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Empty enum value"
)

 value = value.strip()

```

```

 return value

def validate_multi_enum(self, value: str, field_name: str, row_num: int) -> List[str]:
 """
 Validate multi-enum value (list of strings).

 Accepts formats:
 - Comma-separated: "tag1, tag2, tag3"
 - JSON array: ["tag1", "tag2", "tag3"]
 """
 if not value or not value.strip():
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Empty multi-enum value"
)

 value = value.strip()

 # Try to parse as JSON array first
 if value.startswith '[' and value.endswith(']'):
 try:
 parsed = json.loads(value)
 if not isinstance(parsed, list):
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Multi-enum must be a list"
)
 values = [str(v).strip() for v in parsed]
 except json.JSONDecodeError:
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Invalid JSON array format"
)
 else:
 # Parse as comma-separated values
 values = [v.strip() for v in value.split(',')]

 # Validate each value
 for v in values:
 if not v:
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Empty value in multi-enum list"
)

 return values

def validate_link(self, value: str, field_name: str, row_num: int) -> str:
 """Validate URL format for link field."""
 if not value or not value.strip():
 raise ValidationError(

```

```

 f"Row {row_num}, field '{field_name}': Empty link value"
)

value = value.strip()

Parse URL
try:
 parsed = urlparse(value)
 # Check if it has a scheme (http, https, etc.)
 if not parsed.scheme:
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Invalid URL '{value}'. "
 f"Must include protocol (e.g., http:// or https://)"
)
 return value
except Exception:
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Invalid URL format '{value}'"
)

def validate_and_convert_value(self, value: str, field_name: str,
 field_type: str, row_num: int) -> Any:
 """Validate and convert value based on field type."""
 # Skip empty values
 if not value or (isinstance(value, str) and not value.strip()):
 return None

 try:
 if field_type == 'float':
 return self.validate_float(value, field_name, row_num)
 elif field_type == 'datetime':
 return self.validate_datetime(value, field_name, row_num)
 elif field_type == 'enum':
 return self.validate_enum(value, field_name, row_num)
 elif field_type == 'multi-enum':
 return self.validate_multi_enum(value, field_name, row_num)
 elif field_type == 'link':
 return self.validate_link(value, field_name, row_num)
 elif field_type == 'string':
 return str(value).strip()
 else:
 return str(value).strip()
 except ValidationError:
 raise
 except Exception as e:
 raise ValidationError(
 f"Row {row_num}, field '{field_name}': Unexpected validation error: {s

```

```

)

def read_csv(self, csv_path: str, filename_to_media_id: Dict[str, str]) -> Dict[str, str]:
 """
 Read CSV file and return validated metadata dictionary.

 Returns:
 Dict mapping file_fullpath -> metadata
 """
 print(f" Reading and validating CSV file: {csv_path}")

 if not self.field_types:
 raise ValueError(
 "No field types specified. Please provide at least one field type using\n"
 "--string-fields, --float-fields, --datetime-fields, --enum-fields, "\n"
 "--multi-enum-fields, or --link-fields"
)

 try:
 # Load CSV with pandas for analysis
 df = pd.read_csv(csv_path)

 print(f" Loaded {len(df)} rows")

 # Check enum/multi-enum fields for unique value count
 print(f" Checking enum field constraints...")
 enum_errors = []

 for field_name, field_type in self.field_types.items():
 if field_type in ['enum', 'multi-enum']:
 if field_name not in df.columns:
 enum_errors.append(f"Field '{field_name}' not found in CSV columns")
 continue

 if field_type == 'enum':
 # For enum, count unique values directly
 unique_count = df[field_name].nunique()
 if unique_count > self.MAX_ENUM_VALUES:
 enum_errors.append(
 f"Field '{field_name}' (enum): Has {unique_count} unique values\n"
 f"exceeds limit of {self.MAX_ENUM_VALUES}"
)
 else:
 print(f" {field_name} (enum): {unique_count} unique values")

 elif field_type == 'multi-enum':
 # For multi-enum, collect all unique values across all lists

```



```

 unique_values = set()
 for value in df[field_name].dropna():
 if isinstance(value, str):
 # Parse the value
 if value.strip().startswith('['):
 # JSON array format
 try:
 parsed = json.loads(value.strip())
 if isinstance(parsed, list):
 unique_values.update(str(v).strip() for v in parsed)
 except json.JSONDecodeError:
 pass
 else:
 # Comma-separated format
 unique_values.update(v.strip() for v in value.split(','))

 unique_count = len(unique_values)
 if unique_count > self.MAX_ENUM_VALUES:
 enum_errors.append(
 f"Field '{field_name}' (multi-enum): Has {unique_count} unique values across all lists, exceeds limit of {self.MAX_ENUM_VALUES}"
)
 else:
 print(f" ✓ {field_name} (multi-enum): {unique_count} unique values")

 if enum_errors:
 print(f"\n ✗ Enum Constraint Errors ({len(enum_errors)} total):")
 for error in enum_errors:
 print(f" • {error}")
 print()
 raise ValidationError(
 f"Enum fields exceed the {self.MAX_ENUM_VALUES} unique values limit. Please reduce unique values or use 'string' field type instead."
)

 print(f" ✓ Validating {len(self.field_types)} metadata fields...")

 # Print field type configuration
 print(f"\n ✓ Field Configuration:")
 for field_name, field_type in sorted(self.field_types.items()):
 print(f" • {field_name}: {field_type}")
 print()

 # Convert back to list of dicts for row-by-row processing
 rows = df.to_dict('records')

except ValidationError:

```

```

 raise
 except Exception as e:
 print(f" Failed to read CSV: {str(e)}")
 return {}

 metadata_by_path = {}
 skipped_count = 0
 validation_errors = []

 for row_num, row in enumerate(rows, start=2): # Start at 2 (header is row 1)
 # Get file_fullpath for mapping
 file_fullpath = row.get('file_fullpath', '')
 if not file_fullpath:
 skipped_count += 1
 validation_errors.append(f"Row {row_num}: Missing 'file_fullpath' column")
 continue

 # Skip if not in Visual Layer dataset
 if file_fullpath not in filename_to_media_id:
 skipped_count += 1
 continue

 # Validate and build metadata dictionary
 flattened = {}
 for field_name, field_type in self.field_types.items():
 value = row.get(field_name, '')

 try:
 validated_value = self.validate_and_convert_value(
 value, field_name, field_type, row_num
)

 # Only add non-None values
 if validated_value is not None:
 flattened[field_name] = validated_value

 except ValidationError as e:
 validation_errors.append(str(e))
 # Stop at first validation error for this row
 break

 # Only add if we have at least one metadata field
 if flattened:
 metadata_by_path[file_fullpath] = flattened

 # Report validation errors
 if validation_errors:

```

```

 print(f"\n Validation Errors ({len(validation_errors)} total):")
 # Show first 10 errors
 for error in validation_errors[:10]:
 print(f" • {error}")
 if len(validation_errors) > 10:
 print(f" • ... and {len(validation_errors) - 10} more errors")
 print()
 raise ValidationError(f"CSV validation failed with {len(validation_errors)} errors")

 if skipped_count > 0:
 print(f" Skipped {skipped_count} rows not in Visual Layer dataset")

 print(f" Validated {len(metadata_by_path)} rows successfully\n")
 return metadata_by_path

def create_custom_field(self, field_name: str, field_type: str,
 metadata_by_path: Dict[str, Dict[str, Any]]) -> Optional[str]:
 """Create a custom field and return field_id (task_id)."""
 print(f" Creating field: {field_name} ({field_type})")

 field_data = {
 "field_name": field_name,
 "field_type": field_type
 }

 # For enum/multi-enum fields, collect unique values from the data
 if field_type in ['enum', 'multi-enum']:
 unique_values = set()
 for metadata in metadata_by_path.values():
 if field_name in metadata:
 value = metadata[field_name]
 if field_type == 'multi-enum' and isinstance(value, list):
 unique_values.update(value)
 elif value and isinstance(value, str):
 unique_values.add(value)

 enum_options = sorted(list(unique_values))
 field_data["enum_options"] = enum_options

 if field_type == 'multi-enum':
 field_data["field_type"] = 'enum' # API expects 'enum' for both
 field_data["is_multi"] = True
 else:
 field_data["is_multi"] = False

 print(f" Enum options ({len(enum_options)}): {enum_options[:10]}{'...' if len(enum_options) > 10 else ''}")

```

```

url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks"

try:
 response = self.session.post(url, json=field_data)
 if response.status_code == 200:
 result = response.json()
 task_id = result.get('task_id')
 print(f" Created (task_id: {task_id})")
 return task_id
 elif "already exists" in response.text:
 print(f" Field already exists, skipping")
 return None
 else:
 print(f" Failed: {response.status_code} - {response.text}")
 return None
except Exception as e:
 print(f" Request failed: {str(e)}")
 return None

def upload_field_data(self, field_id: str, field_name: str, field_type: str,
 metadata_by_path: Dict[str, Dict[str, Any]],
 filename_to_media_id: Dict[str, str]) -> Optional[str]:
 """Upload data for a custom field in a single request."""
 upload_data = []

 for file_fullpath, metadata in metadata_by_path.items():
 media_id = filename_to_media_id.get(file_fullpath)
 if not media_id or field_name not in metadata:
 continue

 value = metadata[field_name]
 if value is None or value == '':
 continue

 upload_data.append({
 "media_id": media_id,
 "value": value
 })

 if not upload_data:
 print(f" No data to upload")
 return None

 print(f" Uploading {len(upload_data)} values")

 url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks/{field_id}"

```

```

Save data to temp file
with tempfile.NamedTemporaryFile(mode='w', suffix='.json', delete=False) as f:
 json.dump(upload_data, f, indent=2)
 temp_file = f.name

self._temp_files.append(temp_file)

Upload
try:
 with open(temp_file, 'rb') as f:
 files = {'file': ('metadata.json', f, 'application/json')}
 response = self.session.post(url, files=files)

 if response.status_code not in [200, 202]:
 print(f" Upload failed: {response.status_code} - {response.text}")
 return None

 print(f" Upload completed")
 return field_id

except Exception as e:
 print(f" Upload failed: {str(e)}")
 return None

def wait_for_task_completion(self, task_id: str, field_name: str,
 polling_interval: int = 5) -> str:
 """Wait for task completion by polling status endpoint."""
 url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks/{task_id}/status"

 while True:
 try:
 response = self.session.get(url)
 if response.status_code == 200:
 result = response.json()
 status = result.get('status', 'unknown')

 if status == 'COMPLETED':
 inserted_rows = result.get('inserted_rows', 0)
 print(f" Completed ({inserted_rows} rows)")
 return 'COMPLETED'
 elif status == 'COMPLETED_WITH_ERRORS':
 error_count = result.get('error_count', 0)
 inserted_rows = result.get('inserted_rows', 0)
 print(f" Completed with {error_count} errors ({inserted_rows} rows)")
 return 'COMPLETED_WITH_ERRORS'
 elif status == 'IN_PROGRESS':
 time.sleep(polling_interval)

```

```

 continue
 else:
 time.sleep(polling_interval)
 continue
 else:
 print(f" Status check failed")
 return 'error'
except Exception:
 return 'error'

def cleanup_temp_files(self):
 """Remove temporary files."""
 for temp_file in self._temp_files:
 try:
 if os.path.exists(temp_file):
 os.remove(temp_file)
 except OSError:
 pass

def process_workflow(self, csv_path: str):
 """Main workflow: export → read CSV → validate → create fields → upload data."""
 try:
 print("=" * 70)
 print(" Generic CSV Metadata Upload to Visual Layer")
 print("=" * 70)
 print(f" CSV File: {csv_path}")
 print(f" Dataset ID: {self.dataset_id}")
 print(f" Base URL: {self.raw_base_url}")
 print("=" * 70)
 print()

 # Step 1: Export dataset to get media_id mappings
 filename_to_media_id = self.export_dataset()
 if not filename_to_media_id:
 raise Exception("Failed to export dataset")
 print()

 # Step 2: Read CSV, validate, and build metadata
 metadata_by_path = self.read_csv(csv_path, filename_to_media_id)
 if not metadata_by_path:
 raise Exception("No valid rows found in CSV after validation")

 # Step 3: Process each field
 print(f" Creating and uploading {len(self.field_types)} custom fields...")
 print()

 completed_count = 0

```

```

failed_fields = []

for field_name, field_type in self.field_types.items():
 print(f" Processing: {field_name} ({field_type})")

 try:
 # Create field
 field_id = self.create_custom_field(field_name, field_type, metadata)
 if not field_id:
 print(f" Skipping (field exists or creation failed)\n")
 continue

 # Upload data
 task_id = self.upload_field_data(field_id, field_name, field_type,
 metadata_by_path, filename_to_media)

 if not task_id:
 failed_fields.append(field_name)
 print(f" Upload failed\n")
 continue

 # Wait for completion
 status = self.wait_for_task_completion(task_id, field_name)
 if status in ['COMPLETED', 'COMPLETED_WITH_ERRORS']:
 completed_count += 1
 else:
 failed_fields.append(field_name)

 print() # Blank line between fields

 except Exception as e:
 print(f" Error: {str(e)}\n")
 failed_fields.append(field_name)
 continue

Final summary
print("=" * 70)
print(" Workflow Completed!")
print("=" * 70)
print(f" Successfully uploaded: {completed_count}/{len(self.field_types)}")
print(f" Total rows processed: {len(metadata_by_path)}")

if failed_fields:
 print(f" Failed fields ({len(failed_fields)}): {' '.join(failed_fields)}")

print("=" * 70)

except ValidationError as e:

```

```

 print(f"\n Validation failed: {str(e)}")
 sys.exit(1)
 except Exception as e:
 print(f"\n Workflow failed: {str(e)}")
 sys.exit(1)
 finally:
 self.cleanup_temp_files()

def main():
 parser = argparse.ArgumentParser(
 description='Generic CSV metadata upload to Visual Layer with field type validation',
 formatter_class=argparse.RawDescriptionHelpFormatter,
 epilog="""
Example usage:

Basic upload with different field types
python custom_metadata_upload_script.py \\\
--csv metadata.csv \\\
--dataset-id abc-123 \\\
--base-url http://localhost:8080 \\\
--string-fields patient_id notes description \\\
--float-fields score confidence temperature \\\
--datetime-fields created_at updated_at \\\
--enum-fields status category priority \\\
--multi-enum-fields tags labels keywords \\\
--link-fields pdf_url documentation_link

Simple upload with just strings and enums
python custom_metadata_upload_script.py \\\
--csv data.csv \\\
--dataset-id abc-123 \\\
--base-url http://localhost:8080 \\\
--string-fields name description \\\
--enum-fields category status

CSV Format Requirements:
- Must have a 'file_fullpath' column matching Visual Layer export format
- File paths should match those returned by the dataset export API
- Datetime values: YYYY-MM-DDTHH:MM:SSZ or YYYY-MM-DD
- Float values: Valid decimal numbers
- Enum values: Max 20 characters each
- Multi-enum values: Comma-separated or JSON array ["val1", "val2"]
- Link values: Must include protocol (http:// or https://)

Field Type Documentation:
https://docs.visual-layer.com/docs/Creating-Datasets/custom-metadata

```



```

)

parser.add_argument('--csv', required=True,
 help='Path to CSV file with metadata')
parser.add_argument('--dataset-id', required=True,
 help='Visual Layer dataset ID')
parser.add_argument('--base-url', default='http://localhost:8080',
 help='Base URL (default: http://localhost:8080)')
parser.add_argument('--batch-size', type=int, default=50000,
 help='Records per upload batch (default: 50000)')
parser.add_argument('--api-key', default=None,
 help='API key for Visual Layer Cloud authentication')
parser.add_argument('--api-secret', default=None,
 help='API secret for Visual Layer Cloud authentication')

Field type specifications
parser.add_argument('--string-fields', nargs='*', default=[],
 help='List of fields to treat as strings')
parser.add_argument('--float-fields', nargs='*', default=[],
 help='List of fields to treat as floats (decimal numbers)')
parser.add_argument('--datetime-fields', nargs='*', default=[],
 help='List of fields to treat as datetimes (ISO 8601 UTC format)')
parser.add_argument('--enum-fields', nargs='*', default=[],
 help='List of fields to treat as enums (single-select, max 20 c)')
parser.add_argument('--multi-enum-fields', nargs='*', default=[],
 help='List of fields to treat as multi-enums (multi-select, max)')
parser.add_argument('--link-fields', nargs='*', default=[],
 help='List of fields to treat as links (URLs)')

args = parser.parse_args()

Validate that at least one field type is specified
if not any([args.string_fields, args.float_fields, args.datetime_fields,
 args.enum_fields, args.multi_enum_fields, args.link_fields]):
 parser.error(
 "At least one field type must be specified. Use --string-fields, --float-f
 "--datetime-fields, --enum-fields, --multi-enum-fields, or --link-fields"
)

Create uploader and run workflow
uploader = CSVMetadataUploader(
 dataset_id=args.dataset_id,
 base_url=args.base_url,
 string_fields=args.string_fields,
 float_fields=args.float_fields,
 datetime_fields=args.datetime_fields,

```

```
 enum_fields=args.enum_fields,
 multi_enum_fields=args.multi_enum_fields,
 link_fields=args.link_fields,
 batch_size=args.batch_size,
 api_key=args.api_key,
 api_secret=args.api_secret
)

 uploader.process_workflow(args.csv)

if __name__ == "__main__":
 main()
```

## DICOM Converter Script

**Custom Metadata Integration Example** This DICOM workflow demonstrates how to extract domain-specific metadata and upload it as Visual Layer custom fields, enabling powerful search and filtering capabilities. This pattern can be adapted for any specialized data type.

The DICOM converter script extracts medical imaging metadata (patient info, study details, equipment parameters) from DICOM files and converts them to searchable custom metadata fields in Visual Layer. Once uploaded, you can search and filter your dataset by any DICOM field.

**Note:** This example shows how custom metadata transforms basic image browsing into domain-specific dataset analysis. The same pattern applies to e-commerce, manufacturing, research, or any field with structured metadata.

## Complete DICOM-to-Visual Layer Workflow

This section shows the complete end-to-end process for converting DICOM files and creating a fully searchable Visual Layer dataset with comprehensive medical imaging metadata.

### Step: Extract images and metadata from DICOM files

Use the DICOM converter script to convert DICOM files to JPG images and extract comprehensive

### Step: Create Visual Layer dataset

Upload the converted JPG images to Visual Layer to create a new dataset for analysis and export

### Step: Export dataset for media\_id mapping

Export your Visual Layer dataset to obtain the critical mapping between filenames and Visual

### Step: Upload DICOM metadata as custom fields

Use the DICOM metadata upload script to automatically process and upload the extracted meta

### Step: Search and filter by DICOM metadata

Use Visual Layer to search and filter your medical imaging dataset by patient demographics,

## Step 1: Convert DICOM Files

Run the DICOM converter script to extract images and metadata. See Basic Usage section below for command examples and output details.

## Step 2: Create Visual Layer Dataset

Upload the converted JPG images from converted\_images/ to Visual Layer using the web interface or API.

## Step 3: Export Dataset Metadata

Export your dataset to get metadata.json containing the filename-to-media\_id mapping required for the next step.

## Step 4: Upload DICOM Metadata

Use the DICOM upload script to process and upload your extracted metadata:

### For Cloud:

```
python upload_csv_with_json_mapping.py dicom_metadata.csv metadata.json \
 --dataset-id=your-dataset-id \
 --base-url=https://app.visual-layer.com \
 --api-key=your-api-key \
 --api-secret=your-api-secret
```

### For On-premises:

```
python upload_csv_with_json_mapping.py dicom_metadata.csv metadata.json \
 --dataset-id=your-dataset-id \
 --base-url=http://localhost:2080 \
 --api-key=your-api-key
```

## Step 5: Search and Filter by DICOM Metadata

[Image: /images/dicom-custom-metadata-example.png]

## Why Custom Metadata Matters

This DICOM example shows how custom metadata transforms your Visual Layer dataset:

**Before custom metadata:** - Browse images visually - Basic filename-based search

**After custom metadata:** - Filter by patient demographics (age, sex) - Search by equipment manufacturer and modality - Query by technical parameters (slice thickness, kVp) - Group by study dates or series descriptions

**Apply this pattern to your domain:** - **E-commerce:** Product SKU, price, category metadata for inventory analysis - **Manufacturing:** Quality metrics, batch numbers, inspection data for QA workflows - **Research:** Experiment conditions, sample IDs, measurements for dataset curation

## DICOM Converter Script

### Prerequisites

1. **Python environment** with required packages
2. **DICOM files** organized in series directories or as individual files
3. **Sufficient disk space** for converted images

### Installation

```
pip install pandas pydicom dicom2jpg opencv-python numpy
```

## Script Code

**View Complete Script Code** Access the full DICOM converter Python script with complete implementation.

## Basic Usage

```
Convert DICOM files
python dicom_converter.py /path/to/dicom/files

Custom output directory
python dicom_converter.py /path/to/dicom/files --output /custom/output
```

The script creates `converted_images/` and `metadata/dicom_metadata.csv` with comprehensive DICOM metadata including file info, patient data, study details, equipment parameters, and acquisition settings.

## DICOM Metadata Upload Script

Visual Layer provides a customizable script for uploading DICOM metadata extracted by the converter. This script handles DICOM-specific field types and can be modified for your specific use case.

**Note:** Similar to the general custom metadata upload script but includes DICOM-specific field handling (date/time formats, multi-value fields).

## Script Features

- Auto-detects DICOM date/time formats (YYYYMMDD, HHMMSS)
- Handles multi-value fields and complex data types
- Supports cloud and on-premises installations
- Customizable for vendor-specific DICOM tags

## Script Code

**View Complete Script Code** Access the full DICOM metadata upload Python script with complete implementation.

The script automatically analyzes DICOM fields and assigns appropriate types (string, datetime, float, enum, multi-enum) based on DICOM standards.

## Troubleshooting

**No DICOM files found:** - Verify input directory contains `.dcm` files and check file permissions

**Conversion failures:** - Check verbose output for specific errors - Verify DICOM files can be opened with standard viewers

**Upload issues:** - Ensure dataset ID is correct and dataset is in READY state - Use `--resume` flag to continue interrupted uploads

## Adapting This Pattern

The DICOM scripts can be customized for your specific use case:

- **Modify field extraction** to capture vendor-specific tags
- **Add preprocessing** for specialized protocols
- **Extend to other formats** by adapting the extraction logic

## Related Resources

- Custom Metadata API Guide - Manual API workflow
- Custom Metadata Upload Script - General pattern for other data types
- Creating Datasets - Dataset creation guide

## DICOM Converter Script Code

**DICOM to JPG Converter** This page contains the complete, ready-to-use Python script for converting DICOM files to JPG images while extracting comprehensive medical imaging metadata.

This script converts DICOM (Digital Imaging and Communications in Medicine) files to JPG images and extracts comprehensive metadata to CSV format. It handles both single-frame and multiframe DICOM files and supports various medical imaging modalities.

**Back to DICOM Converter Documentation** Return to the main DICOM converter guide for setup instructions, usage examples, and the complete workflow.

## Installation Requirements

Before using this script, install the required Python packages:

```
pip install pandas pydicom dicom2jpg opencv-python numpy
```

## Usage

Save this script as `dicom_converter.py` and run:

```
Basic usage
python dicom_converter.py /path/to/dicom/files

With custom output directory
python dicom_converter.py /path/to/dicom/files --output /custom/output/path

With verbose logging
python dicom_converter.py /path/to/dicom/files --verbose
```

## Complete Script Code

```
#!/usr/bin/env python3
"""
DICOM to JPG Converter with Metadata Extraction

This script converts DICOM files to JPG images and extracts metadata,
creating a CSV file that links each converted image to its metadata.
"""

from pathlib import Path

from datetime import datetime
```

```

Logging will be configured by setup_logging() function

def extract_dicom_metadata(dicom_path):
 """Extract comprehensive metadata from a DICOM file."""
 try:
 ds = pydicom.dcmread(dicom_path)

 # Create metadata dictionary
 metadata = {}

 # Patient Information (anonymized)
 metadata['patient_id'] = getattr(ds, 'PatientID', '')
 metadata['patient_name'] = str(getattr(ds, 'PatientName', ''))
 metadata['patient_birth_date'] = getattr(ds, 'PatientBirthDate', '')
 metadata['patient_sex'] = getattr(ds, 'PatientSex', '')
 metadata['patient_age'] = getattr(ds, 'PatientAge', '')

 # Study Information
 metadata['study_instance_uid'] = getattr(ds, 'StudyInstanceUID', '')
 metadata['study_date'] = getattr(ds, 'StudyDate', '')
 metadata['study_time'] = getattr(ds, 'StudyTime', '')
 metadata['study_description'] = getattr(ds, 'StudyDescription', '')
 metadata['accession_number'] = getattr(ds, 'AccessionNumber', '')

 # Series Information
 metadata['series_instance_uid'] = getattr(ds, 'SeriesInstanceUID', '')
 metadata['series_number'] = getattr(ds, 'SeriesNumber', '')
 metadata['series_date'] = getattr(ds, 'SeriesDate', '')
 metadata['series_time'] = getattr(ds, 'SeriesTime', '')
 metadata['series_description'] = getattr(ds, 'SeriesDescription', '')

 # Instance Information
 metadata['sop_instance_uid'] = getattr(ds, 'SOPInstanceUID', '')
 metadata['instance_number'] = getattr(ds, 'InstanceNumber', '')
 metadata['instance_creation_date'] = getattr(ds, 'InstanceCreationDate', '')
 metadata['instance_creation_time'] = getattr(ds, 'InstanceCreationTime', '')

 # Equipment Information
 metadata['manufacturer'] = getattr(ds, 'Manufacturer', '')
 metadata['manufacturer_model_name'] = getattr(ds, 'ManufacturerModelName', '')
 metadata['device_serial_number'] = getattr(ds, 'DeviceSerialNumber', '')
 metadata['software_versions'] = getattr(ds, 'SoftwareVersions', '')
 metadata['modality'] = getattr(ds, 'Modality', '')

 # Image Acquisition Parameters
 metadata['image_type'] = str(getattr(ds, 'ImageType', ''))
 metadata['acquisition_date'] = getattr(ds, 'AcquisitionDate', '')

```



```

metadata['acquisition_time'] = getattr(ds, 'AcquisitionTime', '')
metadata['slice_thickness'] = getattr(ds, 'SliceThickness', '')
metadata['slice_location'] = getattr(ds, 'SliceLocation', '')
metadata['image_position_patient'] = str(getattr(ds, 'ImagePositionPatient', ''))
metadata['image_orientation_patient'] = str(getattr(ds, 'ImageOrientationPatient', ''))

CT-specific parameters
if hasattr(ds, 'KVP'):
 metadata['kvp'] = ds.KVP
if hasattr(ds, 'XRayTubeCurrent'):
 metadata['tube_current'] = ds.XRayTubeCurrent
if hasattr(ds, 'ExposureTime'):
 metadata['exposure_time'] = ds.ExposureTime
if hasattr(ds, 'ConvolutionKernel'):
 metadata['convolution_kernel'] = ds.ConvolutionKernel

Image Display Parameters
metadata['window_center'] = getattr(ds, 'WindowCenter', '')
metadata['window_width'] = getattr(ds, 'WindowWidth', '')
metadata['rescale_intercept'] = getattr(ds, 'RescaleIntercept', '')
metadata['rescale_slope'] = getattr(ds, 'RescaleSlope', '')

Image Pixel Data Information
metadata['rows'] = getattr(ds, 'Rows', '')
metadata['columns'] = getattr(ds, 'Columns', '')
metadata['pixel_spacing'] = str(getattr(ds, 'PixelSpacing', ''))
metadata['bits_allocated'] = getattr(ds, 'BitsAllocated', '')
metadata['bits_stored'] = getattr(ds, 'BitsStored', '')
metadata['high_bit'] = getattr(ds, 'HighBit', '')
metadata['pixel_representation'] = getattr(ds, 'PixelRepresentation', '')
metadata['photometric_interpretation'] = getattr(ds, 'PhotometricInterpretation', '')

Multiframe-specific information
metadata['number_of_frames'] = getattr(ds, 'NumberOfFrames', '')
metadata['frame_increment_pointer'] = str(getattr(ds, 'FrameIncrementPointer', ''))
metadata['frame_time'] = getattr(ds, 'FrameTime', '')
metadata['frame_time_vector'] = str(getattr(ds, 'FrameTimeVector', ''))

Enhanced timing information
metadata['repetition_time'] = getattr(ds, 'RepetitionTime', '')
metadata['echo_time'] = getattr(ds, 'EchoTime', '')

return metadata

except Exception as e:
 logging.error(f"Error extracting metadata from {dicom_path}: {str(e)}")
 return {}

```

```

def is_multiframe_dicom(dicom_path):
 """Check if a DICOM file contains multiframe images."""
 try:
 ds = pydicom.dcmread(dicom_path, stop_before_pixels=False)

 # Check if pixel data exists
 if not hasattr(ds, 'pixel_array'):
 return False

 pixel_array = ds.pixel_array

 # Multiframe detection: 3D array where first dimension is not RGB (not 3)
 # Shape: [frames, height, width] for grayscale multiframe
 # vs [height, width, 3] for RGB single frame
 if len(pixel_array.shape) == 3 and pixel_array.shape[2] != 3:
 return True

 return False

 except Exception as e:
 logging.error(f"Error checking multiframe status for {dicom_path}: {str(e)}")
 return False

def convert_multiframe_dicom(dicom_file, series_dir, images_dir):
 """Convert multiframe DICOM to individual JPG files."""
 try:
 # Read DICOM file
 ds = pydicom.dcmread(dicom_file)
 pixel_array = ds.pixel_array.astype(float)

 # Get number of frames
 num_frames = pixel_array.shape[0]
 logging.info(f"Processing {num_frames} frames from multiframe DICOM: {dicom_file}")

 converted_files = []
 frame_metadata_list = []

 for frame_idx in range(num_frames):
 # Extract individual frame
 frame_data = pixel_array[frame_idx]

 # Apply DICOM processing (windowing, rescaling)
 processed_frame = apply_dicom_processing(ds, frame_data)

 # Generate output filename
 frame_filename = f"{series_dir.name}_{dicom_file.stem}_frame-{frame_idx + 1}.jpg"
 output_path = images_dir / frame_filename

```

```

 # Save as JPG using OpenCV
 success = cv2.imwrite(str(output_path), processed_frame)

 if success:
 logging.info(f"Converted frame {frame_idx + 1}/{num_frames} -> {frame_filename}")
 converted_files.append(frame_filename)

 # Extract metadata for this frame
 frame_metadata = extract_dicom_metadata(dicom_file)
 frame_metadata['image_filename'] = frame_filename
 frame_metadata['is_multiframe'] = True
 frame_metadata['frame_number'] = frame_idx + 1
 frame_metadata['total_frames'] = num_frames
 frame_metadata['original_dicom_path'] = str(dicom_file.relative_to(dicom_dir))
 frame_metadata['series_folder'] = series_dir.name
 frame_metadata['conversion_timestamp'] = datetime.now().isoformat()

 frame_metadata_list.append(frame_metadata)
 else:
 logging.error(f"Failed to save frame {frame_idx + 1} from {dicom_file}")

 return converted_files, frame_metadata_list

except Exception as e:
 logging.error(f"Error processing multiframe DICOM {dicom_file}: {str(e)}")
 return [], []

def apply_dicom_processing(ds, pixel_data):
 """Apply DICOM windowing and rescaling to pixel data."""
 try:
 # Apply rescale slope and intercept if available
 if hasattr(ds, 'RescaleSlope') and hasattr(ds, 'RescaleIntercept'):
 rescale_slope = float(ds.RescaleSlope)
 rescale_intercept = float(ds.RescaleIntercept)
 pixel_data = pixel_data * rescale_slope + rescale_intercept

 # Apply windowing if available
 if hasattr(ds, 'WindowCenter') and hasattr(ds, 'WindowWidth'):
 window_center = ds.WindowCenter
 window_width = ds.WindowWidth

 # Handle multivalue fields
 if hasattr(window_center, '__iter__') and not isinstance(window_center, str):
 window_center = float(window_center[0])
 else:
 window_center = float(window_center)

```

```

 if hasattr(window_width, '__iter__') and not isinstance(window_width, str):
 window_width = float(window_width[0])
 else:
 window_width = float(window_width)

 # Apply windowing
 img_min = window_center - window_width // 2
 img_max = window_center + window_width // 2
 pixel_data = np.clip(pixel_data, img_min, img_max)

 # Normalize to 8-bit range
 pixel_data = ((pixel_data - pixel_data.min()) / (pixel_data.max() - pixel_data.min())) * 255

 # Handle photometric interpretation
 if hasattr(ds, 'PhotometricInterpretation') and ds.PhotometricInterpretation == 'Inverted':
 pixel_data = 255 - pixel_data

 return pixel_data.astype('uint8')

 except Exception as e:
 logging.error(f"Error in DICOM processing: {str(e)}")
 # Fallback: simple normalization
 pixel_data = ((pixel_data - pixel_data.min()) / (pixel_data.max() - pixel_data.min())) * 255
 return pixel_data.astype('uint8')

def validate_input_directory(input_dir):
 """Validate that the input directory exists and contains DICOM files."""
 input_path = Path(input_dir)

 if not input_path.exists():
 logging.error(f"Input directory does not exist: {input_dir}")
 return False

 if not input_path.is_dir():
 logging.error(f"Input path is not a directory: {input_dir}")
 return False

 # Check for DICOM files (either .dcm files or series directories)
 dicom_files = list(input_path.glob("*/*.dcm"))
 series_dirs = [d for d in input_path.iterdir() if d.is_dir() and d.name.startswith('Series')]

 if not dicom_files and not series_dirs:
 logging.error(f"No DICOM files or series directories found in: {input_dir}")
 return False

 logging.info(f"Found {len(dicom_files)} DICOM files and {len(series_dirs)} series directories")
 return True

```

```

def convert_dicom_to_jpg(input_dir, output_dir):
 """Convert DICOM files to JPG and extract metadata."""

 # Create output directories
 images_dir = Path(output_dir) / "converted_images"
 metadata_dir = Path(output_dir) / "metadata"
 images_dir.mkdir(parents=True, exist_ok=True)
 metadata_dir.mkdir(parents=True, exist_ok=True)

 # List to store all metadata
 all_metadata = []

 # Check for organized series directories first
 series_dirs = [d for d in Path(input_dir).iterdir() if d.is_dir() and d.name.startswith('series')]

 # Also check for DICOM files directly in the input directory
 direct_dicom_files = list(Path(input_dir).glob("*.dcm"))

 logging.info(f"Found {len(series_dirs)} series directories and {len(direct_dicom_files)} direct DICOM files")

 # Process series directories first
 for series_dir in sorted(series_dirs):
 logging.info(f"Processing {series_dir.name}")

 # Find all DICOM files in the series
 dicom_files = list(series_dir.glob("*.dcm"))

 if not dicom_files:
 logging.warning(f"No DICOM files found in {series_dir.name}")
 continue

 logging.info(f"Found {len(dicom_files)} DICOM files in {series_dir.name}")

 for dicom_file in sorted(dicom_files):
 try:
 # Check if this is a multiframe DICOM
 if is_multiframe_dicom(dicom_file):
 logging.info(f"Detected multiframe DICOM: {dicom_file.name}")

 # Convert multiframe DICOM to individual frames
 converted_files, frame_metadata_list = convert_multiframe_dicom(dicom_file)

 if converted_files:
 logging.info(f"Successfully converted multiframe DICOM to {len(converted_files)} frames")
 all_metadata.extend(frame_metadata_list)
 else:
 logging.error(f"Failed to convert multiframe DICOM: {dicom_file.name}")
 except Exception as e:
 logging.error(f"Error processing {dicom_file.name}: {e}")

```

```

else:
 # Handle single-frame DICOM using dicom2jpg
 image_filename = f"{series_dir.name}_{dicom_file.stem}.jpg"
 output_image_path = images_dir / image_filename

 # Convert DICOM to JPG using dicom2jpg
 # dicom2jpg creates its own directory structure, so we convert first
 temp_dir = images_dir / "temp"
 temp_dir.mkdir(exist_ok=True)

 logging.info(f"Converting {dicom_file.name} -> {image_filename}")
 dicom2jpg.dicom2jpg(str(dicom_file), str(temp_dir))

 # Find the generated JPG file and rename it
 generated_files = list(temp_dir.rglob("*.jpg"))
 if generated_files:
 generated_file = generated_files[0] # Take the first (should be only one)
 generated_file.rename(output_image_path)
 logging.info(f"Moved generated file to {image_filename}")
 else:
 raise Exception(f"No JPG file generated for {dicom_file.name}")

 # Clean up temp directory structure
 for item in temp_dir.iterdir():
 if item.is_dir():
 shutil.rmtree(item)

 # Extract metadata
 metadata = extract_dicom_metadata(dicom_file)

 # Add file information to metadata
 metadata['image_filename'] = image_filename
 metadata['original_dicom_path'] = str(dicom_file.relative_to(input_dir))
 metadata['series_folder'] = series_dir.name
 metadata['conversion_timestamp'] = datetime.now().isoformat()
 metadata['is_multiframe'] = False
 metadata['frame_number'] = 1
 metadata['total_frames'] = 1

 all_metadata.append(metadata)

except Exception as e:
 logging.error(f"Error processing {dicom_file}: {str(e)}")
 continue

Process direct DICOM files (flat folder structure)
if direct_dicom_files:

```

```

logging.info(f"Processing {len(direct_dicom_files)} direct DICOM files...")

for dicom_file in sorted(direct_dicom_files):
 try:
 # Create a pseudo series_dir for direct files
 pseudo_series_dir = Path("direct")

 # Check if this is a multiframe DICOM
 if is_multiframe_dicom(dicom_file):
 logging.info(f"Detected multiframe DICOM: {dicom_file.name}")

 # Convert multiframe DICOM to individual frames
 converted_files, frame_metadata_list = convert_multiframe_dicom(dicom_file)

 if converted_files:
 logging.info(f"Successfully converted multiframe DICOM to {len(converted_files)} frames")
 all_metadata.extend(frame_metadata_list)
 else:
 logging.error(f"Failed to convert multiframe DICOM: {dicom_file.name}")

 else:
 # Handle single-frame DICOM using dicom2jpg
 image_filename = f"direct_{dicom_file.stem}.jpg"
 output_image_path = images_dir / image_filename

 # Convert DICOM to JPG using dicom2jpg
 # dicom2jpg creates its own directory structure, so we convert first
 temp_dir = images_dir / "temp"
 temp_dir.mkdir(exist_ok=True)

 logging.info(f"Converting {dicom_file.name} -> {image_filename}")
 dicom2jpg.dicom2jpg(str(dicom_file), str(temp_dir))

 # Find the generated JPG file and rename it
 generated_files = list(temp_dir.rglob("*.jpg"))
 if generated_files:
 generated_file = generated_files[0] # Take the first (should be only one)
 generated_file.rename(output_image_path)
 logging.info(f"Moved generated file to {image_filename}")
 else:
 raise Exception(f"No JPG file generated for {dicom_file.name}")

 # Clean up temp directory structure
 for item in temp_dir.iterdir():
 if item.is_dir():
 shutil.rmtree(item)

```

```

 # Extract metadata
 metadata = extract_dicom_metadata(dicom_file)

 # Add file information to metadata
 metadata['image_filename'] = image_filename
 metadata['original_dicom_path'] = str(dicom_file.relative_to(input
 metadata['series_folder'] = "direct" # Mark as direct files
 metadata['conversion_timestamp'] = datetime.now().isoformat()
 metadata['is_multiframe'] = False
 metadata['frame_number'] = 1
 metadata['total_frames'] = 1

 all_metadata.append(metadata)

 except Exception as e:
 logging.error(f"Error processing {dicom_file}: {str(e)}")
 continue

Save metadata to CSV
if all_metadata:
 df = pd.DataFrame(all_metadata)

 # Reorder columns to put image_filename first
 if 'image_filename' in df.columns:
 cols = ['image_filename'] + [col for col in df.columns if col != 'image_filename']
 df = df[cols]

 csv_path = metadata_dir / "dicom_metadata.csv"
 df.to_csv(csv_path, index=False)
 logging.info(f"Metadata saved to {csv_path}")

 logging.info(f"Conversion complete! Processed {len(all_metadata)} files")
 logging.info(f"Images saved to: {images_dir}")
 logging.info(f"Metadata saved to: {csv_path}")

 return df
else:
 logging.error("No files were successfully processed")
 return None

def parse_arguments():
 """Parse command line arguments."""
 parser = argparse.ArgumentParser(
 description='Convert DICOM files to JPG images with metadata extraction',
 formatter_class=argparse.RawDescriptionHelpFormatter,
 epilog="""
Examples:

```



```

%(prog)s /path/to/dicom/folder
%(prog)s /path/to/dicom/folder --output /custom/output/path
%(prog)s /path/to/dicom/folder --format png --verbose
 """

 parser.add_argument('input_dir',
 help='Directory containing DICOM files or series subdirectories')

 parser.add_argument('-o', '--output',
 help='Output directory (default: input_dir/output)')

 parser.add_argument('-f', '--format',
 choices=['jpg', 'png', 'bmp', 'tiff'],
 default='jpg',
 help='Output image format (default: jpg)')

 parser.add_argument('-v', '--verbose',
 action='store_true',
 help='Enable verbose logging')

 parser.add_argument('--series',
 help='Process only specific series (e.g., series-00001)')

 return parser.parse_args()

def setup_logging(verbose=False):
 """Setup logging configuration."""
 log_level = logging.DEBUG if verbose else logging.INFO

 # Clear any existing handlers
 for handler in logging.root.handlers[:]:
 logging.root.removeHandler(handler)

 logging.basicConfig(
 level=log_level,
 format='%(asctime)s - %(levelname)s - %(message)s',
 handlers=[
 logging.StreamHandler() # Only console output, no file
]
)

if __name__ == "__main__":
 # Parse command line arguments
 args = parse_arguments()

 # Setup logging
 setup_logging(args.verbose)

```

```

Validate input directory
if not validate_input_directory(args.input_dir):
 sys.exit(1)

Determine output directory
if args.output:
 output_directory = args.output
else:
 # Create output directory with timestamp inside input directory
 timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
 output_directory = Path(args.input_dir) / f"output_{timestamp}"

Convert to absolute paths
input_directory = Path(args.input_dir).resolve()
output_directory = Path(output_directory).resolve()

Log configuration
logging.info("DICOM to JPG Converter Starting...")
logging.info(f"Input directory: {input_directory}")
logging.info(f"Output directory: {output_directory}")
logging.info(f"Output format: {args.format}")

if args.series:
 logging.info(f"Processing only series: {args.series}")

Run conversion
try:
 result_df = convert_dicom_to_jpg(str(input_directory), str(output_directory))

 if result_df is not None:
 print(f"\n Conversion Complete!")
 print(f" Summary:")
 print(f" • Total files converted: {len(result_df)}")
 print(f" • Unique series processed: {result_df['series_folder'].nunique()}")
 print(f" • Output directory: {output_directory}")
 print(f" • Images directory: {output_directory}/converted_images")
 print(f" • Metadata file: {output_directory}/metadata/dicom_metadata.csv")
 print("\n Sample metadata:")
 print(result_df[['image_filename', 'series_folder', 'modality', 'instance_

 else:
 print("\n Conversion failed. Check the log file for details.")
 sys.exit(1)

except KeyboardInterrupt:
 print("\n Conversion interrupted by user")
 sys.exit(1)
except Exception as e:

```

```
logging.error(f"Unexpected error: {str(e)}")
print(f"␣ Unexpected error: {str(e)}")
sys.exit(1)
```

## DICOM Metadata Upload Script Code

**DICOM Metadata Upload to Visual Layer** This page contains the complete, ready-to-use Python script for uploading extracted DICOM metadata to Visual Layer datasets with automatic field type detection and DICOM-specific handling.

This script uploads DICOM metadata from the CSV file generated by the DICOM converter to Visual Layer. It automatically detects DICOM date/time formats, categorizes field types, and handles multi-value fields specific to medical imaging data.

**Back to DICOM Converter Documentation** Return to the main DICOM converter guide for the complete workflow, setup instructions, and usage examples.

## Installation Requirements

Before using this script, install the required Python packages:

```
pip install pandas requests pyjwt
```

## Usage

Save this script as `upload_csv_with_json_mapping.py` and run:

```
For cloud installations
python upload_csv_with_json_mapping.py dicom_metadata.csv metadata.json \
 --dataset-id=your-dataset-id \
 --base-url=https://app.visual-layer.com \
 --api-key=your-api-key \
 --api-secret=your-api-secret

For on-premises installations
python upload_csv_with_json_mapping.py dicom_metadata.csv metadata.json \
 --dataset-id=your-dataset-id \
 --base-url=http://localhost:2080 \
 --api-key=your-api-key
```

## Complete Script Code

```
#!/usr/bin/env python3
"""
CSV Metadata with JSON Mapping Workflow for Visual Layer
Uploads DICOM metadata from dicom_metadata.csv to Visual Layer.
Uses metadata.json for file path mapping to Visual Layer media IDs.
"""

from typing import Dict, List, Any, Optional
```

```

from datetime import datetime, timezone, timedelta

class CsvJsonMetadataProcessor:
 def __init__(self, dataset_id: str, base_url: str, api_key: str = None, api_secret: str = None):
 self.dataset_id = dataset_id
 # Store raw base URL for backward compatibility
 self.raw_base_url = base_url.rstrip('/')
 # Automatically add /api/v1/datasets if not present
 if not base_url.endswith('/api/v1/datasets'):
 if base_url.endswith('/'):
 base_url = base_url.rstrip('/')
 self.base_url = f"{base_url}/api/v1/datasets"
 else:
 self.base_url = base_url
 self.session = requests.Session()
 self._temp_files = [] # Track temporary files for cleanup
 self.enum_fields = enum_fields or [] # Store user-specified enum fields

 # Add authentication headers if provided
 if api_key and api_secret:
 # Generate JWT token as per Visual Layer documentation
 jwt_token = self._generate_jwt(api_key, api_secret)
 self.session.headers.update({
 'Authorization': f'Bearer {jwt_token}'
 })

 def _generate_jwt(self, api_key: str, api_secret: str) -> str:
 """Generate JWT token for Visual Layer API authentication."""
 jwt_algorithm = "HS256"
 jwt_header = {
 'alg': jwt_algorithm,
 'typ': 'JWT',
 'kid': api_key,
 }

 now = datetime.now(tz=timezone.utc)
 expiration = now + timedelta(minutes=10)

 payload = {
 'sub': api_key,
 'iat': int(now.timestamp()),
 'exp': int(expiration.timestamp()),
 'iss': 'sdk'
 }

 return jwt.encode(payload=payload, key=api_secret, algorithm=jwt_algorithm, headers=jwt_header)

```

```

def extract_image_id_from_filename(self, filename: str) -> str:
 """Extract unique identifier from DICOM filename.

 Examples:
 - series-00000_image-00000.jpg -> series-00000_image-00000
 - series-00001_image-00005.jpg -> series-00001_image-00005
 """
 # Remove directory path first
 basename = os.path.basename(filename)

 # Remove file extension
 name_without_ext = os.path.splitext(basename)[0]

 return name_without_ext

def read_csv(self, csv_file: str) -> List[Dict[str, Any]]:
 """Read CSV file and return list of records."""
 if not os.path.exists(csv_file):
 raise FileNotFoundError(f"File not found: {csv_file}")

 with open(csv_file, 'r', encoding='utf-8') as f:
 reader = csv.DictReader(f)
 return list(reader)

def read_metadata_json(self, json_file: str) -> List[Dict[str, Any]]:
 """Read metadata JSON file from Visual Layer containing filename->media_id map
 if not os.path.exists(json_file):
 raise FileNotFoundError(f"Metadata JSON file not found: {json_file}")

 with open(json_file, 'r', encoding='utf-8') as f:
 data = json.load(f)

 # Extract media_items array
 media_items = data.get('media_items', [])

 # Convert to the expected format with media_id and filename fields
 export_items = []
 for item in media_items:
 export_items.append({
 'media_id': item.get('media_id'),
 'filename': item.get('file_name') # Use file_name from JSON
 })

 return export_items

def create_image_id_mapping(self, export_items: List[Dict[str, Any]]) -> Dict[str,

```



```

Check if user specified this field as enum
if field_name in self.enum_fields:
 field_types[field_name] = 'enum'
Handle numeric array fields as strings
elif field_name in numeric_array_fields:
 field_types[field_name] = 'string'
Handle image_type as multi-enum (contains text arrays)
elif field_name == 'image_type':
 field_types[field_name] = 'multi-enum'
Check for DICOM date fields (YYYYMMDD format)
elif self._is_dicom_date(field_name, value):
 field_types[field_name] = 'datetime'
Check for DICOM time fields (HHMMSS format)
elif self._is_dicom_time(field_name, value):
 field_types[field_name] = 'datetime' # Convert times to datetime
Determine field type based on value patterns
elif self._is_float(value):
 field_types[field_name] = 'float'
elif self._is_date(value):
 field_types[field_name] = 'datetime'
elif self._is_list(value):
 # Check if it's multi-enum or single enum
 unique_values = set()
 for row in csv_data:
 list_items = self._parse_list_value(row[field_name])
 unique_values.update(list_items)

 if len(unique_values) <= 20: # API enum limit
 field_types[field_name] = 'multi-enum'
 else:
 field_types[field_name] = 'string'
else:
 # For DICOM data, treat most fields as string unless specified as enum
 field_types[field_name] = 'string'

return field_types

def _is_dicom_date(self, field_name: str, value: str) -> bool:
 """Check if field is a DICOM date field (YYYYMMDD format)."""
 date_field_names = ['study_date', 'series_date', 'acquisition_date', 'instance_date']
 if field_name not in date_field_names:
 return False

 if not isinstance(value, str) or len(value.strip()) != 8:
 return False

 try:

```



```

 # Try to parse as YYYYMMDD
 datetime.strptime(value.strip(), '%Y%m%d')
 return True
 except (ValueError, TypeError):
 return False

def _is_dicom_time(self, field_name: str, value: str) -> bool:
 """Check if field is a DICOM time field (HHMMSS format)."""
 time_field_names = ['study_time', 'series_time', 'acquisition_time', 'instance_time']
 if field_name not in time_field_names:
 return False

 if not isinstance(value, str) or len(value.strip()) != 6:
 return False

 try:
 # Try to parse as HHMMSS
 datetime.strptime(value.strip(), '%H%M%S')
 return True
 except (ValueError, TypeError):
 return False

def _is_float(self, value: str) -> bool:
 """Check if value can be converted to float."""
 if not isinstance(value, str) or not value.strip():
 return False
 try:
 float(value)
 return '.' in value or 'e' in value.lower()
 except (ValueError, TypeError):
 return False

def _is_date(self, value: str) -> bool:
 """Check if value looks like a date using pandas parsing."""
 if not isinstance(value, str) or not value.strip():
 return False

 try:
 # Use pandas to parse the date - it's very flexible with formats
 pd.to_datetime(value.strip())
 return True
 except (ValueError, TypeError, pd.errors.ParserError):
 return False

def _is_list(self, value: str) -> bool:
 """Check if value looks like a list."""
 if not isinstance(value, str):

```

```

 return False

 value = value.strip()

 # Check for DICOM image_type format: "['ORIGINAL', 'PRIMARY', 'LOCALIZER']"
 if value.startswith('["[') or value.startswith('["['):
 return True

 # Check for CSV-style quoted lists: "["item1","item2"]"
 if value.startswith('["') and value.endswith('"]'):
 return True

 # Check for standard list formats
 if (value.startswith('[') and value.endswith(']')) or ',' in value:
 try:
 ast.literal_eval(value)
 return True
 except (ValueError, SyntaxError):
 return ',' in value
 return False

def _parse_list_value(self, value: str) -> List[str]:
 """Parse list value from CSV, handling DICOM image_type format."""
 if not value:
 return []

 value = value.strip()

 # Handle DICOM image_type format: "['ORIGINAL', 'PRIMARY', 'LOCALIZER']"
 if value.startswith('["[') and value.endswith('"]'):
 # Remove outer quotes
 inner_value = value[1:-1] # Remove outer quotes
 try:
 return ast.literal_eval(inner_value)
 except:
 pass

 # Remove outer quotes if present (from CSV)
 if value.startswith('"') and value.endswith('"'):
 value = value[1:-1]

 try:
 # Handle double-quoted strings in CSV
 cleaned = value.replace('""', '"')
 return ast.literal_eval(cleaned)
 except:
 # Fallback: split by comma if it looks like a simple list

```

```

 if ',' in value:
 return [item.strip().strip('"\'') for item in value.split(',')]
 return [value.strip().strip('"\'')]

def create_custom_field(self, field_name: str, field_type: str, values: List[str])
 """Create a custom field and return field_id."""
 print(f" Creating custom field: {field_name} ({field_type}")

 # Prepare field data based on type
 field_data = {
 "field_name": field_name,
 "field_type": field_type
 }

 # Add enum values for enum types
 if field_type in ['enum', 'multi-enum'] and values:
 unique_values = list(set(values)) # Remove duplicates
 field_data["enum_options"] = unique_values[:20] # Limit enum values to AP
 if field_type == 'multi-enum':
 field_data["field_type"] = 'enum' # API only accepts 'enum'
 field_data["is_multi"] = True
 else:
 field_data["is_multi"] = False
 print(f" Adding {len(field_data['enum_options'])} enum values")

 url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks"

 try:
 response = self.session.post(url, json=field_data, headers={'Content-Type':
 if response.status_code == 200:
 result = response.json()
 task_id = result.get('task_id')
 print(f" Created field with task ID: {task_id}")
 return task_id
 elif "already exists" in response.text:
 print(f" Field already exists, skipping creation")
 return None # Skip upload for existing fields
 else:
 print(f" Failed to create field: {response.status_code} - {response
 return None
 except requests.RequestException as e:
 print(f" Request failed: {str(e)}")
 return None

def upload_field_data(self, field_id: str, csv_data: List[Dict[str, Any]],
 field_name: str, field_type: str, image_id_mapping: Dict[str,
 """Upload data for a custom field using image_filename mapping."""

```

```

print(f" □ Uploading data for field: {field_name}")

Prepare upload data using image_filename mapping
upload_data = []
matched_count = 0

for row in csv_data:
 image_filename = row.get('image_filename', '').strip()
 if not image_filename:
 continue

 # Extract image_id from filename
 image_id = self.extract_image_id_from_filename(image_filename)

 # Look up media_ids using image_id
 media_ids = image_id_mapping.get(image_id, [])
 if not media_ids:
 continue # Skip if no matching media_ids found

 value = row.get(field_name, '')
 if value: # Only upload non-empty values
 # Convert value based on field type
 if field_type == 'float':
 try:
 value = float(value)
 except (ValueError, TypeError):
 continue
 elif field_type == 'datetime':
 # Handle DICOM dates and times
 try:
 if self._is_dicom_date(field_name, value):
 # Parse DICOM date format YYYYMMDD
 dt = datetime.strptime(value.strip(), '%Y%m%d')
 value = dt.strftime('%Y-%m-%dT00:00:00Z')
 elif self._is_dicom_time(field_name, value):
 # Parse DICOM time format HHMMSS and convert to today's date
 time_str = value.strip()
 if len(time_str) == 6:
 # Create datetime with today's date + DICOM time
 from datetime import date
 today = date.today()
 dt = datetime.strptime(f"{today.strftime('%Y%m%d')}T{time_str}Z", '%Y-%m-%dT%H:%M:%SZ')
 value = dt.strftime('%Y-%m-%dT%H:%M:%SZ')
 else:
 # Invalid time format, skip
 continue
 except:
 else:

```

```

 # Use pandas to parse and format datetime to ISO 8601 UTC
 dt = pd.to_datetime(value.strip())
 # Convert to UTC and format as ISO 8601
 if dt.tz is None:
 # If no timezone, assume UTC
 value = dt.strftime('%Y-%m-%dT%H:%M:%SZ')
 else:
 # Convert to UTC and format
 value = dt.tz_convert('UTC').strftime('%Y-%m-%dT%H:%M:%S')
 except (ValueError, TypeError, pd.errors.ParserError):
 # If parsing fails, skip this value
 continue
elif field_type == 'multi-enum':
 # Parse list for multi-enum - ensure all values are strings
 try:
 parsed_list = self._parse_list_value(value)
 value = [str(item).strip() for item in parsed_list]
 except:
 value = [str(value).strip()] if value else []
elif field_type == 'enum':
 # Single enum - ensure string
 value = str(value).strip()
elif field_type == 'string':
 # Ensure string type
 value = str(value).strip()

 if value != '' and value is not None:
 # Create upload entry for each media_id
 for media_id in media_ids:
 upload_data.append({
 "media_id": media_id,
 "value": value
 })
 matched_count += 1

total_files = len(upload_data)
unique_items = len([row for row in csv_data if row.get('image_filename', '').s
print(f" [green] Processed {unique_items} image files from {len(csv_data)} CSV ro
print(f" [green] Created {total_files} file entries for upload")

if not upload_data:
 print(f" [green] No data to upload for field {field_name}")
 return None

Save metadata to temporary file
import tempfile

```

```

with tempfile.NamedTemporaryFile(mode='w', suffix='.json', delete=False) as f:
 json.dump(upload_data, f, indent=2)
 temp_file = f.name

Upload file to existing task
url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks/{field_id}"

try:
 with open(temp_file, 'rb') as f:
 files = {'file': (f'metadata_{field_name}.json', f, 'application/json')}
 response = self.session.post(url, files=files)

 # Clean up temp file
 os.unlink(temp_file)

 if response.status_code in [200, 202]:
 print(f" Upload completed successfully")
 return field_id
 else:
 print(f" Failed to upload data: {response.status_code} - {response}")
 return None
except requests.RequestException as e:
 print(f" Request failed: {str(e)}")
 return None

def check_task_status(self, task_id: str) -> str:
 """Check the status of an upload task."""
 url = f"{self.base_url}/{self.dataset_id}/custom_metadata/tasks/{task_id}/status"

 try:
 response = self.session.get(url)
 if response.status_code == 200:
 result = response.json()
 status = result.get('status', 'unknown')

 # Check for errors and provide detailed information
 if status == 'COMPLETED_WITH_ERRORS':
 error_count = result.get('error_count', 0)
 inserted_rows = result.get('inserted_rows', 0)
 print(f" Task completed with {error_count} errors, {inserted_rows} rows")

 # Show sample errors if available
 sample_errors = result.get('sample_errors', [])
 if sample_errors:
 print(f" Sample errors:")
 for error in sample_errors[:3]: # Show first 3 errors
 print(f" - Row {error.get('row_index', '?')}: {error.get('error_message', '')}")
 else:
 print(f" Task status: {status}")
 else:
 print(f" Failed to check task status: {response.status_code} - {response}")
 return None
 except requests.RequestException as e:
 print(f" Request failed: {str(e)}")
 return None

```

```

 if len(sample_errors) > 3:
 print(f" - ... and {len(sample_errors) - 3} more errors")

 elif status == 'COMPLETED':
 inserted_rows = result.get('inserted_rows', 0)
 print(f" □ Task completed successfully, {inserted_rows} rows inserted")

 return status
 else:
 return 'error'
except requests.RequestException:
 return 'error'

def wait_for_task_completion(self, task_id: str, field_name: str, polling_interval: int):
 """Wait for task completion by polling status endpoint indefinitely."""
 print(f" □ Waiting for task completion... (Press Ctrl+C to stop)")
 start_time = time.time()

 while True:
 status = self.check_task_status(task_id)

 if status in ['COMPLETED', 'COMPLETED_WITH_ERRORS']:
 elapsed = int(time.time() - start_time)
 print(f" □ Task completed after {elapsed}s")
 return status
 elif status == 'error':
 print(f" □ Error checking task status")
 return 'error'
 elif status == 'IN_PROGRESS':
 elapsed = int(time.time() - start_time)
 print(f" □ Task in progress... (elapsed: {elapsed}s)")
 else:
 elapsed = int(time.time() - start_time)
 print(f" □ Task status: {status} (elapsed: {elapsed}s)")

 # Wait before next check
 time.sleep(polling_interval)

def save_progress(self, task_ids: Dict[str, Dict[str, str]], completed_fields: List[str]):
 """Save progress to a JSON file."""
 progress = {
 'task_ids': task_ids,
 'completed_fields': completed_fields
 }
 with open('csv_json_workflow_progress.json', 'w') as f:
 json.dump(progress, f, indent=2)

```

```

def load_progress(self) -> tuple:
 """Load progress from JSON file."""
 try:
 with open('csv_json_workflow_progress.json', 'r') as f:
 progress = json.load(f)
 return progress.get('task_ids', {}), progress.get('completed_fields', [])
 except FileNotFoundError:
 return {}, []

def cleanup_temp_files(self):
 """Remove temporary files created during processing."""
 for temp_file in self._temp_files:
 try:
 if os.path.exists(temp_file):
 os.remove(temp_file)
 print(f" [] Cleaned up temporary file: {temp_file}")
 except OSError as e:
 print(f" [] Could not remove {temp_file}: {e}")

def process_workflow(self, csv_file: str, json_file: str, resume: bool = False):
 """Main workflow processing function using CSV for metadata and JSON for mapping"""
 try:
 print("[] Starting CSV Metadata Workflow")
 print(f"[] CSV metadata: {csv_file}")
 print(f"[] JSON metadata: {json_file}")

 # Load progress if resuming
 task_ids = {}
 completed_fields = []

 if resume:
 task_ids, completed_fields = self.load_progress()
 print(f"[] Resuming workflow - {len(completed_fields)} fields already")

 # Read CSV data (contains DICOM metadata with image_filename)
 print("\n[] Reading CSV metadata...")
 csv_data = self.read_csv(csv_file)
 print(f"[] Loaded {len(csv_data)} CSV records")

 # Verify image_filename field exists
 if not csv_data or 'image_filename' not in csv_data[0]:
 raise ValueError("CSV must contain 'image_filename' field")

 # Read JSON metadata for filename->media_id mapping
 print("\n[] Reading JSON metadata...")
 export_items = self.read_metadata_json(json_file)
 print(f"[] Loaded {len(export_items)} media items")

```



```

Create image_id mapping from export CSV
image_id_mapping = self.create_image_id_mapping(export_items)
if not image_id_mapping:
 raise ValueError("Could not create image_id mapping - check export CSV")

Analyze fields
print("\n Analyzing CSV fields...")
field_types = self.analyze_csv_fields(csv_data)
print(" Field analysis:")
for field_name, field_type in field_types.items():
 status = " Completed" if field_name in completed_fields else " Pending"
 print(f" {field_name}: {field_type} - {status}")

print(f"\n Processing {len(field_types)} custom fields...")

Process each field
for field_name, field_type in field_types.items():
 if field_name in completed_fields:
 print(f"\n Skipping completed field: {field_name}")
 continue

 print(f"\n Processing field: {field_name} ({field_type})")

 try:
 # Collect unique values for enum types
 values = []
 if field_type in ['enum', 'multi-enum']:
 for row in csv_data:
 if field_type == 'multi-enum':
 values.extend(self._parse_list_value(row[field_name]))
 else:
 val = row[field_name]
 if val:
 values.append(str(val))

 # Create custom field
 field_id = self.create_custom_field(field_name, field_type, values)
 if not field_id:
 print(f" Skipping field {field_name} (already exists or invalid)")
 continue

 # Upload data
 task_id = self.upload_field_data(field_id, csv_data, field_name, field_type)
 if not task_id:
 raise Exception(f"Failed to upload data for {field_name}")

 # Store task info

```

```

 task_ids[field_name] = {
 'field_id': field_id,
 'task_id': task_id
 }

 # Wait for task completion before moving to next field
 print(f" [] Upload completed, monitoring task status...")
 final_status = self.wait_for_task_completion(task_id, field_name)

 if final_status in ['COMPLETED', 'COMPLETED_WITH_ERRORS']:
 completed_fields.append(field_name)
 status_icon = "[] " if final_status == 'COMPLETED' else "[X] "
 print(f" {status_icon} Field {field_name} {final_status.lower()}")
 else:
 print(f" [X] Field {field_name} failed or timed out (status: {final_status})")
 # Continue with next field even if this one failed

 except Exception as e:
 print(f" [X] Error processing field {field_name}: {str(e)}")
 print(f" [X] Skipping field and continuing with next field...")
 self.save_progress(task_ids, completed_fields)
 continue

 print()

print("[] Workflow completed!")
print(f"[] Successfully processed {len(completed_fields)} fields")

Show task summary
if task_ids:
 print("\n[] Task Summary:")
 for field_name, task_info in task_ids.items():
 field_id = task_info['field_id']
 task_id = task_info['task_id']

 if field_name in completed_fields:
 status = "[] completed"
 else:
 status = "[X] failed/timeout"

 print(f" {field_name}: {status}")

Clean up progress file when workflow completes successfully
progress_file = "csv_json_workflow_progress.json"
if os.path.exists(progress_file):
 try:
 os.remove(progress_file)
 except:
 pass

```

```

 print(f"\n Cleaned up progress file: {progress_file}")
 except OSError as e:
 print(f" Could not remove progress file {progress_file}: {e}")

 finally:
 # Always clean up temporary files
 self.cleanup_temp_files()

def main():
 parser = argparse.ArgumentParser(description='CSV Metadata Workflow for Visual Layer')
 parser.add_argument('csv_file', help='Path to dicom_metadata.csv file')
 parser.add_argument('json_file', help='Path to metadata.json file from Visual Layer')
 parser.add_argument('--dataset-id', required=True, help='Visual Layer dataset ID')
 parser.add_argument('--base-url', default='https://app.visual-layer.com',
 help='Base URL (default: https://app.visual-layer.com)')
 parser.add_argument('--api-key', required=True, help='API key for authentication')
 parser.add_argument('--api-secret', help='API secret for authentication')
 parser.add_argument('--enum-fields', nargs='*', default=[], help='List of column names')
 parser.add_argument('--resume', action='store_true', help='Resume from previous progress')

 args = parser.parse_args()

 # Create processor and run workflow
 processor = CsvJsonMetadataProcessor(args.dataset_id, args.base_url, args.api_key,
 args.api_secret, args.enum_fields)
 processor.process_workflow(args.csv_file, args.json_file, args.resume)

if __name__ == "__main__":
 main()

```

## Annotations Overview

### What Are Annotations?

Annotations are metadata labels that describe and categorize images or objects within images. They help structure visual datasets, making them searchable, analyzable, and useful for AI training.

### Common annotation types:

- **Image Annotations:** Assign class labels to entire images, helping categorize datasets.
- **Object Annotations:** Label individual objects within images using bounding boxes, improving model accuracy.

### Why Are Annotations Important?

Annotations play a critical role in machine learning and AI model training by:

- ❑ Improving data organization for easier retrieval and filtering
  - ❑ Enhancing search-ability within datasets
  - ❑ Supporting object detection and classification tasks
  - ❑ Increasing model accuracy with high-quality labeled data
- 

### Supported Formats

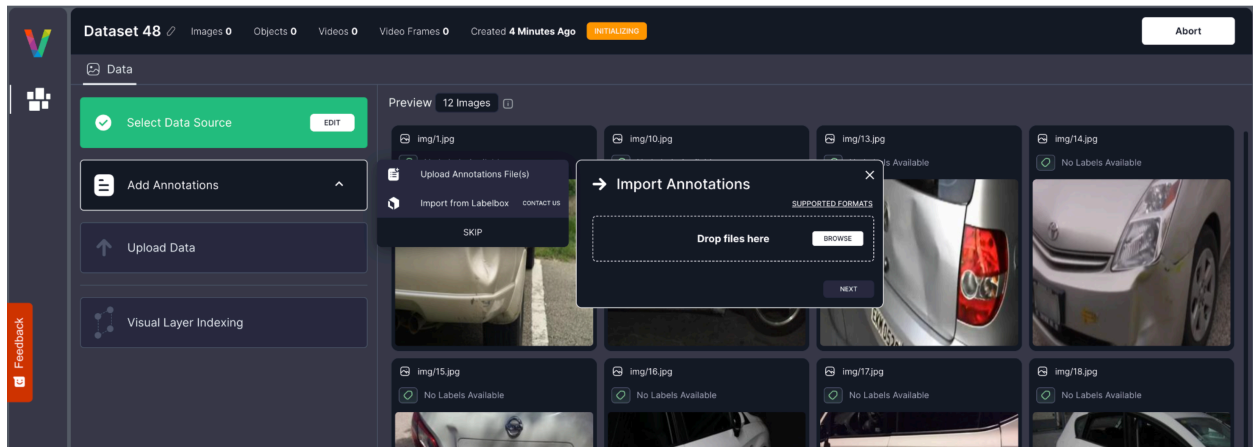
1. **Parquet / CSV**
2. **JSON (COCO format)**

Your annotation **file must be named exactly** as one of the following: - `annotations.json` - `image_annotations.csv` - `object_annotations.csv` - `image_annotations.parquet` - `object_annotations.parquet`

**Note: Important:** Annotations must be added when creating a dataset.

### How to Import Annotations

1. Upload your annotation file **during dataset creation**.  
Files can be uploaded from your **local machine** and **S3 bucket**.
2. Ensure that your file follows the **required format** and has the correct name.



**Note:** For more details about creating your annotation file, visit - [Preparing annotation file](#)

## Preparing Annotation File

**Note:** To convert YOLO annotation to VL format visit: [Converting YOLO Annotations to Visual Layer Format](#)

# Tabular Formats: Parquet / CSV

## Image Annotations

For full-image class labels, create an annotation file named `image_annotations.csv` or `image_annotations.parquet`. Each row in the file represents an image and its corresponding label.

### Example Format:

| filename                              | label  |
|---------------------------------------|--------|
| IDX_DF_SIG21341_PlasmasNeg.png        | IDX_DF |
| IDX_DF_ALM00324_PlasmasPos.png        | IDX_DF |
| folder/IDX_RC_ALM04559_PlasmasNeg.png | IDX_RC |

- The `filename` column must contain relative paths.
- The `label` column assigns a class to entire image.
- **Multiple labels** can be stored as a list, e.g., `['t-shirt', 'SKU12345']`.
- You may also include a `caption` column for textual metadata.

### Example for Multiple Labels:

| filename        | label                   |
|-----------------|-------------------------|
| cool-tshirt.png | ["t-shirt", "SKU12345"] |
| cool-pants.jpg  | ["pants", "SKU231312"]  |

## Object Annotations

For object-level annotations, create an annotation file named `object_annotations.csv` or `object_annotations.parquet`. Each row represents a detected object, including its bounding box coordinates and class label.

### Example Format:

| filename                              | col_x | row_y | width | height | label |
|---------------------------------------|-------|-------|-------|--------|-------|
| Kitti/raw/training/image_2/006149.png | 0     | 240   | 135   | 133    | Car   |
| Kitti/raw/training/image_2/006149.png | 608   | 169   | 59    | 43     | Car   |

- `col_x` and `row_y` define the top-left corner of the bounding box.
- `width` and `height` must be greater than zero.
- Each row corresponds to a single object within an image.



# JSON Annotations (COCO Format)

Ensure The file is named `annotations.json`.

## Example JSON Format:

```
{
 "images": [
 { "id": 1, "width": 640, "height": 480, "file_name": "image1.jpg" },
 { "id": 2, "width": 800, "height": 600, "file_name": "image2.jpg" }
],
 "categories": [
 { "id": 1, "name": "cat" },
 { "id": 2, "name": "dog" },
 { "id": 3, "name": "t-rex" }
],
 "annotations": [
 { "id": 1, "image_id": 1, "category_id": 1, "bbox": [100, 100, 200, 200] },
 { "id": 2, "image_id": 2, "category_id": 2, "bbox": [50, 50, 150, 150] },
 { "id": 3, "image_id": 1, "category_id": 3 },
 { "id": 4, "image_id": 2, "category_id": 3 }
]
}
```

- **Bounding boxes (bbox)** follow the format `[col_x, row_y, width, height]`.
  - `col_x` and `row_y` define the top-left corner of the bounding box.
  - `width` and `height` must be greater than zero.
- **Remove comments** before uploading.

---

## Summary of Bounding Box Formats

| Format                    | Representation                                            | File Type          |
|---------------------------|-----------------------------------------------------------|--------------------|
| <b>JSON (COCO)</b>        | <code>[x_min, y_min, width, height]</code>                | <code>.json</code> |
| <b>CSV(Image Labels)</b>  | <code>[filename, label]</code>                            | <code>.csv</code>  |
| <b>CSV(Object Labels)</b> | <code>filename, col_x, row_y, width, height, label</code> | <code>.csv</code>  |

For instructions on importing annotation files, see [Importing Annotations](#).

By following these guidelines, you can ensure that your annotation files are structured correctly and ready for seamless dataset creation in Visual Layer.

## Bounding Box Visual Layer Format

### Overview

The Visual Layer bounding box annotation format is stored in **CSV format**, making it easy to process and integrate with various machine learning workflows. Each row represents a single bounding box annotation for an object within an image.

### CSV Format Structure

| Column Name | Description                                                                               |
|-------------|-------------------------------------------------------------------------------------------|
| filename    | The name of the image file containing the object.                                         |
| col_x       | The x-coordinate (horizontal position) of the <b>top-left</b> corner of the bounding box. |
| row_y       | The y-coordinate (vertical position) of the <b>top-left</b> corner of the bounding box.   |
| width       | The width of the bounding box, extending from col_x.                                      |
| height      | The height of the bounding box, extending from row_y.                                     |
| label       | The class or category of the detected object.                                             |

### Example Annotation

```
filename,col_x,row_y,width,height,label
image1.jpg,50,30,200,150,car
image2.jpg,120,60,80,100,person
image3.jpg,15,10,50,70,dog
```

The top 5 common annotation formats for bounding boxes in image datasets include:

#### 1.COCO (Common Objects in Context) Format

- **Bounding Box Format:** [x\_min, y\_min, width, height]
- **Details:** Uses JSON format with a dictionary structure. The bounding box is represented by the top-left (x\_min, y\_min) coordinate, along with width and height.

- **Example:**

```
{
 "bbox": [100, 150, 200, 300]
}
```

- **Used In:** Microsoft COCO dataset.

## 2.PASCAL VOC (Visual Object Classes) Format

- **Bounding Box Format:** (x\_min, y\_min, x\_max, y\_max)
- **Details:** Stored in XML files using a <bndbox> tag. The bounding box is represented using absolute pixel values for top-left (x\_min, y\_min) and bottom-right (x\_max, y\_max) coordinates.

- **Example:**

```
<bndbox>
 <xmin>100</xmin>
 <ymin>150</ymin>
 <xmax>300</xmax>
 <ymax>450</ymax>
</bndbox>
```

- **Used In:** VOC 2012 dataset.
- 

## 3.YOLO (You Only Look Once) Format

- **Bounding Box Format:** [x\_center, y\_center, width, height], normalized (0 to 1 scale)
- **Details:** Uses plain text .txt files where each row represents an object as class\_id x\_center y\_center width height, with values normalized relative to the image width and height.

- **Example:**

```
0 0.5 0.6 0.25 0.4
```

- **Used In:** YOLO models and datasets.
- 

## 4.TFRecord / TensorFlow Object Detection API Format

- **Bounding Box Format:** Normalized (y\_min, x\_min, y\_max, x\_max) (0 to 1 scale)
- **Details:** Uses TFRecord format to store images and annotations in protocol buffers (Protobuf). Bounding boxes are stored as normalized float values within .tfrecord files.
- **Example**(within a . record file, usually not human-readable):

```
{
 "ymin": 0.25,
 "xmin": 0.20,
 "ymax": 0.75,
 "xmax": 0.80
}
```

- **Used In:** TensorFlow Object Detection API.

---

## 5.LabelMe Format

- **Bounding Box Format:** (x\_min, y\_min, x\_max, y\_max)
- **Details:** Uses JSON files similar to COCO but with a different structure, including polygon annotations. Bounding boxes are stored as [x\_min, y\_min, x\_max, y\_max].
- **Example:**

```
{
 "shapes": [
 {
 "label": "dog",
 "points": [[100, 150], [300, 450]],
 "shape_type": "rectangle"
 }
]
}
```

- **Used In:** LabelMe annotation tool.

---

### ###Summary of Bounding Box Formats

Format	Representation	Normalized?	File Type
<b>COCO</b>	[x_min, y_min, width, height]	No	.json
<b>VOC</b>	(x_min, y_min, x_max, y_max)	No	.xml
<b>YOLO</b>	[x_center, y_center, width, height]	Yes	.txt
<b>TFRecord</b>	(y_min, x_min, y_max, x_max)	Yes	.tfrecord
<b>LabelMe</b>	[[x_min, y_min], [x_max, y_max]]	No	.json

Each format is optimized for different use cases, with COCO and VOC being the most widely used in academic datasets, YOLO for real-time detection, TFRecord for TensorFlow-based training, and LabelMe for manual annotations.

# Converting to Visual Layer format

1. Convert from rom VOC2012 to Visual Layer Object Detection
  - VL Bounding Boxes
2. Convert from rom Coco to Visual Layer Object Detection
  - VL Bounding Boxes

## Converting YOLO Annotations to Visual Layer Format

In this dataset, the images and labels are organized as follows:

Images: Located in the folder output/images (PNG format).

Annotations: Located in the folder output/labels (text files).

Each text file corresponds to an image and contains zero or more rows in YOLO format.

## YOLO Annotation Format

Each annotation file contains lines with five values in the following format:

```
<class_id> <norm_cx> <norm_cy> <norm_w> <norm_h>
0 0.5869140625 0.2412109375 0.021484375 0.044921875
0 0.8974609375 0.185546875 0.044921875 0.1015625
...
```

class\_id: Represents the object class (e.g. 0 corresponds to “ship”).

norm\_cx, norm\_cy: Normalized center coordinates (values between 0 and 1).

norm\_w, norm\_h: Normalized width and height (values between 0 and 1).

## Conversion Process

The conversion script performs the following steps:

- Load Each Image:  
The script loads each PNG image to retrieve its pixel dimensions (width and height).
- Read the Corresponding Annotation File:  
It reads each text file from output/labels that corresponds to an image.
- Convert Normalized Coordinates:  
For each annotation, the script converts the normalized center coordinates and dimensions into absolute pixel values. The top-left coordinates are calculated as:

```
top_left_x = (center_x - width/2)
top_left_y = (center_y - height/2)
```

- Generate the CSV File:  
The CSV is generated with the following columns:

filename: The image file name.

col\_x: The x-coordinate (column) of the top-left corner.

row\_y: The y-coordinate (row) of the top-left corner.

w: Bounding box width (pixels).

h: Bounding box height (pixels).

label: The object label (e.g., “ship”).

## Python Conversion Script

```
Paths to your folders
labels_folder = "output/labels"
images_folder = "output/images"
output_csv = "annotations.csv"

Open CSV for writing
with open(output_csv, "w", newline="") as csvfile:
 writer = csv.writer(csvfile)
 # Write header row
 writer.writerow(["filename", "col_x", "row_y", "width", "height", "label"])

Process each label file
for label_file in os.listdir(labels_folder):
 if label_file.endswith(".txt"):
 # Assume the image file has the same base name with .png extension
 image_base = os.path.splitext(label_file)[0]
 image_filename = image_base + ".png"
 image_path = os.path.join(images_folder, image_filename)
 label_file_path = os.path.join(labels_folder, label_file)

 # Check if the image exists
 if not os.path.exists(image_path):
 print(f"Image not found for {label_file}")
 continue

 # Read image to get dimensions
 image = cv2.imread(image_path)
 if image is None:
 print(f"Failed to load image {image_path}")
 continue

 h_img, w_img = image.shape[:2]

 # Open and read the label file
 with open(label_file_path, "r") as f:
 lines = f.readlines()
 for line in lines:
 parts = line.strip().split()
 if len(parts) != 5:
 continue # Skip lines not in expected format

 # YOLO format: class, norm_cx, norm_cy, norm_w, norm_h
 class_id, norm_cx, norm_cy, norm_w, norm_h = map(float, parts)
```



```

Convert normalized values to absolute pixel values
abs_cx = norm_cx * w_img
abs_cy = norm_cy * h_img
abs_w = norm_w * w_img
abs_h = norm_h * h_img

Calculate the top-left corner coordinates
top_left_x = abs_cx - abs_w / 2
top_left_y = abs_cy - abs_h / 2

Map class_id to a string label; here we assume 0 means "ship"
label_str = "ship" if class_id == 0 else str(int(class_id))

Write the row to CSV (coordinates cast to int)
writer.writerow([
 image_filename,
 int(top_left_x),
 int(top_left_y),
 int(abs_w),
 int(abs_h),
 label_str
])

```

## Script Explanation

- **Input & Output Folders:**  
The script iterates over all .txt files in output/labels and assumes a matching .png image exists in output/images.
- **Image Dimensions:**  
OpenCV (cv2) is used to load each image to determine its width and height, which are necessary for converting normalized coordinates to pixel values.
- **Coordinate Conversion:**  
The normalized center coordinates and dimensions are multiplied by the image dimensions. The top-left corner of the bounding box is then computed by subtracting half the width and height from the center coordinates.
- **CSV Output:**  
The CSV file (annotations.csv) is written with each annotation as a row containing the image filename, the computed top-left coordinates, bounding box width and height in pixels, and the object label (e.g., "ship").

### Example CSV Output

After running the script, the CSV output will look similar to this:

filename	col_x	row_y	width	height	label
output_000001.png	150	100	30	40	ship
output_000002.png	200	80	25	35	ship
output_000003.png	175	90	28	32	ship

filename	col_x	row_y	width	height	label
----------	-------	-------	-------	--------	-------

filename: Name of the image file.

col\_x & row\_y: Top-left pixel coordinates of the bounding box.

w & h: Width and height of the bounding box in pixels.

label: Object label ("ship").

## Converting Segmentation Masks to Visual Layer Format

In this dataset, the images and annotations are organized as follows:

Images: Located in a directory (PNG/JPG format).

Annotations: JSON files containing polygon-based segmentation masks.

Each JSON file corresponds to an image and contains polygon annotations with coordinate points defining object boundaries.

## Segmentation Mask Annotation Format

Each annotation file contains polygon shapes in the following JSON structure:

```
{
 "version": "4.5.6",
 "shapes": [
 {
 "label": "QSBD",
 "points": [
 [64, 10],
 [64, 15],
 [67, 15],
 [68, 14],
 [68, 10]
],
 "shape_type": "polygon"
 }
],
 "imagePath": "example_image.png"
}
```

Example annotation structure:

```
{
 "label": "QSBD",
 "points": [
 [64, 42], [68, 43], [69, 41], [71, 40], [75, 40],
 [77, 38], [77, 36], [78, 33], [76, 30], [71, 29],
 [70, 27], [68, 26], [64, 26], [64, 29]
],
 "shape_type": "polygon"
}
```

label: Represents the object class (e.g., "QSBD").

points: Array of [x, y] coordinates defining the polygon boundary.

shape\_type: Type of annotation ("polygon" for segmentation masks).

imagePath: Corresponding image filename.

## Conversion Process

The conversion script performs the following steps:

- Load Each JSON File:  
The script reads each JSON annotation file to extract polygon coordinates.
- Extract Polygon Points:  
For each shape, it extracts the array of [x, y] coordinate points.
- Calculate Bounding Box:  
The script converts polygon coordinates to bounding boxes by finding the minimum and maximum x,y values:

```
min_x = minimum x-coordinate from all points
max_x = maximum x-coordinate from all points
min_y = minimum y-coordinate from all points
max_y = maximum y-coordinate from all points
```

```
col_x = min_x (left edge)
row_y = min_y (top edge)
width = max_x - min_x
height = max_y - min_y
```

- Generate the CSV File:  
The CSV is generated with the following columns:

filename: The image file name.  
col\_x: The x-coordinate (column) of the top-left corner.  
row\_y: The y-coordinate (row) of the top-left corner.  
width: Bounding box width (pixels).  
height: Bounding box height (pixels).  
label: The object label (e.g., "QSBD").

## Python Conversion Script

```
from pathlib import Path
from typing import List, Tuple, Dict, Any

def extract_polygon_points(shape: Dict[str, Any]) -> List[Tuple[int, int]]:
 """Extract polygon points from a shape annotation."""
 if 'points' not in shape:
 return []

 points = []
 for point in shape['points']:
 if len(point) >= 2:
 x, y = int(point[0]), int(point[1])
 points.append((x, y))
```

```

 return points

def polygon_to_bbox(points: List[Tuple[int, int]]) -> Tuple[int, int, int, int]:
 """Convert polygon points to bounding box coordinates."""
 if not points:
 return (0, 0, 0, 0)

 x_coords = [point[0] for point in points]
 y_coords = [point[1] for point in points]

 min_x = min(x_coords)
 max_x = max(x_coords)
 min_y = min(y_coords)
 max_y = max(y_coords)

 col_x = min_x
 row_y = min_y
 width = max_x - min_x
 height = max_y - min_y

 return (col_x, row_y, width, height)

def process_json_file(json_path: Path) -> List[Dict[str, Any]]:
 """Process a JSON annotation file and extract bounding boxes."""
 try:
 with open(json_path, 'r', encoding='utf-8') as f:
 data = json.load(f)
 except (json.JSONDecodeError, FileNotFoundError) as e:
 print(f"Error reading {json_path}: {e}")
 return []

 if 'shapes' not in data:
 print(f"No 'shapes' key found in {json_path}")
 return []

 # Get the corresponding image filename
 image_filename = data.get('imagePath', json_path.stem + '.png')

 bboxes = []
 for shape in data['shapes']:
 if shape.get('shape_type') != 'polygon':
 continue

 label = shape.get('label', 'unknown')
 points = extract_polygon_points(shape)

 if not points:

```

```

 continue

 col_x, row_y, width, height = polygon_to_bbox(points)

 # Skip invalid bounding boxes (zero width or height)
 if width <= 0 or height <= 0:
 continue

 bbox_info = {
 'filename': image_filename,
 'col_x': col_x,
 'row_y': row_y,
 'width': width,
 'height': height,
 'label': label
 }
 bboxes.append(bbox_info)

 return bboxes

Paths to your folders
annotations_folder = "annotations"
output_csv = "segmentation_annotations.csv"

Process all JSON files and convert to CSV
all_bboxes = []
json_files = list(Path(annotations_folder).glob('*.json'))

for json_file in json_files:
 print(f"Processing: {json_file.name}")
 bboxes = process_json_file(json_file)
 all_bboxes.extend(bboxes)
 print(f" Found {len(bboxes)} bounding boxes")

Write to CSV
fieldnames = ['filename', 'col_x', 'row_y', 'width', 'height', 'label']

with open(output_csv, 'w', newline='', encoding='utf-8') as csvfile:
 writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
 writer.writeheader()
 writer.writerows(all_bboxes)

print(f"Conversion complete! Output saved to: {output_csv}")
print(f"Total bounding boxes: {len(all_bboxes)}")

```

## Script Explanation

- **Input Processing:**  
The script iterates over all .json files in the annotations folder and extracts polygon shapes from each file.
- **Polygon Extraction:**  
For each polygon shape, the script extracts the coordinate points and validates the data structure.
- **Bounding Box Calculation:**  
The polygon coordinates are converted to bounding boxes by finding the minimum and maximum x,y values to determine the rectangular bounds.
- **CSV Output:**  
The CSV file is written with each bounding box as a row containing the image filename, top-left coordinates, dimensions, and object label.

### Example CSV Output

After running the script, the CSV output will look similar to this:

filename	col_x	row_y	width	height	label
example.png	64	10	4	5	QSBD
example.png	64	26	14	17	QSBD
example.png	65	68	13	24	QSBD

filename: Name of the image file.

col\_x & row\_y: Top-left pixel coordinates of the bounding box.

width & height: Width and height of the bounding box in pixels.

label: Object label from the segmentation mask.

## Usage

1. Place your JSON annotation files in the annotations folder
2. Run the conversion script: `python convert_segmentation.py`
3. The output CSV file `segmentation_annotations.csv` will be generated
4. Import this CSV file into Visual Layer for object detection tasks

The converted bounding boxes provide approximate rectangular bounds for the original polygon segmentation masks, making them compatible with Visual Layer's object annotation format.

## Creating Annotation File with Class Labels

This example shows how to create VL annotations file using subfolder structure, each subfolder is the class label.

The root folder of the dataset contains 3 subfolders which are the classes: Ulcer, Normal and AVM.

```
def create_annotation_csv(root_dir, output_csv):
 rows = []

 # Iterate over all items in the root directory.
 for subdir in os.listdir(root_dir):
 subdir_path = os.path.join(root_dir, subdir)
 if os.path.isdir(subdir_path):
 label = subdir # The subfolder name is the label.
 # Iterate over each file in the subdirectory.
 for filename in os.listdir(subdir_path):
 file_path = os.path.join(subdir_path, filename)
 if os.path.isfile(file_path):
 # Build the relative file path (e.g., "b/image1.jpg").
 relative_path = os.path.join(subdir, filename)
 rows.append([relative_path, label])

 # Write the CSV file with header "filename,label".
 with open(output_csv, mode='w', newline='') as csv_file:
 writer = csv.writer(csv_file)
 writer.writerow(['filename', 'label'])
 writer.writerows(rows)
 print(f"CSV file '{output_csv}' created successfully.")

if __name__ == "__main__":
 root_directory = "." # Folder containing your subfolders.
 output_csv_file = "image_annotations.csv"
 create_annotation_csv(root_directory, output_csv_file)
```

Example output:

```
filename,label
Ulcer/Ulcer_2024-08-07-08-28-10_81061.bmp,Ulcer
Ulcer/Ulcer_2024-08-07-08-29-37_82025.bmp,Ulcer
Ulcer/Ulcer_2024-08-07-08-29-07_43887.bmp,Ulcer
Ulcer/Ulcer_2024-08-07-08-28-10_43888.bmp,Ulcer
Ulcer/Ulcer_2024-08-07-08-30-31_30154.bmp,Ulcer
Ulcer/Ulcer_2024-08-07-08-27-27_96319.bmp,Ulcer
```



## Reusing Caption Data from Previous Datasets

### Introduction

Caption generation is one of the most time-consuming operations in Visual Layer's dataset pipeline. When creating multiple datasets with the same images, re-running caption generation for each dataset wastes valuable time and computational resources.

This guide shows you how to **extract and reuse caption data** from previous pipeline runs on the same data, allowing you to:

- ☐ Skip caption generation on subsequent dataset creations
- ☐ Maintain consistent captions across multiple datasets

**Note: Use Case:** This approach is ideal when you need to create multiple datasets or dataset versions using the same images but with different configurations.

### Overview

After running a dataset pipeline, Visual Layer stores processed data in:

```
/.vl/tmp/[dataset-id]/input/metadata/image_annotations.parquet
```

This parquet file contains all the caption data you need to reuse.

### Using the Caption Extraction Script

#### Prerequisites

- Python 3.x
- pandas library (pip install pandas pyarrow)

#### What the Script Does

The extraction script processes Visual Layer's internal parquet files to create a clean annotation file ready for reuse:

1. **Extracts relevant columns:** Keeps only filename and caption
2. **Removes system paths:** Strips prefixes like /hostfs, /mnt, etc.
3. **Creates relative paths:** Converts absolute paths to relative filenames
4. **Outputs clean parquet:** Generates a properly formatted image\_annotations.parquet

#### Script Location

**View Complete Script Code** The complete Python script is available on a separate page. Click [here](#) to view and copy the code.

## Installation

```
Ensure pandas and pyarrow are installed
pip install pandas pyarrow

Make script executable (optional)
chmod +x process_annotations.py
```

## Step-by-Step Workflow

### Step 1: Create Initial Dataset (with Captioning)

Create your first dataset with captioning enabled as usual. This will generate the initial captions and store them in the internal parquet file.

After the pipeline completes, find the dataset ID and locate the parquet file:

```
List recent datasets
ls -lt /.vl/tmp/

Navigate to your dataset's metadata
cd /.vl/tmp/[your-dataset-id]/input/metadata/

Verify the file exists
ls image_annotations.parquet
```

### Step 3: Run the Extraction Script

Process the parquet file to extract captions:

```
Basic usage - creates image_annotations_processed.parquet in same directory
python3 process_annotations.py /.vl/tmp/[dataset-id]/input/metadata/image_annotations.

Specify custom output location
python3 process_annotations.py /.vl/tmp/[dataset-id]/input/metadata/image_annotations.
 -o /path/to/new-dataset/image_annotations.parquet

Custom prefix removal (if needed)
python3 process_annotations.py input.parquet --prefix /custom/prefix/to/remove
```

### Script Output:

```
Reading parquet file: /.vl/tmp/abc123.../input/metadata/image_annotations.parquet
Original shape: (12, 9)
Columns: ['filename', 'file_size_bytes', 'video', 'frame_timestamp', 'caption', ...]
```

```
Removing prefix '/hostfs' from filenames...
```

```
Processed shape: (12, 2)
```

Sample filenames after processing:  
['dog\_1.jpg', 'dog\_2.jpg', 'dog\_3.jpg']

- ✓ Successfully processed 12 rows
- ✓ Output saved to: /path/to/output.parquet

## Step 4: Copy to New Dataset Directory

Place the extracted parquet file in your new dataset directory alongside the images:

```
Copy to new dataset location
cp image_annotations_processed.parquet /path/to/new-dataset/image_annotations.parquet

Directory structure should look like:
/path/to/new-dataset/
|— image_annotations.parquet (your extracted file)
|— dog_1.jpg
|— dog_2.jpg
|— dog_3.jpg
```

**Warning:** The parquet file must be named exactly `image_annotations.parquet` for Visual Layer to recognize it.

## Step 5: Create New Dataset (Fast!)

Now create your new dataset. Visual Layer will:

- ☐ Detect the existing `image_annotations.parquet` file
- ☐ Use the provided captions
- ☐ Complete much faster!

**Warning:** Note you will need to remove the captioning step from your dataset configuration to avoid conflicts.

## Understanding Relative Paths

**Critical Concept:** Filenames in the parquet file must be **relative** to the dataset directory location.

### Why Relative Paths?

Visual Layer looks for images relative to where the `image_annotations.parquet` file is located. Absolute paths won't work because they reference specific system locations that may not exist or may differ across environments.

### Examples

#### ☐ Wrong - Absolute Paths

filename: /home/ubuntu/images/dog\_1.jpg

filename: /mnt/data/dogs/dog\_2.jpg  
filename: /hostfs/workspace/dog\_3.jpg

**Problem:** These paths are tied to specific locations. If the parquet is in /new/location/, Visual Layer can't find /home/ubuntu/images/dog\_1.jpg.

## ❑ Correct - Relative Paths

### Scenario 1: Parquet in same directory as images

Dataset directory: /any/path/dataset/

```
|— image_annotations.parquet
|— dog_1.jpg
|— dog_2.jpg
|— dog_3.jpg
```

Filenames in parquet:

- dog\_1.jpg
- dog\_2.jpg
- dog\_3.jpg

### Scenario 2: Images in subdirectory

Dataset directory: /any/path/dataset/

```
|— image_annotations.parquet
|— images/
| |— dog_1.jpg
| |— dog_2.jpg
| |— dog_3.jpg
```

Filenames in parquet:

- images/dog\_1.jpg
- images/dog\_2.jpg
- images/dog\_3.jpg

## How the Script Handles Paths

The script automatically removes common system prefixes:

Original Path (from VL)	After Processing	Notes
/hostfs/home/ubuntu/images/dog.jpg	dog.jpg	Removed /hostfs/home/ubuntu/ir
/mnt/data/dogs/dog.jpg	dog.jpg	Custom prefix with --prefix /mnt/data/dogs/
/workspace/project/images/subdir/dog.jpg	dog.jpg or subdir/dog.jpg	Depends on prefix specified

**Tip: Key Insight:** The dataset directory can be anywhere on your system. The important thing is that filenames are relative to wherever you place the `image_annotations.parquet` file.

## Complete Example

Let's walk through a real example using dog images.

### Initial State: Dataset 1 (with captioning)

After running the first dataset pipeline:

```
Pipeline completed, captions generated
Dataset ID: b451f7c6-f911-4ceb-8b8a-dd6c1ebb50fd
```

#### Internal parquet file:

Location: `/.vl/tmp/b451f7c6-f911-4ceb-8b8a-dd6c1ebb50fd/input/metadata/image_annotations.parquet`  
Columns: `[filename, file_size_bytes, video, frame_timestamp, caption, captions_source_id, default_embedding_index, _vl_stats, stats]`  
Rows: 12

**Sample data:** `| filename | caption | |`  
`| /hostfs/home/ubuntu/images/dog_1.jpg | "A Golden Retriever sitting on grass. The dog has a friendly expression..." |`  
`| /hostfs/home/ubuntu/images/dog_2.jpg | "A playful puppy with a red collar running through a park..." |`

### Processing with Script

```
Run extraction script
python3 process_annotations.py \
 /.vl/tmp/b451f7c6-.../input/metadata/image_annotations.parquet \
 -o /home/ubuntu/new-dataset/image_annotations.parquet
```

#### Output parquet file:

Location: `/home/ubuntu/new-dataset/image_annotations.parquet`  
Columns: `[filename, caption]`  
Rows: 12

**Processed data:** `| filename | caption | |`  
`| dog_1.jpg | "A Golden Retriever sitting on grass. The dog has a friendly expression..." |`  
`| dog_2.jpg | "A playful puppy with a red collar running through a park..." |`

### Dataset 2 (without captioning - fast!)

```
Directory structure
/home/ubuntu/new-dataset/
├── image_annotations.parquet (extracted file)
├── dog_1.jpg
```

```
| dog_2.jpg
| ... (all 12 images)
```

```
Create new dataset
Visual Layer detects image_annotations.parquet
```

## Troubleshooting

### Images Not Found

**Error:** “Could not find image at path: dog\_1.jpg”

**Cause:** Filenames in parquet don’t match actual file locations.

**Solutions:** 1. Verify parquet is in the same directory as images 2. Check that filenames match exactly (case-sensitive) 3. Inspect parquet contents to verify paths are relative

### Checking Parquet Contents

```
View parquet file contents
python3 -c "

df = pd.read_parquet('image_annotations.parquet')
print('Columns:', df.columns.tolist())
print('\nFirst 5 filenames:')
print(df['filename'].head().tolist())
"
```

**Expected output:**

Columns: ['filename', 'caption']

First 5 filenames:

['dog\_1.jpg', 'dog\_2.jpg', 'dog\_3.jpg', 'dog\_4.jpg', 'dog\_5.jpg']

### Captions Not Being Used

**Issue:** Visual Layer is still generating captions even though image\_annotations.parquet exists.

**Solutions:** 1. Verify filename is exactly image\_annotations.parquet (not image\_annotations\_processed) 2. Ensure file is in the correct location relative to images 3. Check that parquet file has both filename and caption columns

### Script Errors

**Error:** “Missing required columns: [‘caption’]”

**Cause:** Source parquet file doesn’t contain caption data.

**Solution:** The source dataset must have had captions generated. Check if captioning was enabled in the original pipeline.

## Related Documentation

- [Preparing Annotation Files](#) - Format requirements for annotation files
- [Annotations Overview](#) - Complete guide to importing annotations
- [Creating Datasets](#) - Dataset creation fundamentals

By following this workflow, you can significantly reduce dataset creation time when working with the same images across multiple datasets or configurations. The initial investment of generating captions once pays off through faster subsequent dataset creations.

## Caption Extraction Script

**Caption Extraction Script** Complete Python script for extracting filename and caption columns from Visual Layer's internal parquet files.

**Back to Usage Guide** See the main guide for detailed usage instructions, workflow, and examples.

## Installation

```
pip install pandas pyarrow
```

## Quick Usage

```
Basic usage
python3 process_annotations.py /.vl/tmp/[dataset-id]/input/metadata/image_annotations.

Specify output location
python3 process_annotations.py input.parquet -o /path/to/output.parquet

Custom prefix removal
python3 process_annotations.py input.parquet --prefix /custom/prefix
```

## Script Code

```
#!/usr/bin/env python3
"""
Process parquet annotation files to extract filename and caption columns.
Removes path prefixes from filenames.
"""

from pathlib import Path

def process_parquet(input_path, output_path=None, prefix_to_remove='/hostfs'):
 """
 Process a parquet file to extract filename and caption columns.

 Args:
 input_path: Path to input parquet file
 output_path: Path to output parquet file (optional)
 prefix_to_remove: Prefix to remove from filenames (default: '/hostfs')

 Returns:
 """
```



```

 Path to output file
 """
 # Validate input file
 input_file = Path(input_path)
 if not input_file.exists():
 raise FileNotFoundError(f"Input file not found: {input_path}")

 if not input_file.suffix == '.parquet':
 raise ValueError(f"Input file must be a parquet file, got: {input_file.suffix}")

 # Determine output path
 if output_path is None:
 output_file = input_file.parent / f"{input_file.stem}_processed.parquet"
 else:
 output_file = Path(output_path)

 print(f"Reading parquet file: {input_file}")

 # Read parquet file
 try:
 df = pd.read_parquet(input_file)
 except Exception as e:
 raise RuntimeError(f"Failed to read parquet file: {e}")

 print(f"Original shape: {df.shape}")
 print(f"Columns: {df.columns.tolist()}")

 # Validate required columns exist
 required_columns = ['filename', 'caption']
 missing_columns = [col for col in required_columns if col not in df.columns]
 if missing_columns:
 raise ValueError(f"Missing required columns: {missing_columns}")

 # Select only filename and caption columns
 df = df[['filename', 'caption']]

 # Remove prefix from filename
 print(f"Removing prefix '{prefix_to_remove}' from filenames...")
 df['filename'] = df['filename'].apply(
 lambda x: x.replace(prefix_to_remove, '', 1)
 if isinstance(x, str) and x.startswith(prefix_to_remove)
 else x
)

 # Show sample of processed data
 print(f"\nProcessed shape: {df.shape}")
 print(f"\nSample filenames after processing:")

```

```

print(df['filename'].head(3).tolist())

Save to output file
print(f"\nSaving to: {output_file}")
df.to_parquet(output_file, index=False)

print(f"✓ Successfully processed {len(df)} rows")
print(f"✓ Output saved to: {output_file}")

return output_file

def main():
 parser = argparse.ArgumentParser(
 description='Process parquet annotation files to extract filename and caption',
 formatter_class=argparse.RawDescriptionHelpFormatter,
 epilog="""
Examples:
%(prog)s input.parquet
%(prog)s input.parquet -o output.parquet
%(prog)s input.parquet --prefix //hostfs/mnt
 """
)

 parser.add_argument(
 'input',
 help='Path to input parquet file'
)

 parser.add_argument(
 '-o', '--output',
 help='Path to output parquet file (default: <input>_processed.parquet)'
)

 parser.add_argument(
 '--prefix',
 default='/hostfs',
 help='Prefix to remove from filenames (default: /hostfs)'
)

 args = parser.parse_args()

 try:
 process_parquet(args.input, args.output, args.prefix)
 return 0
 except Exception as e:
 print(f"Error: {e}", file=sys.stderr)

```

```
return 1
```

```
if __name__ == '__main__':
 sys.exit(main())
```

## Curating Datasets

## User Tags

**How This Helps** Tagging makes it easy to group, track, and manage data for training, QA, relabeling, or experimentation. Whether you're cleaning data, preparing a subset, or tracking annotation status — tags let you move faster and stay organized.

Tags help you **quickly organize, filter, and take action** on relevant items in your dataset. They can be applied across Clusters, Images, or Objects, and used to streamline curation, quality review, or experimental workflows.

Use cases include:

- **Data Curation:** Group items for training, validation, or internal evaluation. Tags like `train`, `test`, `representative` help define clear splits.
- **Data Cleaning:** Identify problematic items with tags like `low_quality`, `duplicate`, or `needs_review`.
- **Workflow Management:** Add review state tags such as `approved`, `rejected`, `for-review`, or priority labels like `urgent`.
- **Experiment Tracking:** Label data used in experiments (`exp-103`, `baseline-set`) for traceability and auditability.

## Tag Multiple Items

You can assign tags to many items at once across views.

1. Navigate to a dataset and select Clusters, Images, or Objects of interest.
2. Click the **Selected Items** icon in the top right.
3. Click **Manage Tags** and choose from existing tags or create new ones.

## View and Remove Tags

You can view and edit tags in the Assigned Tags panel.

1. Select your items and open the **Assigned Tags** panel.
2. Click a tag to remove it from any selected items — even if it's not applied to all of them.

## Tag Individual Images

You can also tag single images directly in the Image Details Page.

**How to Apply Tags to an Image** Click **Assign User Tags** in the right-hand pane of the Image Details Page. You can create a new tag or apply an existing one.

Assign tags to a specific image from the side panel

Select or create a new tag

View or remove assigned tags directly in the right pane

To remove a tag, click the **X** next to its label.

**Tip:** You can mix and match tag workflows — start with bulk tagging, refine at the image level, or export tagged items for review, relabeling, or exclusion.

## Using Exported VL Metadata

### Step 1: Extracting Data Using Visual Layer's API

First, let's set up a Python class to handle the API calls for exporting datasets.

#### Simplified Python API

We'll use the following class to encapsulate the API calls:

```
from typing import Any, Dict

class DatasetExporter:
 def __init__(self, base_url: str):
 self.base_url = base_url.rstrip("/")
 self.headers = {"Accept": "application/json, text/plain, */*"}

 def initiate_export(
 self,
 dataset_id: str,
 file_name: str,
 export_format: str = "json",
 include_images: bool = False,
) -> Dict[str, Any]:
 """
 Initiate an export of a dataset.

 Args:
 dataset_id: The ID of the dataset to export
 file_name: Name of the export file
 export_format: Format of the export (default: 'json')
 include_images: Whether to include images in export (default: False)

 Returns:
 Dict containing the export task information
 """
 url = f"{self.base_url}/api/v1/dataset/{dataset_id}/export_context_async"
 params = {
 "file_name": file_name,
 "export_format": export_format,
 "include_images": str(include_images).lower(),
 }

 response = requests.get(url, headers=self.headers, params=params)
 response.raise_for_status()
 return response.json()
```

```

def check_export_status(
 self, dataset_id: str, export_task_id: str
) -> Dict[str, Any]:
 """
 Check the status of an export task.

 Args:
 dataset_id: The ID of the dataset
 export_task_id: The ID of the export task to check

 Returns:
 Dict containing the status information
 """
 url = f"{self.base_url}/api/v1/dataset/{dataset_id}/export_status"
 params = {"export_task_id": export_task_id}

 response = requests.get(url, headers=self.headers, params=params)
 response.raise_for_status()
 return response.json()

def wait_for_export(
 self,
 dataset_id: str,
 export_task_id: str,
 check_interval: int = 5,
 timeout: int = 300,
) -> Dict[str, Any]:
 """
 Wait for an export task to complete.

 Args:
 dataset_id: The ID of the dataset
 export_task_id: The ID of the export task to check
 check_interval: Time in seconds between status checks (default: 5)
 timeout: Maximum time to wait in seconds (default: 300)

 Returns:
 Dict containing the final status information

 Raises:
 TimeoutError: If the export doesn't complete within the timeout period
 """
 start_time = time.time()
 while True:
 status = self.check_export_status(dataset_id, export_task_id)

 if status["status"] == "COMPLETED":

```

```

 return status

 if status["status"] == "FAILED":
 raise Exception(f"Export failed: {status.get('result_message')}")

 if time.time() - start_time > timeout:
 raise TimeoutError("Export timed out")

 time.sleep(check_interval)

def download_export(self, download_uri: str, output_path: str) -> None:
 """
 Download the exported file from the given URI.

 Args:
 download_uri: The URI path from the export status
 output_path: Local path where the file should be saved

 Returns:
 None
 """
 url = f"{self.base_url}{download_uri}"
 response = requests.get(url, headers=self.headers, stream=True)
 response.raise_for_status()

 with open(output_path, "wb") as f:
 for chunk in response.iter_content(chunk_size=8192):
 f.write(chunk)

```

## Usage Example

You'll need to replace the url, dataset\_id, and file\_name with your own values:

```

def main():
 # Replace with your own
 url = "http://3.129.25.115:2080"
 dataset_id = "4cd4d9a2-e793-11ef-80a1-0242ac13000a"
 file_name = "export.zip"

 exporter = DatasetExporter(url)

 try:
 print("Initiating export...")
 export_task = exporter.initiate_export(dataset_id, file_name)
 print(f"Export task created: {export_task}")

```



```

print("Waiting for export to complete...")
final_status = exporter.wait_for_export(dataset_id, export_task["id"])

print("\nExport completed!")
print(f"Download URI: {url}{final_status['download_uri']}")

print("Downloading file...")
exporter.download_export(final_status["download_uri"], file_name)
print(f"File downloaded successfully to: {file_name}")

print("Unzipping file...")
output_dir = file_name.replace('.zip', '')
os.makedirs(output_dir, exist_ok=True)
with zipfile.ZipFile(file_name, "r") as zip_ref:
 zip_ref.extractall(output_dir)
print(f"File unzipped successfully to: {output_dir}")

except Exception as e:
 print(f"Error: {e}")

if __name__ == "__main__":
 main()

```

## Expected Output

Initiating export...

Export task created: {'id': 'cf8b3e78-cb20-4954-a782-436a03006dab', 'dataset\_id': '4cd4de793-11ef-80a1-0242ac13000a', 'created\_at': '2025-02-10T14:41:14.390429', 'download\_uri': 'http://3.129.25.115:2080/cdn/4cd4d9a2-e793-11ef-80a1-0242ac13000a/images/cf8b3e78-cb20-4954-a782-436a03006dab/cf8b3e78-cb20-4954-a782-436a03006dab.zip'}

Waiting for export to complete...

Export completed!

Download URI: http://3.129.25.115:2080/cdn/4cd4d9a2-e793-11ef-80a1-0242ac13000a/images/cf8b3e78-cb20-4954-a782-436a03006dab/cf8b3e78-cb20-4954-a782-436a03006dab.zip

Downloading file...

File downloaded successfully to: export.zip

Unzipping file...

File unzipped successfully to: export

## Step 2: Parsing Exported Metadata

After extracting your data, you'll have a metadata.json file. Let's learn how to parse and analyze it.

## Loading the Exported Metadata

First, let's load the exported metadata into a pandas dataframe:

```
with open("food101/metadata.json") as f:
 data = json.load(f)

This is the dataset level information
info_df = pd.DataFrame([data["info"]])

This is the image level information
media_items_df = pd.json_normalize(data["media_items"])
```

## View the Dataset Level Information

info\_df

schema_name	dataset_description	dataset_url	export_time	dataset_created_time	exported_items	total_media_items
1.1	food101 Exported from food101 at Visual Layer	Link	2025-02-10T14:29:53.030065Z	2024-12-03T06:55:27.125598Z	Dickson	118

## Get Image Level Details

Each row in the dataframe corresponds to an image:

```
media_items_df
```

media_id	media_type	file_name	file_size	frequency	height	width	cluster_id	metadata_items
d5227901-22c9-4744-a264-407d9671aa4a	image	548954.jpg	32,00KB	0417	512	512	fbcad8ef-d863-46c9-83b7-1a3bd85e2e2b	{'type': 'issue', 'properties': {'is-true_type':...
2546c70a-e0a4-4bfb-ac59-e2895bb96456	image	548254.jpg	32,00KB	0605	512	512	fbcad8ef-d863-46c9-83b7-1a3bd85e2e2b	{'type': 'issue', 'properties': {'is-true_type':...
45c226e0-daba-4ca8-8eac-fec9d490ea36	image	835983.jpg	52,00KB	0355	384	384	5bc63415-76d8-49ec-975a-9a021bf98779	{'type': 'issue', 'properties': {'is-true_type':...

## Step 3: Analyzing Your Data

### Filter By Uniqueness Score

By filtering by uniqueness score, you can get a representative sample of the images in the dataset:

```
UNIQUENESS_SCORE_THRESHOLD = 0.85

coreset_df = media_items_df[
 media_items_df["uniqueness_score"] > UNIQUENESS_SCORE_THRESHOLD
]
coreset_df
```

This will show media items with a uniqueness score greater than **0.85**.

### Get Duplicate Images

The `metadata_items` column contains a list of issues for each image. We can filter for images with duplicate issues above a certain confidence threshold:

```
def has_duplicate_issue(metadata_items, confidence_threshold=0.8):
 if not isinstance(metadata_items, list):
 return False

 for item in metadata_items:
 if (
 item.get("type") == "issue"
 and item.get("properties", {}).get("issue_type") == "duplicates"
 and item.get("properties", {}).get("confidence", 0) > confidence_threshold
):
 return True
 return False

Replace with your confidence threshold
CONFIDENCE_THRESHOLD = 0.8

duplicate_df = media_items_df[
 media_items_df["metadata_items"].apply(
 lambda x: has_duplicate_issue(x, confidence_threshold=CONFIDENCE_THRESHOLD)
)
]

duplicate_df
```

This table includes images with duplicate issues above a confidence threshold of 0.8.

### Get Mislabels

We can filter for images with mislabel issues above a certain confidence threshold:

```

def has_mislabel_issue(metadata_items, confidence_threshold=0.8):
 if not isinstance(metadata_items, list):
 return False

 for item in metadata_items:
 if (
 item.get("type") == "issue"
 and item.get("properties", {}).get("issue_type") == "mislabels"
 and item.get("properties", {}).get("confidence", 0) > confidence_threshold
):
 return True
 return False

Replace with your confidence threshold
CONFIDENCE_THRESHOLD = 0.8

mislabel_df = media_items_df[
 media_items_df["metadata_items"].apply(
 lambda x: has_mislabel_issue(x, confidence_threshold=CONFIDENCE_THRESHOLD)
)
]

mislabel_df

```

## Step 4: Converting Video Frame Data to CSV

If your exported JSON contains video frame data, here's how to parse it into CSV format:

### Example Video Frame JSON Structure

```

{
 "info": {
 "schema_version": "1.1",
 "dataset": "Extreme",
 "description": "Exported from Extreme at Visual Layer",
 "dataset_url": "http://app.visual-layer.com/dataset/42e3b7ae-dd5e-11ef-9fca-1a226f",
 "export_time": "2025-02-15T18:23:34.164184",
 "dataset_creation_time": "2025-01-24T18:35:57.719308",
 "exported_by": "Danny Bickson",
 "total_media_items": 56
 },
 "media_items": [
 {
 "media_id": "8621054d-1d23-4ca7-a7ca-65feb2c83eb1",
 "media_type": "video_frame",

```

```

"file_name": "output_000001.jpg",
"file_path": "7370082532203988000_7450129782785756438_2024-12-19_14:28:36.mp4/ou
"file_size": "48.27KB",
"uniqueness_score": 0.4989460029187489,
"height": 720,
"width": 576,
"url": "http://app.visual-layer.com/dataset/42e3b7ae-dd5e-11ef-9fca-1a226f3de670
"cluster_id": "49997382-b7c3-4c83-83a3-fa4e0a586f2b",
"metadata_items": [
 {
 "type": "video_info",
 "properties": {
 "video_name": "7370082532203988000_7450129782785756438_2024-12-19_14:28:36
 "frame_timestamp": 0.0
 }
 },
 {
 "type": "issue",
 "properties": {
 "issue_type": "duplicates",
 "confidence": 0.977624,
 "duplicate_group_id": "35395121-5cb1-4a92-906f-dd4cb9ff62d3",
 "duplicate_threshold": 0.96
 }
 }
]
}

```

## Converting Video Frame Data to CSV

```

Load JSON file
with open("metadata 8.json", "r") as f:
 data = json.load(f)

Prepare list for DataFrame
records = []

Iterate over media items
for item in data.get("media_items", []):
 video_name = None
 frame_timestamp = None
 categories = []
 file_name = item.get("file_name", "")

```

```

for metadata in item.get("metadata_items", []):
 if metadata["type"] == "video_info":
 video_name = metadata["properties"].get("video_name")
 frame_timestamp = metadata["properties"].get("frame_timestamp")
 elif metadata["type"] == "image_label":
 categories.append(metadata["properties"].get("category_name"))

Extract frame number from file name
match = re.search(r'output_(\d+).jpg', file_name)
frame_number = int(match.group(1)) if match else None

Append record with categories as a list
records.append({
 "video_name": video_name,
 "file_name": file_name,
 "frame_number": frame_number,
 "time_in_video": frame_timestamp,
 "categories": categories
})

Convert to DataFrame
df = pd.DataFrame(records)
print(df.tail())

```

## Step 5: Converting Object Detection Data to CSV

For datasets with bounding box annotations and user tags, here's how to convert them to CSV format:

### Example Object Detection JSON Structure

```

{
 "info": {
 "schema_version": "1.1",
 "dataset": "General pics",
 "description": "Exported from General pics at Visual Layer",
 "total_media_items": 2
 },
 "media_items": [
 {
 "media_id": "01305347-b812-4717-99a3-aab54a206d4c",
 "file_name": "000000046252.jpg",
 "width": 640,
 "height": 480,

```

```

 "metadata_items": [
 {
 "type": "object_label",
 "properties": {
 "category_name": "person",
 "bbox": [148, 177, 127, 188],
 "metadata_items": [
 {
 "type": "user_tag",
 "properties": {"tag_name": "test-tags"}
 }
]
 }
 },
 {
 "type": "object_label",
 "properties": {
 "category_name": "shirt",
 "bbox": [192, 194, 75, 65]
 }
 }
]
 }
}

```

## Converting Object Detection Data to CSV

```

Load JSON file
input_file = "metadata_5.json" # Change this to your actual file path
output_file = "output.csv" # Output CSV file name

with open(input_file, "r") as f:
 data = json.load(f)

Prepare CSV file
header = ["filename", "x", "y", "w", "h", "label", "user tag"]
rows = []

Extract data
for media in data.get("media_items", []):
 filename = media.get("file_name", "")
 for item in media.get("metadata_items", []):
 if item.get("type") == "object_label":
 properties = item.get("properties", {})

```

```

bbox = properties.get("bbox", [0, 0, 0, 0])
label = properties.get("category_name", "")
user_tag = ""

Extract user tag if available
for meta in properties.get("metadata_items", []):
 if meta.get("type") == "user_tag":
 user_tag = meta.get("properties", {}).get("tag_name", "")
 break

Append row
rows.append([filename, bbox[0], bbox[1], bbox[2], bbox[3], label, user_tag])

Write to CSV
with open(output_file, "w", newline="") as f:
 writer = csv.writer(f)
 writer.writerow(header)
 writer.writerows(rows)

print(f"CSV file saved to {output_file}")

```

### Expected CSV Output

Filename	X	Y	W	H	Label	User Tag
000000046252.jpg	148	177	127	188	Person	test-tags
000000046252.jpg	192	194	75	65	Shirt	test-tags
000000012639.jpg	28	368	80	118	Vest	test-tags

## Step 6: Analyzing Video Frame Similarity

For datasets with video frames, you can analyze similarity between frames to understand which videos are most similar. This is useful for identifying duplicate content, finding related videos, or grouping similar content.

### Creating a Video Frame Data Table

First, let's extract video frame information along with their similarity data:

```

from collections import defaultdict

Load the exported metadata
with open("metadata.json", "r") as f:
 data = json.load(f)

```



```

Extract video frame data with similarity information
frame_records = []

for item in data.get("media_items", []):
 if item.get("media_type") == "video_frame":
 # Extract basic frame info
 media_id = item.get("media_id")
 file_name = item.get("file_name")

 # Extract video info
 video_name = None
 frame_timestamp = None
 similar_frames = []

 for metadata in item.get("metadata_items", []):
 if metadata.get("type") == "video_info":
 video_name = metadata.get("properties", {}).get("video_name")
 frame_timestamp = metadata.get("properties", {}).get("frame_timestamp")
 elif metadata.get("type") == "issue" and metadata.get("properties", {}).get("confidence") > 0:
 # Extract similarity information
 confidence = metadata.get("properties", {}).get("confidence")
 duplicate_group_id = metadata.get("properties", {}).get("duplicate_group_id")
 duplicate_threshold = metadata.get("properties", {}).get("duplicate_threshold")

 similar_frames.append({
 "confidence": confidence,
 "group_id": duplicate_group_id,
 "threshold": duplicate_threshold
 })

 frame_records.append({
 "media_id": media_id,
 "file_name": file_name,
 "video_name": video_name,
 "frame_timestamp": frame_timestamp,
 "similar_frames": similar_frames,
 "cluster_id": item.get("cluster_id")
 })

Create DataFrame
frames_df = pd.DataFrame(frame_records)
print(f"Total video frames: {len(frames_df)}")
print(frames_df.head())

```

## Creating a Video Similarity Aggregation Table

Now let's create a table that shows video-to-video similarity. This analysis only compares frames between different videos, excluding similarities within the same video:

```
Create a mapping of frames to their duplicate groups
frame_to_groups = defaultdict(set)
group_to_frames = defaultdict(set)

for idx, row in frames_df.iterrows():
 media_id = row['media_id']
 video_name = row['video_name']

 for similar_frame in row['similar_frames']:
 group_id = similar_frame['group_id']
 confidence = similar_frame['confidence']

 # Only consider high-confidence similarities (>= 0.95)
 if confidence >= 0.95:
 frame_to_groups[media_id].add(group_id)
 group_to_frames[group_id].add((media_id, video_name, confidence))

Create video-to-video similarity matrix
video_similarity_records = []

Get all unique videos
all_videos = frames_df['video_name'].unique()

for video_a in all_videos:
 for video_b in all_videos:
 if video_a != video_b:
 # Get frames from both videos
 frames_a = frames_df[frames_df['video_name'] == video_a]['media_id'].tolist()
 frames_b = frames_df[frames_df['video_name'] == video_b]['media_id'].tolist()

 # Find similarities between frames from different videos
 inter_video_similarities = []
 shared_groups = set()

 for frame_a in frames_a:
 for frame_b in frames_b:
 groups_a = frame_to_groups.get(frame_a, set())
 groups_b = frame_to_groups.get(frame_b, set())

 # Find groups that contain both frames (cross-video similarity)
 common_groups = groups_a.intersection(groups_b)
 if common_groups:
 shared_groups.update(common_groups)
```

```

 # For each shared group, get the similarity confidence
 # We know these frames are from different videos by constructi
 for group_id in common_groups:
 frames_in_group = group_to_frames[group_id]
 # Get confidence scores for frames from both videos in thi
 confidences_in_group = [conf for _, _, conf in frames_in_g
 if confidences_in_group:
 # Use the average confidence for this similarity group
 inter_video_similarities.append(sum(confidences_in_gro

 if inter_video_similarities:
 avg_similarity = sum(inter_video_similarities) / len(inter_video_simil
 num_similar_frames = len(shared_groups)

 video_similarity_records.append({
 "video_a": video_a,
 "video_b": video_b,
 "average_similarity": round(avg_similarity, 4),
 "number_of_similar_frames": num_similar_frames
 })

Create the final similarity DataFrame
similarity_df = pd.DataFrame(video_similarity_records)

Remove duplicates (keep only one direction of each pair)
similarity_df = similarity_df[similarity_df['video_a'] < similarity_df['video_b']]

Sort by average similarity (descending)
similarity_df = similarity_df.sort_values('average_similarity', ascending=False)

print(f"Video pairs with similarities: {len(similarity_df)}")
print("\nTop 10 most similar video pairs:")
print(similarity_df.head(10))

```

## Alternative Approach Using Clustering

You can also use the `cluster_id` field to find similar frames:

```

Create similarity table using cluster IDs
cluster_similarity_records = []

Group frames by cluster
cluster_groups = frames_df.groupby('cluster_id')

for cluster_id, cluster_frames in cluster_groups:
 if len(cluster_frames) > 1: # Only clusters with multiple frames
 videos_in_cluster = cluster_frames['video_name'].unique()

```

```

Create pairs of videos in the same cluster
for i, video_a in enumerate(videos_in_cluster):
 for video_b in videos_in_cluster[i+1:]:
 frames_a_in_cluster = len(cluster_frames[cluster_frames['video_name']
 frames_b_in_cluster = len(cluster_frames[cluster_frames['video_name']

 cluster_similarity_records.append({
 "video_a": video_a,
 "video_b": video_b,
 "cluster_id": cluster_id,
 "frames_from_video_a": frames_a_in_cluster,
 "frames_from_video_b": frames_b_in_cluster,
 "total_frames_in_cluster": len(cluster_frames)
 })

Create DataFrame
cluster_similarity_df = pd.DataFrame(cluster_similarity_records)

Aggregate by video pair
video_cluster_summary = cluster_similarity_df.groupby(['video_a', 'video_b']).agg({
 'cluster_id': 'count', # Number of shared clusters
 'frames_from_video_a': 'sum',
 'frames_from_video_b': 'sum'
}).rename(columns={'cluster_id': 'shared_clusters'}).reset_index()

print("\nVideo similarity based on shared clusters:")
print(video_cluster_summary.sort_values('shared_clusters', ascending=False).head())

```

## Export Results to CSV

Finally, export your analysis results:

```

Export the main similarity table
similarity_df.to_csv("video_similarity_analysis.csv", index=False)

Export the frame-level data
frames_df.to_csv("video_frames_with_similarity.csv", index=False)

Export the cluster-based analysis
video_cluster_summary.to_csv("video_cluster_similarity.csv", index=False)

print("Analysis complete! Files exported:")
print("- video_similarity_analysis.csv")
print("- video_frames_with_similarity.csv")
print("- video_cluster_similarity.csv")

```

## Expected Output

The analysis will produce tables like:

**Video Similarity Table:**

video_a	video_b	average_similarity	number_of_similar_frames
video1.mp4	video2.mp4	0.9850	15
video3.mp4	video4.mp4	0.9720	8

**Video Cluster Summary:**

video_a	video_b	shared_clusters	frames_from_video_a	frames_from_video_b
video1.mp4	video2.mp4	5	12	18

This analysis helps you identify: - Which videos have the most similar content - How many frames are similar between video pairs - The confidence level of similarities - Clusters of related video content

## Step 7: Analyzing Mislabel Issues

For datasets with mislabel detection, you can analyze which labels are potentially incorrect and what the suggested corrections are. This is valuable for improving dataset quality and understanding systematic labeling issues.

### Understanding Mislabel Structure

Mislabel issues are stored within object labels and have this structure:

```
{
 "type": "issue",
 "properties": {
 "issue_type": "mislabels",
 "issues_description": "3D-CONTAMINATION",
 "confidence": 0.8059
 }
}
```

Where: - issues\_description: The suggested correct label - confidence: How confident the system is about the mislabel (0-1)

### Comprehensive Mislabel Analysis

Here's how to extract and analyze all mislabel information:

```
from collections import defaultdict, Counter

Load the exported metadata
with open("metadata.json", "r") as f:
 data = json.load(f)
```

```

print(f"Dataset: {data['info']['dataset']}")
print(f"Total media items: {data['info']['total_media_items']}")

Extract mislabel information
mislabel_records = []
total_objects = 0
objects_with_mislabels = 0

for item in data.get("media_items", []):
 media_id = item.get("media_id")
 file_name = item.get("file_name")

 # Look for object labels with mislabel issues
 for metadata in item.get("metadata_items", []):
 if metadata.get("type") == "object_label":
 total_objects += 1
 object_props = metadata.get("properties", {})
 original_category = object_props.get("category_name")
 bbox = object_props.get("bbox", [])

 # Check for mislabel issues in this object
 object_mislabels = []
 for obj_metadata in object_props.get("metadata_items", []):
 if (obj_metadata.get("type") == "issue" and
 obj_metadata.get("properties", {}).get("issue_type") == "mislabels"):

 mislabel_props = obj_metadata.get("properties", {})
 suggested_label = mislabel_props.get("issues_description")
 confidence = mislabel_props.get("confidence")

 object_mislabels.append({
 "suggested_label": suggested_label,
 "confidence": confidence
 })

 if object_mislabels:
 objects_with_mislabels += 1
 for mislabel in object_mislabels:
 mislabel_records.append({
 "media_id": media_id,
 "file_name": file_name,
 "original_label": original_category,
 "suggested_label": mislabel["suggested_label"],
 "confidence": mislabel["confidence"],
 "bbox": bbox
 })

```

```

print(f"Total objects analyzed: {total_objects}")
print(f"Objects with mislabel issues: {objects_with_mislabels}")
print(f"Total mislabel records: {len(mislabel_records)}")

Create DataFrame for analysis
mislabels_df = pd.DataFrame(mislabel_records)

```

## Mislabel Statistics and Analysis

```

1. How many mislabels total?
print(f"TOTAL MISLABELS: {len(mislabels_df)}")

2. Confidence distribution
print(f"\nCONFIDENCE DISTRIBUTION:")
print(f"Average confidence: {mislabels_df['confidence'].mean():.4f}")
print(f"Min confidence: {mislabels_df['confidence'].min():.4f}")
print(f"Max confidence: {mislabels_df['confidence'].max():.4f}")
print(f"High confidence (>0.8): {len(mislabels_df[mislabels_df['confidence'] > 0.8])}")
print(f"Medium confidence (0.7-0.8): {len(mislabels_df[(mislabels_df['confidence'] >= 0.7) & (mislabels_df['confidence'] < 0.8)])}")
print(f"Lower confidence (<0.7): {len(mislabels_df[mislabels_df['confidence'] < 0.7])}")

3. From which classes are the mislabels?
print(f"\nMISLABELED CLASSES (Original Labels):")
original_label_counts = mislabels_df['original_label'].value_counts()
print(original_label_counts.head(10))

4. What are the suggested corrections?
print(f"\nSUGGESTED CORRECTIONS (Most Common):")
suggested_label_counts = mislabels_df['suggested_label'].value_counts()
print(suggested_label_counts.head(10))

5. Most common mislabel corrections (original -> suggested)
print(f"\nMOST COMMON MISLABEL CORRECTIONS:")
correction_pairs = mislabels_df.groupby(['original_label', 'suggested_label']).size().reset(index=True, name='count')
correction_pairs = correction_pairs.sort_values('count', ascending=False)
print("Original Label -> Suggested Label (Count)")
for _, row in correction_pairs.head(15).iterrows():
 print(f"{row['original_label']} -> {row['suggested_label']} ({row['count']})")

```

## High-Confidence Corrections Analysis

```

Analyze high-confidence corrections for the most reliable suggestions
high_conf_corrections = correction_pairs[
 correction_pairs.apply(lambda x: mislabels_df[
 (mislabels_df['original_label'] == x['original_label']) &

```

```

 (mislabels_df['suggested_label'] == x['suggested_label'])
]['confidence'].mean() > 0.8, axis=1)
]

print(f"High confidence corrections (>0.8): {len(high_conf_corrections)}")
print("\nTop high-confidence corrections:")
for _, row in high_conf_corrections.head(10).iterrows():
 avg_conf = mislabels_df[
 (mislabels_df['original_label'] == row['original_label']) &
 (mislabels_df['suggested_label'] == row['suggested_label'])
]['confidence'].mean()
 print(f"{row['original_label']} -> {row['suggested_label']} ({row['count']} times,

```

## Summary Statistics and Export

```

Summary statistics
print(f"\nSUMMARY STATISTICS:")
print(f"Unique original labels with issues: {len(mislabels_df['original_label'].unique())}")
print(f"Unique suggested labels: {len(mislabels_df['suggested_label'].unique())}")
print(f"Unique correction pairs: {len(correction_pairs)}")
print(f"Images with mislabels: {len(mislabels_df['file_name'].unique())}")

Export results
mislabels_df.to_csv("mislabel_analysis.csv", index=False)
correction_pairs.to_csv("mislabel_corrections.csv", index=False)

print("\nFiles exported:")
print("- mislabel_analysis.csv: Detailed mislabel data")
print("- mislabel_corrections.csv: Correction patterns summary")

```

## Expected Output

The analysis will produce results like:

TOTAL MISLABELS: 422

CONFIDENCE DISTRIBUTION:

Average confidence: 0.8059

High confidence (>0.8): 120

Medium confidence (0.7-0.8): 300

MISLABELED CLASSES (Original Labels):

3D-SHINYMARK	52
3D-SCRATCH	40
3D-CONTAMINATION	38
3D-SHINYLINE	28
3D-DENT	28



#### MOST COMMON MISLABEL CORRECTIONS:

Bandslot\_discolor -> BS-DISCOLOR-B (20)

3D-SHINYMARK -> 3D-CONTAMINATION (18)

3D-SHINYLINE -> 3D-CONTAMINATION (12)

3D-SCRATCH -> 3D-CONTAMINATION (10)

3D-CHIP -> 3D2-CHIP (10)

This analysis helps you: - **Identify systematic labeling issues**: Classes that are frequently mislabeled - **Prioritize corrections**: Focus on high-confidence suggestions first - **Understand label confusion**: See which classes are commonly confused with each other - **Improve dataset quality**: Apply suggested corrections to improve training data - **Analyze labeling patterns**: Understand if certain types of objects are consistently mislabeled

## Next Steps

Now that you know how to work with exported metadata files, you can:

1. **Analyze data quality** by filtering for different issue types and confidence thresholds
2. **Create data subsets** using uniqueness scores and clustering information
3. **Export to different formats** for use in other tools and platforms
4. **Build automated workflows** to process exported datasets at scale
5. **Analyze video similarity** to find duplicate content, group related videos, or identify the most unique content
6. **Analyze mislabel issues** to identify systematic labeling problems and prioritize corrections

For more information on exporting datasets, see our [Export Dataset](#) guide.

## Managing Datasets

## Save Views

**How This Helps** Saved Views let you freeze and reuse queries across large datasets. This makes it easy to organize, revisit, or share filtered subsets without manually recreating queries each time.

Saved Views are dynamic, named queries that live inside a Dataset. They allow you to define a set of filters, searches, or similarity conditions and revisit that exact data slice any time. The underlying data stays current—so the view always reflects the latest results that match your criteria.

Key characteristics of Saved Views:

- Views **don't duplicate or store data**—they act as a persistent filter definition.
- Views are **dataset-specific** and visible to all collaborators.
- Each View uses a **View Descriptor**, which includes filters and/or search logic.
- Views appear in both the **Views tab** and within the **Explore tab** of a dataset.
- Views are currently **immutable**—they must be deleted and recreated to be changed.

## Why Use Views

Saving frequently used queries helps teams stay aligned, reduce repetition, and scale their workflows across different objectives or team members.

Use Views to:

- **Boost efficiency**— Save time by avoiding repeated filter configuration
- **Ensure consistency**— Standardize how teams work with data subsets
- **Create flexible workflows**— Define views for tasks like Training Candidates, Outliers, or QC Required

## Save a Query as a View

You can save any filter combination or search query as a View.

### Step: Apply Filters or Search

Open a dataset and apply any filters, label constraints, or similarity searches.

### Step: Click 'Save as View'

Once your query is ready, click the **Save as View** button.

### Step: Name the View

Enter a descriptive, unique name for the view and click **Create View**.

## Explore a Saved View

Once created, Views can be accessed at any time:

### Step: Open the Views Tab

Navigate to the dataset's **Views** tab.

### Step: Select the View

Click on the desired View to load the matched results.

## Share a View

You can easily share a View with another user on your team:

### Step: Click 'Share View'

Go to the **Views** tab and locate the View you'd like to share.

### Step: Enter Email

Click **Share View**, enter your colleague's email, and click **\*\* Share\*\***.

### Step: Confirm Delivery

Your teammate will receive an email with the View details and a link to open it.

## Delete a View

You can delete Views that are no longer needed.

### Step: Locate the View

Go to the **Views** tab and find the View you wish to delete.

### Step: Click 'Delete'

Click **Delete**, confirm the action, and the system will remove the View.

**Tip:** Saved Views are ideal for reproducibility and team collaboration—use them to build repeatable workflows that scale across experiments or roles.

## Share Dataset

**How This Helps** Dataset sharing makes it easy to collaborate across roles and organizations. It provides a secure way to grant access to curated image and video datasets without manual handoffs or duplication.

You can share any private dataset with individual collaborators or entire teams. This allows others to view, curate, review, or manage data—depending on the permissions you assign.

## Share a Dataset

To share a dataset with teammates or collaborators:

### Step: Go to Dataset Inventory

Navigate to the **Dataset Inventory** page.

### Step: Open Dataset Details

Click on the dataset you want to share to open its **Details** page.

### Step: Click ‘Share’

Click the **Share** button in the toolbar or action menu.

### Step: Enter Recipients

In the sharing panel, enter the email addresses of users or teams you want to invite.

### Step: Send Invite

Set the appropriate permissions and click **Send** to complete the share.

## Accept an Invite

Once someone is invited, here’s what they need to do:

### Step: Check Email

The user receives an email invite with a link to access the dataset.

### Step: Click ‘Accept Invite’

Clicking the link will open the dataset (after login or sign-up, if required).

### Step: Log In or Sign Up

New users must create an account. Existing users can sign in.

### Step: Access the Dataset

Once authenticated, the user will be directed to the **Dataset Details** page.

## Manage Sharing Access

You can manage dataset access after sharing:

- **View who has access:** Open **Sharing Settings** to see current members or teams.
- **Revoke access:** Remove any user or team to revoke access instantly.
- **Extend an invite:** Invites expire after **72 hours**. You can extend them for another 72 hours via the **Sharing panel**.

**Tip:** Sharing is flexible—grant access to internal teams or trusted external partners. Just ensure everyone is registered and signed in before accessing the dataset.

## Export Dataset

**How This Helps** Export datasets for use in training pipelines, labeling platforms, analytics workflows, or external review. You can export everything or just what you need.

## Export a Dataset

### Step: Open the Dataset

Go to the **Dataset Exploration** page.

### Step: Click 'Export'

Click the **Export** button in the top-right corner.

### Step: Name the Export

Provide a descriptive name for your export file.

### Step: Choose Export Scope

Select the scope—entire dataset, filtered view, or specific selections.

### Step: Pick a Format

Choose from supported formats: - **JSON**— available now  
- **COCO**— coming soon

### Step: Include Images (Optional)

Toggle **Include image files** to include up to 5,000 images in your export.

### Step: Export

Click **Export File** to download.

## Export Options

### Entire Dataset

Exports **all items** in the dataset, regardless of filters or selections.

### Matching Filters

Exports only the items that match **your currently applied filters**, such as issue types, tags, or labels.

### Current Selection

Exports only **manually selected items**—images, objects, or clusters.

## JSON Export Format

Exported data is structured into a .json file with two sections:

### 1. info

General metadata about the dataset and export:

```
"info": {
 "dataset": "Name",
 "dataset_creation_time": "2025-04-02T12:15:02.510168",
 "dataset_url": "http://app.visual-layer.com/dataset/.../data",
 "description": "Exported from Visual Layer",
 "export_time": "2025-04-02T14:31:30.931324",
 "exported_by": "User_Name",
 "schema_version": "1.1",
 "total_media_items": 10000000
}
```

### 2. media\_items

An array of media items (images or video frames), each with all available metadata:

```
{
 "media_id": "<uuid>",
 "media_type": "image", // or "video_frame"
 "file_name": "example.jpg",
 "file_path": "example.jpg",
 "file_size": "48.96KB",
 "height": 360,
 "width": 360,
 "url": "http://...",
 "cluster_id": "<uuid>",
 "uniqueness_score": 0.85,
 "metadata_items": [...]
}
```

## Metadata Items

Each media item includes a list of metadata\_items. Below are the possible types:

### 1. type: "issue"

Used to represent a detected problem:

```
{
 "type": "issue",
 "properties": {
```

```

 "issue_type": "duplicates",
 "confidence": 0.85,
 "duplicate_group_id": "<uuid>",
 "duplicate_threshold": 0.88
 }
}

```

## 2. type: "user\_tag"

Tags applied by a user to a media item:

```

{
 "type": "user_tag",
 "properties": {
 "tag_name": "spider",
 "assigned_date": "2025-04-02T12:34:00.189534"
 }
}

```

## 3. type: "video\_info"

Provides context on a video frame:

```

{
 "type": "video_info",
 "properties": {
 "frame_number": 17,
 "video_id": "<uuid>",
 "timestamp": "00:00:00.567"
 }
}

```

## 4. type: "object\_label"

Describes an annotated object (bounding box), including nested issues:

```

{
 "type": "object_label",
 "properties": {
 "bbox": [x, y, width, height],
 "category_name": "Joey",
 "source": "User",
 "original_id": -1,
 "uniqueness_score": 0.85,
 "url": "http://...",
 "metadata_items": [
 {
 "type": "issue",

```



```

 "properties": {
 "issue_type": "blur",
 "confidence": 0.81
 }
]
}

```

## 5. type: "image\_label"

A classification label applied to the entire image:

```

{
 "type": "image_label",
 "properties": {
 "category_name": "car",
 "source": "Model",
 "original_id": 1234
 }
}

```

## 6. type: "caption"

```

{
 "type": "caption",
 "properties": {
 "text": "A dog playing in the field",
 "source": "AI model"
 }
}

```

**Note:** - Duplicates often include duplicate\_group\_id and threshold values. - Object-level issues are nested inside object\_label blocks.

## Exploring Datasets

## Exploration Overview

**What This Section Covers** Get a high-level understanding of the tools available in Visual Layer for search, filtering, quality control, and data curation. Use this section as your launchpad for working with large visual datasets.

This Section Covers:

- **Semantic Search:** Locate relevant data using natural language-powered search across your dataset.
- **Caption search:** Surface relevant content by matching queries to labels and captions.
- **Visual Search** Find similar images and objects through visual comparison.
- **Filter Dataset** Narrow down data based on specific attributes, labels, or conditions.
- **Detect Mislabeled** Identify and correct incorrectly labeled data to improve dataset accuracy.
- **Detect Unlabeled** Find and review entries that are missing annotations.
- **Detect Duplicates** Spot and manage duplicate entries in your dataset.
- **Detect Outliers** Detect irregular data points that may affect analysis or model performance.
- **Select Unique Items** Extract only essential samples for AI model training.
- **Dataset quality** Detect and manage image issues (e.g., blurry, dark, or overexposed) to ensure clean, usable data.

## Semantic Search

**How this Helps** Semantic Search helps you discover relevant visual data based on concepts and context—not just labels or keywords. It's ideal for exploration, discovery, and working with unstructured datasets.

## What Is Semantic Search?

Semantic Search allows you to retrieve images and objects based on the meaning and context of your text queries, instead of relying on exact keyword matches. This enables more intuitive, concept-driven exploration of your dataset.

It works by matching your queries against metadata generated by Visual Layer's enrichment models:

- VL-Image Semantic Search
- VL-Object Semantic Search

These models interpret visual content and associate it with descriptive embeddings, enabling deeper, context-aware querying.

## How It Works

- Queries are matched against generated semantic metadata (not manual tags).
- Available at both the image and object level, depending on your current view.
- Requires enrichment using either VL-Image Semantic Search or VL-Object Semantic Search.
- Ideal for:
  - Surfacing subtle visual patterns
  - Exploring abstract ideas like “urban solitude” or “festival crowd”
  - Navigating datasets with minimal labeling

## Example Queries

Try using natural, descriptive phrases. The system is designed to parse both broad concepts and specific scenes:

**Simple Query** “sunset over mountains”

**Detailed Scene** “bright sunset over rocky mountains with clear sky”

**Combined Attributes** “blue sports car on city street at night”

**Tip:** More detailed phrases yield more focused results. But even simple terms like "crowd", "outdoor event", or "forest animals" can surface useful matches.

## Caption Search

**How this Helps** Caption Search directly matches queries to labels and captions, relying on exact or logical matches to surface content. It's ideal for precise searches where explicit labeling is crucial.

[Image: /images/caption.gif]

## Caption Search Operators

Visual Layer supports several search operators to help you build flexible, accurate caption queries:

Operator	Description	Usage Example
<b>AND Operator</b>	Finds results that contain both terms.	black AND cat, black and cat, black + cat, black cat
<b>OR Operator</b>	Finds results that contain either one term or the other—or both.	black cat OR black dog, black cat or black dog
<b>NOT Operator</b>	Excludes results containing the specified term.	dog -black
<b>Exact Match</b>	Matches results with the exact phrase, wrapping terms in double quotes.	"yellow flower"

{/\* ## AND Operator

The AND operator is implied when you separate terms with a space, or you can write it explicitly.

- **Usage:**  
Use AND, and, or + between terms. Example:  
black AND cat  
black and cat  
black + cat  
black cat

## OR Operator

Use the OR operator to find results that contain either one term or the other—or both.

- **Usage:**  
Use OR or or between terms. Example:  
black cat OR black dog  
black cat or black dog

## NOT Operator

Use the NOT operator to exclude terms from search results.

- **Usage:**  
Prefix a word with a minus sign -. Example:  
dog -black

## Exact Match

Wrap terms in double quotes to return captions that match the exact phrase.

- **Usage:**  
Use double quotation marks " " around a phrase. Example:  
"yellow flower" \*/}

## Combining Operators

You can combine operators to write more advanced queries.

- **Example:**  
flower -bird OR tree AND "red leave"
- **Result:**  
This query includes:
  - Any caption that includes “flower” but not “bird”
  - Or includes “tree”
  - And must also include the exact phrase “red leave”

**Tip:** Use quotes and parentheses to group logic and ensure the query works as expected.

## Visual Search

**How this Helps** Visual Search allows you to find near-duplicates, repeated patterns, and conceptually similar content in your dataset—whether at the image or object level. It's ideal for identifying anomalies, clustering related visuals, and accelerating QA review.

## Overview

Visual Layer enables intuitive similarity search across datasets using deep visual embeddings. You can run a similarity search in several ways:

- On a cluster
- On an image or object
- Using a region of interest
- By Image Upload

## Similarity Search on a Cluster

To find similar results based on a full cluster:

1. Hover over the desired cluster in your dataset.
2. Click **Find Similar** at the bottom of the card.

[Video content]

## Similarity Search on an Image or Object

You can also trigger similarity search directly from any image or object. There are multiple entry points depending on your view:

### Search from the Dataset Page

1. Click into a cluster.
2. Hover over the image or object and click **Find Similar**.

[Image: [https://files.readme.io/67833bc9d2959b2165dc17f6c69fa0e6c6eadd09b98f090dba08e0e6ca813a76-Scene-1\\_2.gif](https://files.readme.io/67833bc9d2959b2165dc17f6c69fa0e6c6eadd09b98f090dba08e0e6ca813a76-Scene-1_2.gif)]

### Search from the Cluster Details Page

1. Click an image or object to open the **Image Details Page**.
2. Click **Find Similar**, then click the similarity icon centered on the image.

[Image: [https://files.readme.io/1e3113710ffe12699bc92844d8bbf90f8a35bcae9c7b9e8da826047ebf5e9a3b-Scene-1\\_2\\_copy.gif](https://files.readme.io/1e3113710ffe12699bc92844d8bbf90f8a35bcae9c7b9e8da826047ebf5e9a3b-Scene-1_2_copy.gif)]

## Search by Object

To run similarity search based on a specific object in the image:

1. Hover over the object's bounding box.
2. Click the **Find Similar** icon.

## Similarity Search Using a Region of Interest

You can crop part of an image and use that region as a visual anchor:

1. Open an image from any cluster.
2. Click **Find Similar**.
3. Drag to define the Region of Interest.
4. Click the similarity icon again to start the search.

## Similarity Search by Image Upload

You can also start a similarity search using an external image:

1. On the Dataset Page, click the **Find Similar** icon near the search bar.
2. Upload an image from your local device.
3. Optionally crop it before starting the search.

[Image: [https://files.readme.io/632ec1043a7acd38bb9f928d26c6cf7f96b6f5f496561e7905f25bcf623f3c52-Scene-1\\_2\\_2.gif](https://files.readme.io/632ec1043a7acd38bb9f928d26c6cf7f96b6f5f496561e7905f25bcf623f3c52-Scene-1_2_2.gif)]

## Filter Dataset

**How this Helps** Filtering helps you instantly surface relevant slices of your dataset — whether you’re tracking mislabels, inspecting object classes, or exploring edge cases.

Visual Layer provides a flexible filtering system powered by logical structures. You can build filters using the **Filter Menu**, define them in the **Query Modal**, and review or edit them in the **Query Panel**. All applied filters are combined using **AND** logic.

## Filter Menu

The **Filter Menu** is where you select filters to apply. It **dynamically adapts** to your dataset and the context of your view.

### Available filter types include:

The filters shown change depending on your current logic mode and view context. [Image: /images/Drop%20Down\_Filters%20Menu.png]

## Query Modal

Each filter opens a **Query Modal**, where you can configure:

- Condition: IS, IS NOT, CONTAINS, STARTS WITH, ENDS WITH and more

**Note:** Conditions and input options vary by filter type.

## Query Panel

The **Query Panel** shows all active filters, grouped by **AND** logic.

## Filter Logic & Behavior

Filter Type	Supported Logic
Foldes	IS ONE OF, STARTS WITH, ENDS WITH, CONTAINS
Files	IS ONE OF, STARTS WITH, ENDS WITH, CONTAINS
Labels	IS, IS NOT, IS ONE OF, IS NOT ONE OF
User Tags	IS, IS NOT, IS ONE OF
Captions	Free Text Search
Mislabels	IS, IS NOT



Filter Type	Supported Logic
Duplicates	Similarity Level
Outliers	IS, IS NOT
Quality Issues	IS, IS NOT
Select Uniques	Slide Bar

[Image: /images/Conditions%20(1).png]

## Editing & Removing Filters

Users can delete any active filter, which will remove it from the query structure, allowing you to refine your dataset exploration further.

## Unlabeled

### Why Unlabeled Data Matters

Unlabeled content is a common byproduct of large, evolving datasets. Whether you're ingesting new data from external sources or curating internal samples, having a method to quickly surface unlabeled items is essential to improving data coverage and model training effectiveness.

Visual Layer makes it easy to locate and take action on unlabeled data—whether at the image or object level.

### How to Find Unlabeled Data

You can filter for unlabeled content directly within the Visual Layer UI using the “Labels” filter.

#### Steps to Apply the Filter

1. Navigate to the Dataset Inventory and click on the dataset you want to review.
2. Choose either the **Images view** or the **Objects view**.
3. In the top filter bar, open the **Labels** filter.
4. Select “**Unlabeled**” and apply the filter.

[Image: <https://files.readme.io/f4e5633-image.png>]

#### What Happens Next

Once the filter is applied:

- You'll see all images or objects (or clusters) that currently have no label.
- These can be reviewed, grouped, exported, or flagged for labeling.

### What You Can Do with Unlabeled Data

After surfacing unlabeled items, you can take additional actions to keep your dataset clean and complete:

- **Organize:** Add relevant data points to the Selected Items list for triage or review.
- **Export:** Use the Export feature to send selected data for manual annotation, third-party labeling, or internal review.

Keeping up with unlabeled content helps maintain dataset integrity and ensures no high-impact images are left out of training.

## Duplicates

```
<iframe width="600" height="300" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/Hm4TCh9xiFw?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

## What Are Duplicate Images and Objects?

Duplicate images refer to multiple copies or near-identical versions of the same visual content. These duplicates can occur within the same dataset or across different datasets and may differ in file format, resolution, or size—but share the same or nearly the same visual data.

### Common Causes of Duplicates

- **Replication:** Duplicate copies may be created intentionally (backups, versioning) or unintentionally (manual copy-paste, syncing between systems).
- **Data collection:** Duplicates are common when compiling datasets from multiple sources (e.g., scraping public images from the web or stock libraries).
- **Data augmentation:** Slight transformations applied during augmentation (e.g., small rotations, brightness tweaks) may result in visually similar but redundant images.
- **Annotation overlap:** Duplicate bounding boxes can occur when the same object is labeled more than once, usually with slight variation in position or size.

### Why It Matters

Issue	Impact
Storage inefficiency	Duplicates increase dataset size without adding value, consuming compute and storage resources.
Skewed distributions	Repeated instances of the same image can bias learning, causing overfitting or model redundancy.
Noise in analysis	Duplicate images can distort statistical metrics or validation scores.
Annotation clutter	Object-level duplication creates confusion and inconsistencies for downstream models.

### How to Prevent and Fix Duplicates

Maintaining a clean dataset improves training efficiency and reduces risk of performance degradation. It’s good practice to scan for duplicates before fine-tuning or model evaluation.

Visual Layer provides built-in tools to identify and clean up duplicate content using **visual similarity detection**. This approach looks at image features rather than just metadata or file hashes, making it effective even when duplicates vary slightly in format or compression.

## How to Detect Duplicates

Visual Layer supports multiple workflows to help detect redundancy:

Detect Duplicates: Go to “Add Filter” ☐ select “Duplicates” ☐ set the desired confidence threshold (default is 1). Export the results using “Matching the applied filter.”

## Outliers

**Tip:** Outlier images are visuals that differ significantly from the majority of your dataset. These anomalies may affect model performance, skew results, or reveal labeling issues.

```
<iframe width="600" height="300" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/OpSLWjy6pGE?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

Outliers typically fall into one of the following categories:

Categories	Description
<b>Domain or Content Outliers</b>	Images that feel “out of place” compared to the rest of your data. From a different source, domain, or modality (e.g., a drawing in a photo dataset). Captured in unexpected conditions (e.g., extreme lighting, occlusions). Showing objects in rare or ambiguous contexts.
<b>Quality Outliers</b>	Images that are technically flawed or inconsistent with the rest of the set. Blurry, overexposed, or too dark. Containing heavy compression artifacts or noise. Corrupted or visually incomplete.
<b>Class (Label) Outliers</b>	Images that don’t semantically match any class in your label vocabulary. True outliers: e.g., an image of a sheep in a dog/cat dataset — it doesn’t belong to any existing class. Out-of-distribution examples: e.g., a sketch of a dog in a dataset of real dog photos. These are different from mislabels, where an image is mislabeled but still belongs to a known class.

## Common Causes of Outliers

Cause	Description
<b>Data collection errors</b>	Samples from unrelated categories or domains may be incorrectly included.

Cause	Description
<b>Artifacts and anomalies</b>	Distortions like blur, noise, or overexposure can make an image an outlier.
<b>Rare instances</b>	Rare objects, edge-case events, or unconventional perspectives may introduce visual outliers.

## Why It Matters

Problem	Impact
Reduced data quality	Outliers introduce noise and reduce consistency across your dataset.
Weaker model performance	Models trained on unfiltered outliers may generalize poorly or become unstable in production.
Hidden skew	Outliers may distort validation results or inflate perceived class diversity.

## How to Detect Outliers in Visual Layer

Visual Layer provides a one-click method for detecting and correcting outliers using automated issue detection.

- **Detect Outliers:**  
Go to “Add Filter” □ “Outliers” □ select “IS” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the results using “Matching the applied filter.”
- **Correct Outliers:**  
Go to “Add Filter” □ “Outliers” □ select “IS NOT” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the results using “Matching the applied filter.”

Managing outliers is an essential step in building reliable, balanced models.

## Class Outlier

Visual Layer enables you to identify images with labels belonging to your defined classes that don't visually match the standard appearance of that class. The class outlier filter helps surface unexpected or inconsistent samples that may degrade model performance.

## What Are Class Outliers?

Class outliers are images that are technically assigned a valid label but don't visually align with the common patterns for that label. They typically fall into one of two categories:

- **Unusual visual appearance**

Labeled correctly, but the image looks very different from others in the same class Example: A drawing of a dog in a dataset of real dog photographs

- **No matching class exists**

The image visually belongs to a different category, even though its label is technically valid Example: A photo of a sheep labeled as "dog" in a dataset that only includes "dog" and "cat" classes

- A drawing of a dog in a dataset of real dog photos
- A sheep labeled as "dog" in a dataset with only "dog" and "cat" classes

## Why It Matters

Need	Description
Model confusion	Outliers within a class can distort the learned distribution
Cleaner training Dataset	Identifying and optionally excluding such samples can improve performance
debugging	Helps uncover unexpected edge cases or data drift

## How to Detect

1. Navigate to **Add Filter** ☐ **Class Outlier**.
2. Set the **confidence threshold** (default set to 1).
3. Export results using **Export** ☐ **Matching the applied filter**.

## Select Unique Items

Visual Layer reduces dataset redundancy by highlighting the most visually distinct samples in your dataset. This workflow:

- Reduces redundancy before model training or export.
- Creates compact datasets that preserve diversity.
- Reveals long tail content often hidden behind common or repetitive examples.

Visual Layer assigns each image or object a **uniqueness score**, then filters out near-duplicates according to a user-defined threshold.

## How It Works

### Step: Uniqueness Scoring

Visual Layer assigns every image or object a **uniqueness score** between 0 and 1: - 0 = highly redundant. - 1 = highly unique.

### Step: Filter Application

Visual Layer retains only samples with the highest uniqueness scores when you apply the filter

### Step: Query Integration

The action is automatically applied as an **Exclude rule** in the Query Panel

## How to Apply the Filter

This workflow lets you focus on the most varied content in your dataset—ideal for debugging, QA, or preparing smaller benchmark sets.

1. Navigate to the **Dataset Exploration View**.
2. Click the **Select Uniques** button (located next to the search bar).
3. Adjust the **threshold slider** to your preferred level of uniqueness.
4. The filter will appear in the Query Panel under the **Exclude** section.

## Uniqueness in Exported Data

If you export data with this filter applied, the retained items will include their uniqueness score in the JSON payload:

```
"uniqueness_score": 0.9285714285714288
```



## Overview

High-quality datasets are essential for training reliable and accurate machine learning models. Poor dataset quality—such as **blur**, **dark**, or **bright** images—can significantly impact performance, leading to **biased predictions, reduced model accuracy, and suboptimal decision-making**.

Identifying and addressing these quality issues ensures that datasets remain **clean, representative, and effective** for AI applications. Visual Layer provides **advanced tools** to **automatically** detect and manage dataset quality issues, helping users **streamline preprocessing, reduce errors, and optimize model training**.

```
<iframe width="90%" height="280" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/xVsUgSfUgWE?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

This section covers common dataset quality issues and how to quickly detect and remove them with **One Click**:

- Detect and Remove **Dark** images & Objects
- Detect and Remove **Blur** images & Objects
- Detect and Remove **Bright** images & Objects

Each issue provides tools to filter by **IS** (show affected items) and **IS NOT** (exclude affected items) to help you clean your dataset effectively.

Dark

```
<iframe width="90%" height="280" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/C_IJ3cRf1nk?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

What Are Dark Images and Objects?

Dark images are underexposed visuals that lack sufficient brightness, making it difficult to interpret or extract visual details. They often appear dim, shadowed, or low-contrast—reducing both human and model-level interpretability.

Common Causes of Darkness in Visual Data

- **Poor lighting conditions:** Capturing in low-light environments or scenes without adequate illumination leads to dark outputs.
- **Camera settings:** Improper settings like fast shutter speed, low ISO, or narrow aperture can result in underexposed images.
- **Scene characteristics:** Some subjects—like night scenes, shaded objects, or predominantly dark materials—naturally produce low-light imagery.

Why It Matters

Problem	Impact
Visual ambiguity	Annotators may miss important elements due to lack of detail or poor visibility.
Model degradation	Object detection and segmentation performance can drop significantly on low-exposure inputs.
Poor user experience	In public-facing applications, dark visuals can appear unpolished or unprofessional.

How to Detect and Remove Dark Images

- **Detect Dark Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Dark**” □ choose “**IS**” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the dark images using **Export** □ “**Matching the applied filter.**”
- **Remove Dark Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Dark**” □ choose “**IS NOT**” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the images without dark issues using **Export** □ “**Matching the applied filter.**”

By detecting and resolving dark visuals early, you can improve dataset quality, annotation accuracy, and model performance—especially in environments where lighting is a critical factor.

## Blurry

```
<iframe width="90%" height="280" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/YtaC6wFXVU4?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

## What Are blurry Images and Objects?

Blurry images lack sharpness and clarity, often making it difficult to recognize objects or extract visual detail. This can significantly reduce the performance of machine learning models, lower data quality, and hinder accurate human annotation.

### Common Causes of Blurriness

- **Focus issues:** Autofocus or manual focus failures can result in out-of-focus shots.
- **Motion blur:** Movement of the subject or camera during exposure leads to streaking or distortion.
- **Camera shake:** Handheld capture or slow shutter speeds often introduce unintended blur.
- **Compression artifacts:** Over-compressed images may lose edge detail and overall clarity.

## Why It Matters

Problem	Impact
Visual interpretation errors	Annotators may struggle to label blurry data, increasing labeling noise.
Model accuracy drop	Object detection, OCR, and segmentation models perform poorly on blurred inputs.
Poor user experience	Low-quality images degrade trust and engagement in user-facing applications.

## How to Detect and Remove Blurry Images

- **Detect Blurry Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Blurry**” □ choose “**IS**” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the blurry images using **Export** □ “**Matching the applied filter.**”
- **Remove Blurry Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Blurry**” □ choose “**IS NOT**” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the images without blurry issues using **Export** □ “**Matching the applied filter.**”

Managing blurriness helps improve data clarity, annotation quality, and downstream model performance—especially in high-precision environments.

## Identifying Mislabeled Images and Objects

**How this Helps** Mislabeling is one of the most common and costly sources of noise in visual datasets. Visual Layer helps surface mislabeled images and objects quickly, reducing model drift, training errors, and bias in production systems.

Mislabeled annotations—whether on an image or object level—can significantly reduce the performance, fairness, and accuracy of machine learning models. These errors are often introduced during human labeling, automated pipelines, or changes in how classes are understood over time.

Common causes of mislabeling include:

- **Human error:** Annotators misinterpret content or confuse similar classes
- **Ambiguity:** Some images lack clear labeling boundaries or contain overlapping concepts
- **Algorithmic error:** Auto-labeling models may introduce bias or misclassify unfamiliar samples
- **Evolving definitions:** Dataset classes can shift based on new understanding, making prior labels obsolete

## Why It Matters

Pain Point	Impact
Poor training data quality	Flawed labels misguide your model’s learning process, leading to lower accuracy and higher generalization error.
Skewed model behavior	Repeated label errors introduce systemic bias, especially across edge cases or underrepresented classes.
Wasted compute and time	Training on bad data wastes GPU cycles, engineering time, and annotation effort.
Broken downstream performance	Incorrect labels propagate into production failures, such as incorrect medical diagnoses or autonomous navigation errors.
Erosion of user trust	Visible errors in model output (e.g., UI predictions) reduce confidence and increase frustration.

## Mislabeled Images vs. Mislabeled Objects

Understanding the distinction is key to taking action.

**Mislabeled Images** These are entire **images** that are likely labeled with the wrong class. Example: “Found 3 images likely mislabeled as *husky* — possible corrections: *wolf*, *coyote*, *Alaskan malamute*.”

Another scenario is multi-object confusion:

An image shows a **cat** on a **table**, but the image is labeled only as “cat” or only as “table”—even though both are valid classes.

**Mislabeled Objects** These refer to individual **objects** within an image that have the wrong label. Example: “Found 3 objects likely mislabeled as *husky* — possible corrections: *wolf*, *coyote*, *Alaskan malamute*.”

Object-level issues often come from: - Incorrect class identification - Bounding boxes that are too small, too large, or misaligned - Overlapping or occluded objects treated as one

## How to Fix It

Visual Layer provides several options to detect and resolve label issues:

### Step: Use Label Filters

Surface misclassified data using Visual Layer’s label filters, which help compare assigned vs. predicted labels across your dataset to spot likely errors.

### Step: Enable Auto-Detection

Let Visual Layer automatically flag mislabeled images or objects using its auto-detection system, powered by dataset-level anomaly detection.

### Step: Curate via Data Selection

Use data selection filters to isolate label conflicts, outliers, or visually similar groups before sending data into training.

### Step: Export for Re-labeling

Export mislabeled data for annotation review. You can route exports to your labeling platform, vendor, or internal QA team with one click.

**Tip:** Label QA is most effective when done iteratively. Visual Layer makes it easy to re-import corrected data versions and track improvements over time.

## Related Articles

**Find Mislabeled Data with Filters** Use label predictions and metadata to surface likely errors.

**Auto-Detect Mislabeled Data** Automatically identify label issues across your dataset.

**Export Data for Re-Annotation** Send mislabeled data back to annotation teams.

## Bright

```
<iframe width="90%" height="280" style={{ borderRadius: '10px'}}
src="https://www.youtube.com/embed/Hv56dF7oAWY?autoplay=1&mute=1&rel=0" title="Detecting
Dark Image Issues" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share" allowfullscreen />
```

## What Are Bright Images and Objects?

Bright images are overexposed visuals with elevated brightness levels that wash out details and distort colors. These typically occur due to excessive lighting, reflective surfaces, or misconfigured camera settings.

### Common Causes of Brightness Issues

- **Excessive lighting conditions:** Strong sunlight, intense indoor lighting, or flash can over-expose parts of an image.
- **Reflective surfaces:** Shiny objects like glass, metal, or water reflect excessive light and may create bright hotspots.
- **Camera settings:** A high ISO, wide aperture, or slow shutter speed can result in the image sensor capturing too much light.

## Why It Matters

Problem	Impact
Loss of detail	Overexposure reduces contrast and texture, especially in highlights.
Color distortion	Washed-out areas may appear inaccurate or completely white, affecting labeling or recognition.
Lower model performance	Vision algorithms may struggle to detect or segment bright objects due to low edge visibility.
Visual quality degradation	In public-facing use cases, poor exposure impacts perceived quality and credibility.

## How to Detect and Remove Bright Images

- **Detect Bright Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Bright**” □ choose “**IS**” as the logic operator □ set the desired confidence threshold (default is 0.5).  
Export the bright images using **Export** □ “**Matching the applied filter.**”
- **Correct Bright Images:**  
Go to “**Add Filter**” □ “**Quality Issue**” □ select “**Bright**” □ choose “**IS NOT**” as the logic

operator ☐ set the desired confidence threshold (default is 0.5).

Export the images without bright issues using **Export** ☐ **“Matching the applied filter.”**

Improving brightness balance ensures your dataset retains visual clarity, maintains labeling accuracy, and supports better-performing models.

## Dataset Enrichment



## Enrich Your Datasets with AI Models

Dataset enrichment in Visual Layer is a two-step process:

1. Select an enrichment model
2. Run enrichment to generate new metadata for your dataset

This process adds structured metadata (e.g., labels, captions) to help improve dataset quality, enable better filtering, and power downstream automation.

## Verify Dataset is Ready

To ensure your dataset is ready for enrichment:

1. Navigate to the **Dataset View** and confirm the status is **READY**.
2. Alternatively, check the dataset status in the **Dataset Inventory**.

## Step 1: Select Models

### Step: Open the Enrich Tab

Go to your dataset and click the **Enrich** tab.

### Step: Choose a Model

Select a model from the list and click **Generate Models Preview**.

[Image: <https://files.readme.io/48d302b7b8975a547fc20895f2261373080d920b4f2f0ce32c3daebb0148bbd0-image.png>]

### Step: (Optional) Add More Models

Click **Add Models** to include additional enrichment models.

[Image: <https://files.readme.io/4af51591dd776faa73acd1bd869415127750bc4afa8b5e991d62365609a7b58a-image.png>]

### Step: Preview Model Outputs

Switch between selected models to preview their metadata output before enrichment.

[Image: <https://files.readme.io/935f0fd46f8a22191a17bdee3c294b4abc67ed9a0f057669633e759ae2e7734f-image.png>]

## Step 2: Enrich the Dataset

### Step: Start Enrichment

Click **Enrich Dataset** to run all selected models on the dataset.

- All images or frames will be enriched.

- Processing time varies based on dataset size.

### **Step: Track Progress**

You can monitor enrichment status in either location:

- **Enrich tab** inside the dataset view
- **Notifications panel** in the Dataset Inventory

[Image: <https://files.readme.io/99f5ed83ecaa06d4131a434a34188d5fdc5121a0097e87d75910e3a5d4c797ec-image.png>]

[Image: <https://files.readme.io/d30ad5ed33f77fa4aba768ef5ca9e53af5a31aa3b35323c0a426cbd42f618b9b-image.png>]

**Tip:** You can continue exploring and interacting with your dataset while enrichment is in progress.

**Note:** Now, you are ready to explore your enriched Dataset here

## Explore Model Catalog

Visual Layer's Enrichment Hub lets you generate high-value metadata using pre-trained models tailored for image and video datasets. These enrichment models can:

- Extract descriptive labels, captions, and tags
- Enable advanced semantic and object-level search
- Power downstream filtering, QA, and automation
- Improve annotation coverage and data understanding

## Available Enrichment Models

Visual Layer provides a wide range of built-in models designed for diverse enrichment tasks:

Model Name	Task Type	Description
<b>VL-Object-Detector</b>	Object Detection	Identifies and localizes objects with bounding boxes and class labels.
<b>VL-Image-Tagger</b>	Multi-Class Classification	Applies multiple labels to the entire image for categorization and metadata generation.
<b>VL-Object-Captioner</b>	Object to Text	Generates short captions describing individual objects in context.
<b>VL-Image-Captioner</b>	Image to Text	Summarizes the scene or image with natural language.
<b>VL-Image-Semantic Search</b>	Semantic Image Search	Enables conceptual search over images using natural language queries.
<b>VL-Object-Semantic Search</b>	Semantic Object Search	Enables contextual search for specific objects based on semantics.
<b>NVILA-Lite-2B</b>	Image-Text-to-Text (VQA)	Efficient VQA model for visual understanding tasks across multiple frames or images.
<b>Janus-Pro-1B</b>	Image-Text-to-Text (VQA)	Autoregressive model for multi-modal reasoning and question answering.

**Note:** Some models require pre-existing labels before enrichment. These models include:

- **Object/Image Captioners**
- **Semantic Search (Object or Image)**

Labels may come from user annotations, the Object Detector, or the Image Tagger model.

## Coming Soon

These models are in development and will be available soon in the enrichment catalog:

Model Name	Description
<b>GPT-4o</b>	Multimodal LLM for advanced captioning and reasoning
<b>Nv-grounding dino</b>	Grounded object detection with natural language prompts
<b>Neva-22B</b>	High-performance image-to-text transformer
<b>Nvclip</b>	Lightweight vision-language model for fast embeddings
<b>Qwen-VL-2B</b>	VQA and captioning model for diverse use cases
<b>Moondream2</b>	Compact vision-language model
<b>Molmo-7B</b>	Multimodal model for contextual enrichment
<b>Advanced-Image-Search</b>	Conceptual image retrieval using complex queries
<b>Advanced-Object-Search</b>	Enhanced semantic object retrieval
<b>VL-Object-Tagger</b>	Assigns tags at the object level across frames or images

## Want Early Access?

**Get in Touch** Have questions or want to try out upcoming models early?  
Contact us to request access or learn more.

## Analyze Enriched Dataset

Once your dataset has been enriched using Visual Layer's AI models, you can begin analyzing the generated metadata to surface patterns, identify issues, and prepare the dataset for export, training, or delivery.

Use the tools below to take full advantage of your enriched data. ## Search with Semantic Models

Use Semantic Search to explore your dataset using **natural language queries**.

### Common use cases:

- Search for visual concepts (e.g., "sunset over forest")
- Find fine-grained differences across scenes
- Match by meaning, not just tags

## Filter by Issues

Detect and isolate problematic data using Visual Layer's automated filters:

- Mislabeled
- Duplicates
- Outliers
- Unlabeled Items

These tools help you clean your dataset quickly and improve downstream performance.

## Use Captions & Tags

Explore model-generated metadata such as:

- **Captions** for entire images
- **Tags** and labels for quick filtering
- **Descriptions** of objects or scenes

Use Caption Search or User Tags to filter and organize your enriched data more effectively.

## Export Enriched Data

Use the Export tool to download either the full dataset or a filtered subset.

### Included in the export:

- Model-generated labels
- Captions and tags
- Object metadata and annotations

All exported data is formatted in clean, structured JSON.

## Changelog

## Release Notes & Product Updates

### New Features

This release introduces:

- a new embedding model that enhances textual search, clustering, and mislabel detection capabilities
- a new filter to allow users to filter bounding boxes by object (pixel) size.

### New Embedding Model

We are excited to announce the introduction of a new embedding model that expands the way teams can explore, organize, and validate their data. By combining these improvements, the model:

- Reduces dependency on expensive infrastructure
- Strengthens insights through better clustering
- Raises the overall trustworthiness of datas through improved quality checks

### Textual Search Without GPUs

Until now, running rich text-based searches across large datasets often required specialized hardware such as GPUs and additional compute time. With this release, our model delivers efficient, accurate textual search on standard CPUs, making it easier and more cost-effective to integrate into everyday workflows.

[Image: /images/improved-embedding.png]

### Improved Clustering Capabilities

The new architecture enhances how related items are grouped together, producing clusters that are more coherent and meaningful. This makes pattern discovery faster and the results more reliable, whether you're mapping themes or identifying anomalies.

### Stronger Mislabel Detection

Identifying mislabeled data has always been a challenge. This version offers a step forward with sharper, more consistent detection methods, reducing noise and improving dataset quality with less manual intervention.

### New Filter

This version introduces a filter by object (pixel) size (width, height).

This filter allows users to view only bounding boxes (objects) above/below a specified size (in squared pixels).

[Image: /images/filter-by-object-size.png]

[Image: /images/filter-by-object-size2.png]

This is intended to help QA reviewers and annotators quickly focus on large or significant objects without visual noise from very small detections.

## Enhancements & Fixes

- Duplicates filter availability issues after performing a visual search by uploading an external image have been resolved.
- Custom metadata: Increased the enum values limitation from 20 to 100.
- Resolved thumbnail generation inefficiencies from S3 buckets.

We are excited to present the latest release of Visual Layer. This update introduces powerful new ways to search, explore, and manage visual datasets while streamlining the UI for speed and clarity.

## New Features & Enhancements

We've overhauled the filtering experience for intuitive navigation, custom quality thresholds, and advanced query logic, making it easier to zoom in on what matters or explore freely at scale.

### Improved UI Navigation

Filter menu organization and iconography have been updated including a unified slice & dice experience and clearer visual cues for advanced logic combinations.

[Image: /images/highlight-filters-and-insights.png]

### Optimized Filter Logic

Visual Layer now provides logical query operators.

We've refined the right panel to remove outdated filter controls. Now, you can build filters and review or edit. All applied filters are currently combined using AND logic. Slice and dice your dataset more precisely and seamlessly combine visual and metadata filters for high-volume review.

Set flexible conditions per filter type, including:

- IS, and IS ONE OF
- IS NOT: Explicitly exclude values and easily remove irrelevant items from your analysis
- CONTAINS and free-text search that dynamically adapts to your dataset and the current view context

Read more about filters.

[Image: /images/filters-operators-2.png]

### Streamlined Insights

All your dataset insights are now accessible in one place through the redesigned right panel. Easily navigate between statistics, enrichment models, and quality issues without visual clutter. You can



also toggle between # (absolute values) and % (relative values) to view data in the format that matters most to you.

[Image: /images/issues-toggle.gif]

## **Class Outlier Detection**

Class outliers are a new issue type that help you detect subtle, high-impact inconsistencies in labeled datasets.

Unlike mislabels and general outliers, class outliers highlight images that don't fit the expected structure or semantics of a specific dataset. Class outliers often reflect edge cases, data drift, or content mismatches that can compromise model performance if left unreviewed.

You can now:

- View class outlier stats in the right panel
- Adjust detection thresholds for both general and class-level outliers WIP
- Filter by outliers or class outliers directly from the main filter menu

[Image: /images/filters-outliers-classoutliers.png]

Learn more about class outliers.

## **Better Documentation to Support You**

We've upgraded our documentation to make it faster, clearer, and easier to navigate.

You'll notice:

- Cleaner layouts that help you focus on what matters
- Collapsible guides and tabs for scanning complex topics
- Inline tips and examples to guide you through advanced use cases
- Mobile-friendly design for quick answers on the go

## **Fixed Issues**

We've cleaned up some bugs and made improvements to enhance your experience, including improved search results, fixes to filters and other similar issues.

# API Reference

## API Documentation

## **API Introduction**

This comprehensive guide will help you understand and use the Visual Layer API effectively. Visual Layer provides powerful tools for managing, exploring, enriching, and analyzing visual data through an intuitive API.

The Visual Layer API provides a comprehensive set of endpoints to interact with Visual Layer's powerful data processing, enrichment, and exploration capabilities. This guide covers everything you need to start working with the API.

## **Cloud API vs On-Premises API**

There are important differences between cloud and on-premises deployments:

- Cloud API: All cloud API calls require authentication with JWT tokens
- On-Premises API: Authentication is not required for on-premises deployments

If you're using an on-premises deployment, you can disregard authentication sections and omit authorization headers in requests.

## Authentication

**On-Prem Deployments** If you're using an on-premises deployment, you can disregard authentication sections and omit authorization headers in requests.

### Step 1: Get Your API Key and Secret

To obtain your API key and secret:

#### Step: Log in to Visual Layer

Go to <https://app.visual-layer.com> and sign in with your user account.

#### Step: Visit the API Credentials Page

Navigate to [https://app.visual-layer.com/api/v1/api\\_credentials](https://app.visual-layer.com/api/v1/api_credentials) while logged in.

You must be authenticated in the browser for this to work. The request depends on your active browser session.

## Request

```
GET /api/v1/api_credentials
Cookies: SESSION=<session from authenticated user browser session>
```

## Example Response

```
{
 "api_key": "<api_key>",
 "api_secret": "<api_secret>",
 "message": "API key created. Write down the secret – it's only shown once! If you ne
}
```

You can also run this using curl:

```
curl -b SESSION=<session from authenticated user browser session> https://app.visual-l
```

### Step 2: Generate JWT Token for Authentication

Use your API key and secret to generate a JWT token, which is required for authenticated API requests.

**JWT Token Authentication** JWTs must be included in the Authorization header as a Bearer token type on every request.

These tokens are short-lived and should be regenerated periodically.

## Example Python Code

□ Please make sure you have PyJWT installed. If not, install it using:

```
python -m pip install PyJWT
```

```
from datetime import datetime, timezone, timedelta

def generate_jwt(api_key: str, api_secret: str):
 jwt_algorithm = "HS256"
 jwt_header = {
 'alg': jwt_algorithm,
 'typ': 'JWT',
 'kid': api_key,
 }

 # Convert datetime objects to Unix timestamps (seconds since epoch)
 now = datetime.now(tz=timezone.utc)
 expiration = now + timedelta(minutes=10) # You can change this to a larger number

 payload = {
 'sub': api_key,
 'iat': int(now.timestamp()), # Convert to Unix timestamp
 'exp': int(expiration.timestamp()), # Convert to Unix timestamp
 'iss': 'sdk'
 }

 return jwt.encode(payload=payload, key=api_secret, algorithm=jwt_algorithm, header=jwt_header)

api_token = generate_jwt(api_key, api_secret)

print("VL-Token:" + api_token)
```

**Tip:** JWT tokens expire quickly by design. For best security and performance, regenerate tokens periodically based on your security requirements.

## Step 3: Test Your JWT Token

After generating your JWT token, you should verify it works by testing it against one of your datasets.

**Why Test Your Token?** Testing your token immediately after generation helps you: - Verify the token was generated correctly - Confirm you have access to your datasets - Ensure the token is valid before integrating it into your applications

### Test Using curl

You can test your JWT token by retrieving the status of one of your datasets:

```
Replace <jwt_token> with your generated JWT token
Replace <dataset_id> with one of your dataset IDs
curl -H "Authorization: Bearer <jwt_token>" \
 https://app.visual-layer.com/api/v1/dataset/<dataset_id>
```

## Test Using Python

```
Your generated JWT token
jwt_token = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

One of your dataset IDs (you can find this in the URL when viewing a dataset)
dataset_id = "your-dataset-id-here"

Make the API request
url = f"https://app.visual-layer.com/api/v1/dataset/{dataset_id}"
headers = {"Authorization": f"Bearer {jwt_token}"}

response = requests.get(url, headers=headers)

Check the response
if response.status_code == 200:
 dataset_info = response.json()
 print(f"✓ Success! Dataset: {dataset_info['display_name']}")
 print(f" Status: {dataset_info['status']}")
 print(f" Progress: {dataset_info['progress']}%")
elif response.status_code == 401:
 print("✗ Authentication failed – check your JWT token")
elif response.status_code == 404:
 print("✗ Dataset not found – check your dataset ID")
else:
 print(f"✗ Error: {response.status_code} – {response.text}")
```

## Expected Success Response

If your token is valid, you should receive a 200 OK response with dataset information:

```
{
 "id": "abc-123-def-456",
 "display_name": "My Dataset",
 "status": "READY",
 "progress": 100,
 "owned_by": "VL",
 "created_at": "2025-10-30T10:00:00.000000"
}
```

## Common Error Responses

Status Code	Meaning	Solution
<b>401 Unauthorized</b>	JWT token is invalid or expired	Regenerate your JWT token with correct API credentials
<b>404 Not Found</b>	Dataset ID doesn't exist or you don't have access	Verify the dataset ID from your Visual Layer dashboard
<b>403 Forbidden</b>	Valid token but insufficient permissions	Check that the API key has access to this dataset

**Tip: Where to find your dataset ID?** Your dataset ID is in the browser URL when viewing a dataset: [https://app.visual-layer.com/dataset/<dataset\\_id>/data](https://app.visual-layer.com/dataset/<dataset_id>/data)

## Create a Dataset from a Public S3 Bucket

**How This Helps** Use this method to quickly create datasets from publicly hosted S3 buckets. Ideal for demos, testing, or ingesting open datasets without manual upload.

## Prerequisites

This feature is **only supported in cloud-hosted environments**.

Before using this endpoint, ensure your S3 bucket is configured for public access. For setup instructions, see [Public S3 Bucket Setup](#).

## Make the API Request

To create a dataset using a public S3 bucket, send a POST request to the following endpoint:

### Request

```
POST /api/v1/dataset
Headers: Authorization: Bearer <jwt>
```

### Required Form Parameters

- `dataset_name`: (*string*) – The name of the dataset to create
- `bucket_path`: (*string*) – The path to the public S3 bucket containing your data

## Example

```
curl -X POST -H "Authorization: Bearer <jwt>" \
 -F dataset_name=api_test \
 -F bucket_path=public_test_bucket \
 https://app.visual-layer.com/api/v1/dataset
```

**Tip:** The `dataset_id` returned in the response will be used for all future operations on that dataset.



## Create a Dataset from a Local Folder for On-Prem Installations

**How This Helps** This method allows teams running Visual Layer on-prem to create datasets directly from local folders, bypassing manual upload or cloud storage.

### API Overview

To create a dataset using a local folder, issue a request to the processing endpoint with the dataset name and path.

### Endpoint Format

```
GET api/v1/process?path=[full data folder path]&name=[dataset name]
```

### Parameters

Parameter	Description
host	The server address where the Visual Layer API is hosted
path	Full system path to the folder containing files for processing
name	The desired name of the dataset

### Example Request

Assume you want to process a dataset located at `/data/project/images` and name it `visual_analysis_01`.

```
http://example.com:2080/api/v1/process?path=/data/project/images&name=visual_analysis_01
```

### Example Response

```
{
 "status": "success",
 "message": "Dataset processing started",
 "dataset_id": "12345",
 "name": "visual_analysis_01"
}
```

## Error Handling

In case of a failure, the API returns a descriptive error response. Common issues include:

- Invalid or non-existent path
- Missing path or name parameters
- Server unavailable or internal failure

### Example Error Response

```
{
 "status": "error",
 "message": "Invalid path specified"
}
```

## Notes & Considerations

- Only **one dataset** can be processed at a time.
- If another dataset is already being processed, the request will be rejected.
- Make sure the server is up and the API port is accessible.
- The folder path must contain valid files for enrichment and must be readable by the system.
- Dataset names should be **unique** to prevent overwriting or collisions.

**Tip:** Contact your system admin or IT team if you encounter connection errors or permission issues during API usage.

## Retrieve Dataset Status

**How This Helps** This API call allows you to programmatically check the current status of a dataset—including readiness, progress, and metadata—using its unique `dataset_id`.

## API Request

To retrieve the status of a dataset, send a GET request to the following endpoint:

```
GET /api/v1/dataset/{dataset_id}
Headers: Authorization: Bearer <jwt>
```

## Example Request

```
curl -H "Authorization: Bearer <jwt>" https://app.visual-layer.com/api/v1/dataset/{dataset_id}
```

## Example Output

```
{
 "id": "DATASET_ID",
 "source_dataset_id": null,
 "owned_by": "VL",
 "display_name": "dataset1",
 "description": "",
 "source_type": "BUCKET",
 "created_at": "2025-04-24T07:44:46.136915",
 "filename": null,
 "sample": null,
 "status": "READY",
 "fatal_error_msg": null,
 "progress": 100,
}
```

## Dataset Status Values

The status field in the response can have the following values:

Status	Description
NEW	Dataset has been created but not yet started processing
PRE_PROCESSING	Dataset is being prepared for processing
INITIALIZING	Dataset processing is being initialized

Status	Description
UPLOADING	Data is being uploaded to the system
SAVING	Dataset data is being saved
INDEXING	Dataset is being indexed for search and retrieval
READY	Dataset is fully processed and ready for use
FATAL_ERROR	An error occurred during processing that prevents completion
ENRICHING	Dataset is being enriched with additional metadata or processing

**Tip:** Use this endpoint to track dataset readiness before continuing with enrichment, filtering, or export operations.

□ For detailed reference, visit the Dataset Status API.

## Share a Dataset with Another User

**How This Helps** Sharing datasets allows for seamless collaboration across team members. You can grant access to reviewers, annotators, or developers without duplicating data.

## API Request

To share a dataset with another user:

```
POST /api/v1/dataset/{dataset_id}/share
Headers: Authorization: Bearer <jwt>
```

## Form Parameters

Parameter	Description
user_email	Email of the user to share with. Must be a registered Visual Layer user.

## Example Request

```
curl -X POST -H "Authorization: Bearer <jwt>" \
 -F user_email=annotator1@company.com \
 http://localhost:9999/api/v1/dataset/{dataset_id}/share
```

## Possible Responses

Status Code	Meaning
200 OK	☐ Dataset successfully shared
404	☐ Failed to share dataset

**Tip:** Make sure the recipient's email matches an existing registered user in your Visual Layer environment.

## Duplicate Retrieval

**How This Helps** Identify duplicate media in your dataset using internal media IDs. This helps streamline cleanup, reduce redundancy, and improve data quality before training or export.

### Step 1: Retrieve Internal Media ID

To begin, retrieve the internal media ID for a file based on its `original_media_uri`.

```
GET /api/v1/dataset/{dataset_id}/search_media_metadata/id?original_media_uri={url_encoded_uri}
Headers: Authorization: Bearer <jwt>
```

#### Example

```
curl -H "Authorization: Bearer <jwt>" \
 "https://app.visual-layer.com/api/v1/dataset/{dataset_id}/media/id?original_media_uri={url_encoded_uri}"
```

#### Response:

- `200 OK`: Returns the internal media ID as text
- `404`: Media not found in the dataset

### Step 2: Retrieve Duplicates Using Media ID

Once you have the media ID, use this endpoint to find duplicates:

```
GET /api/v1/dataset/{dataset_id}/media/{media_id}/duplicates
Headers: Authorization: Bearer <jwt>
```

#### Example

```
curl -H "Authorization: Bearer <jwt>" \
 https://app.visual-layer.com/api/v1/dataset/{dataset_id}/media/{media_id}/duplicates
```

#### Response:

Returns a JSON array containing 0 or more duplicate media IDs.

**Tip:** This feature is currently under development and subject to change.

## Visual Search

To search by external query image, upload external the image and make it available as a query image by issuing the following call:

## Upload an Image for Search

```
POST /api/v1/dataset/{dataset-id}/search-image-similarity?entity_type=IMAGES&bounding_
Headers: Authorization: Bearer <jwt>
```

### Request Parameters:

- file: Query image file
- bounding\_box (optional): X, Y, Width, Height

## Visual Layer API - Dataset Exploration

After uploading the image you can call exploration endpoint with anchor media id:

```
GET /api/v1/explore/{dataset-id}
GET /api/v1/explore/{dataset-id}?threshold=0&anchor_type={anchor_type}&entity_type={EN
Headers: Authorization: Bearer <jwt>
```

### Query Parameters

- entity\_type IMAGES or OBJECTS
- anchor\_type MEDIA or UPLOAD for when you uploaded the image using the upload endpoint
- anchor\_media\_id id of the image you want to use as anchor, returned from the upload for search endpoint
- threshold similarity threshold(0-4) the search results are clustered according to this value
- PAGE\_NUMBER: Which page of results to view (e.g., 0 for start)

### relevance\_score

Each result item includes a relevance\_score, which measures similarity to the anchor image.

- **Scale:**
  - 0 = Most similar
  - 2 = Least similar
- This score is based on cosine distance and is available per media item in the response.

## Sample Response

```
{
 "clusters": [
 {
 "cluster_id": "bffbc083-f5a6-426d-a9cf-c0b8a3ed232a",
 "type": "IMAGES",
 "n_images": 7,
 "n_objects": -1,
 "n_videos": -1,
 "n_frames": -1,
 "size_display_value": "0B",
 "similarity_threshold": "1",
 "previews": [
 {
 "type": "IMAGE",
 "media_id": "8c...e4c",
 "media_uri": "/.../e4c.jpg",
 "media_thumb_uri": "/.../e4c.jpg.webp",
 "caption": "The image presents..",
 "image_id": "8c...e4c",
 "image_uri": "/..3e4c.jpg",
 "bounding_box": null,
 "file_name": "5049.tif",
 "original_uri": "visual-layer/datasets/huji-geo/5049.tif",
 "video_uri": "",
 "frame_timestamp": null,
 "labels": null,
 "user_tags": null,
 "relevance_score": 0.1304212444996986,
 "relevance_score_type": "cosine_distance"
 }
],
 "metadata": {
 "used_duckdb": false
 }
 }
]
}
```

## Example:

```
curl -H "Authorization: Bearer <jwt>" https://app.visual-layer.com/api/v1/explore/{dat
```

## Notes:

- Results are paginated with a fixed page size of 100 clusters per page



- Higher threshold values (0-4 depending on the dataset granularity options) create more granular clusters with stricter similarity requirements
- When using anchor-based similarity search, results are ordered by similarity to the anchor
- The `entity_type` parameter controls whether to return image clusters, object clusters, or both
- For similarity search with an uploaded image, first use the `/search-image-similarity` endpoint

More detailed example is [here](#)

## Semantic Search

**How This Helps** Semantic Search returns results based on the **meaning** of your query—not just keyword matches. It’s perfect for finding relevant content using natural language.

## Overview

The GET `/api/v1/explore/{dataset_id}` endpoint enables semantic search over captions—using models that understand language and context, not just keywords.

Example query:

dog playing with a ball

May return:

- “A golden retriever catching a frisbee in mid-air”
- “Puppy chasing a red toy across the grass”
- “Canine fetching a tennis ball at the park”

Results are sorted by **relevance score** (0.0–1.0).

## Basic Caption Search

```
GET /api/v1/explore/{dataset_id}
```

### Query Parameters

- `image_caption` (string): your semantic query
- `textual_similarity_threshold` (float): e.g. `0.75`
- `entity_type` (string): e.g. `IMAGES`

### Example (cURL)

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
"https://app.visual-layer.com/api/v1/explore/95233006-eddc-11ef-b303-76dbc3993eb2?imag
```

## Sample Response

```
{
 "clusters": [
 {
```

```

 "id": "4e0e4d51-0fef-4fe1-a8ec-1a82b6f4880b",
 "entity_type": "IMAGES",
 "size": 3,
 "representative_id": "8dedd5e3-a95c-4d6c-aecb-bdf7c92b599a",
 "media": [
 {
 "id": "8dedd5e3-a95c-4d6c-aecb-bdf7c92b599a",
 "caption": "A dog playing with a red ball in the grass",
 "relevance_score": 0.93
 }
]
 },
 "total_pages": 1,
 "current_page": 0
}

```

## Response Concepts

- clusters: Groups of visually similar results
- media: Items in each cluster
- caption: AI-generated or user-supplied description
- relevance\_score: Semantic match score (0.0–1.0)

## Advanced Filtering

You can add filters for labels and tags:

```

curl -H "Authorization: Bearer YOUR_TOKEN" \
 "https://app.visual-layer.com/api/v1/explore/95233006-eddc-11ef-b303-76dbc3993eb2?imag

```

## Retrieve Dataset Metadata

```

curl -H "Authorization: Bearer YOUR_TOKEN" \
 "https://app.visual-layer.com/api/v1/explore/95233006-eddc-11ef-b303-76dbc3993eb2/exp

```

```

{
 "dataset_name": "Dog Images Dataset",
 "clusters_count": 42,
 "media_count": 1250,
 "labels": [

```

```

 { "name": "dog", "count": 870 },
 { "name": "person", "count": 350 }
]
}

```

## Python Client Example

```

from typing import Dict, List, Optional

class VisualLayerClient:
 def __init__(self, api_url: str, api_token: str):
 self.api_url = api_url
 self.headers = {'Authorization': f'Bearer {api_token}'}

 def search_by_caption(self, dataset_id: str, caption_query: str,
 textual_similarity_threshold: float = 0.75,
 entity_type: str = 'IMAGES',
 page_number: int = 0,
 labels: Optional[List[str]] = None,
 tags: Optional[List[str]] = None) -> Dict:
 params = {
 'image_caption': caption_query,
 'textual_similarity_threshold': textual_similarity_threshold,
 'entity_type': entity_type,
 'page_number': page_number
 }
 if labels:
 params['labels'] = f"[{'','.join(labels)}]"
 if tags:
 params['tags'] = "untagged" if tags == ["untagged"] else f"[{'','.join(tags)}]"

 response = requests.get(f"{self.api_url}/api/v1/explore/{dataset_id}", headers=self.headers, params=params)
 response.raise_for_status()
 return response.json()

 def extract_media_info(self, results: Dict) -> List[Dict]:
 media_items = []
 for cluster in results.get('clusters', []):
 for item in cluster.get('media', []):
 media_items.append({
 'id': item['id'],
 'caption': item.get('caption'),
 'relevance_score': item.get('relevance_score')
 })

```

```
return media_items
```

## Example Usage

```
from visual_layer_client_test import VisualLayerClient

Initialize client with your API token
client = VisualLayerClient("https://app.visual-layer.com", "YOUR_TOKEN")

Basic search
results = client.search_by_caption(
 dataset_id="2348e2ec-860c-11ef-968d-76f1445e4fca",
 caption_query="dog"
)

Extract and print media results
media_items = client.extract_media_info(results)
print("Search results:")
for item in media_items:
 print(f"- ID: {item['id']}")
 print(f" Caption: {item['caption']}")
 print(f" Relevance: {item['relevance_score']}")
 print()

Advanced search with filters
filtered_results = client.search_by_caption(
 dataset_id="2348e2ec-860c-11ef-968d-76f1445e4fca",
 caption_query="dog",
 textual_similarity_threshold=0.75,
 labels=["dog", "puppy"]
)

filtered_items = client.extract_media_info(filtered_results)
print(f"\nFound {len(filtered_items)} filtered results")
```

## Caption Search

**How This Helps** Caption Search enables you to retrieve relevant media using natural phrases like “beach sunset” or “red car”—not just exact keyword matches.

## Overview

The GET `/api/v1/explore/{dataset_id}` endpoint enables semantic caption search using the `image_caption` parameter.

This type of search returns results based on **intent** and **meaning**, not just exact text.

## Example

A query for "beach sunset" may return:

- “A man at the beach on sunset”
- “Golden sun setting over the ocean waves”

Searching for:

- `red car` — matches loosely related red items and cars
- `"red car"` — matches images of an actual red car

Each result includes a **relevance score** that quantifies the quality of the match (0.0–1.0).

## Authentication

All API calls require a bearer token:

```
Authorization: Bearer {your_api_token}
```

## Textual Caption Search

### Endpoint

```
GET /api/v1/explore/{dataset_id}
```

### Required Parameters

Name	Type	Description
<code>image_caption</code>	string	The search query (e.g. "beach sunset")
<code>threshold</code>	integer	Clustering threshold (0–4)
<code>entity_type</code>	string	Must be IMAGES or OBJECTS

Name	Type	Description
textual_similarity_threshold	float	Score cutoff (0.0–1.0) for relevance

## Example (cURL)

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
"https://app.visual-layer.com/api/v1/explore/95233006-eddc-11ef-b303-76dbc3993eb2?thre
```

## Response Example

```
{
 "clusters": [
 {
 "id": "...",
 "entity_type": "IMAGES",
 "size": 4,
 "representative_preview": "...",
 "media": [
 {
 "id": "...",
 "preview": "...",
 "filename": "sunset_beach_aerial.jpg",
 "caption": "Golden sun setting over the ocean waves",
 "relevance_score": 0.92
 }
]
 }
],
 "total_pages": 1,
 "current_page": 0
}
```

## Response Breakdown

- clusters: Visual similarity groups
- media: Individual results
- caption: Caption match

- relevance\_score: Quality of the match
- preview: Thumbnail preview

## Filter and Refine

### Add Similarity Threshold

```
...&textual_similarity_threshold=0.85
```

### Filter by Tags and Labels

```
...&labels=[beach,sunset]&tags=7b89e36c-c2d1-4af9-9d23-f74e018e67c5
```

## Python Client

```
from typing import Dict, List, Optional

class VisualLayerClient:
 def __init__(self, api_url: str, api_token: str):
 self.api_url = api_url
 self.headers = { 'Authorization': f'Bearer {api_token}' }

 def search_by_caption(self, dataset_id: str, caption_query: str, threshold: int =
 entity_type: str = 'IMAGES',
 textual_similarity_threshold: Optional[float] = None,
 page_number: int = 0,
 labels: Optional[List[str]] = None,
 tags: Optional[List[str]] = None) -> Dict:
 params = {
 'image_caption': caption_query,
 'threshold': threshold,
 'entity_type': entity_type,
 'page_number': page_number
 }
 if textual_similarity_threshold is not None:
 params['textual_similarity_threshold'] = textual_similarity_threshold
 if labels:
 params['labels'] = f"[{' '.join(labels)}]"
 if tags:
 params['tags'] = "untagged" if tags == ["untagged"] else f"[{' '.join(tags)}]"
 response = requests.get(f"{self.api_url}/api/v1/explore/{dataset_id}",
```



```

 headers=self.headers, params=params)
 response.raise_for_status()
 return response.json()

def extract_media_info(self, results: Dict) -> List[Dict]:
 media_items = []
 for cluster in results.get('clusters', []):
 for item in cluster.get('previews', []):
 media_items.append({
 'id': item['media_id'],
 'preview_url': item.get('media_uri'),
 'caption': item.get('caption'),
 'relevance_score': item.get('relevance_score'),
 'label': item.get('label')
 })
 return media_items

```

## Example Usage

```

client = VisualLayerClient(API_URL, API_TOKEN)
results = client.search_by_caption(dataset_id=DATASET_ID, caption_query="beach sunset"
media = client.extract_media_info(results)

```

## Advanced Techniques

### Phrase Match

```
image_caption="\\"golden sunset\\""
```

### Semantic Exclusions

```
image_caption=beach%20sunset%20without%20people
```

## Best Practices

- Use specific queries
- Set a minimum similarity threshold
- Use tags and labels to focus results

- Sort using `relevance_score`
- Remember: visual clusters  $\neq$  semantic clusters

## Pagination Helper

```
def get_all_caption_matches(client, dataset_id, caption_query, min_similarity=0.75):
 all_media = []
 page = 0
 total_pages = 1
 while page < total_pages:
 results = client.search_by_caption(dataset_id, caption_query,
 textual_similarity_threshold=min_similarity,
 page_number=page)
 all_media.extend(client.extract_media_info(results))
 total_pages = results.get('total_pages', 1)
 page += 1
 return all_media
```

## Error Handling

```
def search_with_retries(client, dataset_id, caption_query, max_retries=3):
 for attempt in range(max_retries):
 try:
 return client.search_by_caption(dataset_id, caption_query)
 except requests.exceptions.HTTPError as e:
 if e.response.status_code == 429:
 time.sleep((2 ** attempt) + random.uniform(0, 1))
 elif e.response.status_code == 401:
 raise Exception("Check your API token.")
 elif e.response.status_code == 404:
 raise Exception("Dataset not found.")
 else:
 raise
```

## Limitations

- Max 100 results per page
- Minimum threshold: 0.5

- Quality depends on caption accuracy
- Complex phrasing may yield fuzzy results

## Exporting a Dataset via curl

**How This Helps** Export datasets directly using curl without needing Python or SDKs. Ideal for automation scripts or CLI-based workflows.

### Initiate Export

Use a GET request to trigger export:

```
curl -X GET "https://app.visual-layer.com/api/v1/dataset//export_context_async" \
-H "Accept: application/json, text/plain, */*" \
-G \
--data-urlencode "file_name=export.zip" \
--data-urlencode "export_format=json" \
--data-urlencode "include_images=false"
```

- “: Your dataset ID
- export\_format: Supported values are json or parquet
- include\_images: true or false
- file\_name: Name for the output ZIP

### Sample Response

```
{
 "id": "fdb84834-d19b-4797-861a-d48b7a16f908"
}
```

### Poll Export Status

Use the export id to monitor progress:

```
curl -X GET "https://app.visual-layer.com/api/v1/dataset//export_status?export_task_id=" \
-H "Accept: application/json, text/plain, */*" \
```

Response includes a download URI once "status": "COMPLETED".

### Download ZIP

When ready, download your dataset:

```
curl -L "" --output exported_dataset.zip
```

Use the `-L` flag to follow redirects (required for S3 links).

## Unzip and Explore

Extract the ZIP archive:

```
unzip exported_dataset.zip -d exported_dataset
```

Example folder structure:

```
exported_dataset/
├─ metadata.parquet
└─ images/
 ├─ image1.jpg
 └─ image2.jpg
```

## Full Automation Script

```
#!/bin/bash
```

```
DATASET_ID="your-dataset-id"
FILENAME="export.zip"
```

```
echo "Initiating export..."
```

```
EXPORT_TASK_RESPONSE=$(curl -s -G "https://app.visual-layer.com/api/v1/dataset/$DATASET_ID/export" \
-H "Accept: application/json, text/plain, */*" \
--data-urlencode "file_name=$FILENAME" \
--data-urlencode "export_format=json" \
--data-urlencode "include_images=false")
```

```
EXPORT_TASK_ID=$(echo $EXPORT_TASK_RESPONSE | grep -o '"id":"[^"]*' | cut -d':' -f2 | tr -d '"')
```

```
echo "Export Task ID: $EXPORT_TASK_ID"
```

```
echo "Waiting for export to complete..."
```

```
STATUS="PENDING"
```

```
while ["$STATUS" != "COMPLETED"]; do
```

```
 STATUS_RESPONSE=$(curl -s "https://app.visual-layer.com/api/v1/dataset/$DATASET_ID/export/$EXPORT_TASK_ID" \
 -H "Accept: application/json, text/plain, */*")
```

```
 STATUS=$(echo $STATUS_RESPONSE | grep -o '"status":"[^"]*' | cut -d':' -f2 | tr -d '"')
 echo "Status: $STATUS"
```

```
 ["$STATUS" = "FAILED"] && echo "Export failed." && exit 1
```

```
 ["$STATUS" != "COMPLETED"] && sleep 5
```

done

```
DOWNLOAD_URI=$(echo $STATUS_RESPONSE | grep -o '"download_uri": "[^"]*' | cut -d ':' -f2)
echo "Downloading ZIP..."
curl -L "$DOWNLOAD_URI" --output $FILENAME

echo "Unzipping..."
unzip $FILENAME -d exported_dataset

echo "Done!"
```

## Filter Exported Data by Uniqueness

Use pandas to find the most unique images:

```
df = pd.read_parquet("exported_dataset/metadata.parquet")
top_unique_images = df.sort_values(by="uniqueness_score", ascending=False).head(100)
top_unique_images.to_csv("top_unique_images.csv", index=False)
```

## Copy Filtered Images

```
os.makedirs("top_unique_images", exist_ok=True)
for fname in top_unique_images["image_filename"]:
 shutil.copy(f"exported_dataset/images/{fname}", f"top_unique_images/{fname}")
```

## Summary

- Initiated export
- Polled export status
- Downloaded and unzipped archive
- Parsed metadata using pandas
- Used uniqueness\_score for image filtering

## Running Visual Layer Within an Iframe

### Embedding Visual Layer Search with Seamless Authentication

You can easily integrate **Visual Layer's** powerful visual search engine into your web application via an iframe.

This guide also explains how to automatically authenticate your users when embedding the search engine.

### Step 1: Basic Iframe Integration

Embed Visual Layer into your web page with a simple iframe:

```
[Embedded content]
```

#### What this does:

- Displays the Visual Layer search engine inside your app.
- Fills the width of the container (`width="100%"`).
- Uses a custom domain `vl.customer.com` for seamless authentication (see Step 2).

### Step 2: Cross-Domain Authentication Setup

To **automatically authenticate** your users into Visual Layer without requiring a manual login:

#### DNS Configuration

- Create a CNAME DNS record:  
`vl.customer.com → app.visual-layer.com`

This allows the iframe source to stay under the same parent domain (`.customer.com`).

#### Authentication Cookie

When a user accesses your app (`vl.customer.com/vl`), you need to **set a special cookie**:

- **Cookie Name:** `VL_AUTH`
- **Cookie Domain:** `vl.customer.com`
- **Cookie Value:**  
An **encrypted token** consisting of:  
`"customer_name+<current timestamp in unix seconds>"`

## Cookie Encryption

Use **Fernet symmetric encryption** for securing the cookie value.

### Example in Python

```
from cryptography.fernet import Fernet

Load your shared secret key
cipher = Fernet(shared_key)

Generate token
current_time = str(int(time.time()))
plain_text = f"customer_name+{current_time}"

Encrypt
encrypted_token = cipher.encrypt(plain_text.encode('utf-8'))

Set the cookie VL_AUTH to encrypted_token
```

### Example in PHP

```
use Defuse\Crypto\Crypto;
use Defuse\Crypto\Key;

$currentTime = time();
$cookieValue = "customer_name+" . $currentTime;
$key = Key::loadFromAsciiSafeString($sharedKey);

$ciphertext = Crypto::encrypt($cookieValue, $key);
// Set VL_AUTH cookie with $ciphertext
```

❑ Make sure both sides (your server and Visual Layer) share the **same encryption key** securely.

## Visual Layer Server Setup

- Create a **read-only Visual Layer user** limited to a specific dataset.
- Visual Layer will validate the VL\_AUTH cookie for iframe sessions.

## Full Example Flow

1. User logs into customer.com.
2. Your app sets the encrypted VL\_AUTH cookie.
3. User navigates to a page with the Visual Layer iframe.



4. Visual Layer reads the cookie, decrypts it, and logs the user in automatically.
5. User enjoys seamless visual search!

# **Self-Hosting**

## **Installation and Configuration**

## System Requirements

### System Configuration

The following hardware specifications are required for Visual Layer on-premises deployment:

Requirement	Specification
<b>Operating System</b>	Ubuntu LTS 20/22/24
<b>Architecture</b>	X86
<b>RAM</b>	64GB
<b>Disk Space</b>	256GB + 2.5× estimated data size
<b>CPU</b>	16 cores
<b>GPU (Optional)</b>	Common Nvidia GPU types with at least 24GB GPU memory (Nvidia A10, A100, H100, RTX 4090, etc.)

### Dependencies

Install these software dependencies before setting up Visual Layer:

- Docker v.27 or above - follow the official instructions and the post-install guide
- jq - json data parsing library
- psql - postgres command line client
- zip/unzip

### NVIDIA GPU Setup (GPU machines only)

If you are using a GPU machine for acceleration, you need to install the NVIDIA Container Toolkit:

1. Add NVIDIA package repositories:

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor
curl -fsSL https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.deb
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]#g'
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

2. Download the nvidia-container-toolkit:

```
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```

3. Configure Docker to use the NVIDIA runtime:

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

4. Verify the installation:

```
docker run --rm --runtime=nvidia --gpus all nvidia/cuda:11.5-base-ubuntu20.04 nvidia-smi
```

This should display your GPU information if the toolkit is properly installed.

## **Advised Tools for Troubleshooting/Maintenance**

These tools are recommended for system maintenance and troubleshooting:

- vi - or equivalent required for file editing
- object store CLIs if applicable (aws cli, mc)

## **Required Permissions and Access**

Ensure proper user permissions for installation:

- Make sure you run the installation as a regular user (not a root user), with sudo privileges

## Set Up the Environment

### Log into Docker Image Repo

To authenticate with Docker, run:

```
docker login -u visuallayer
```

When prompted for a password, copy and paste the **Docker Hub access token** provided by your Visual Layer representative.

- If you do not have the token or it is incorrect, contact support@visual-layer.com.

### Run the installer

#### GPU Deployment (Full Enrichment Enabled)

```
smarty Shell HARDWARE=gpu VL_VERSION="2.33.3-rc7" bash -c "$(curl -fsSL https://vl-public-bucket.s3.us-east-2.amazonaws.com/installer/new/install_helper.sh)"
```

#### CPU Deployment (No Enrichment)

```
HARDWARE=cpu VL_VERSION="2.33.3-rc7" bash -c "$(curl -fsSL https://vl-public-bucket.s3.us-east-2.amazonaws.com/installer/new/install_helper.sh)"
```

## Air-Gapped Environment Installation

### Prerequisites

- Docker installed on your system
- Docker login credentials for Visual Layer repository

### Step 1: Docker Authentication

First, authenticate with DockerHub using the provided credentials:

```
docker login
```

When prompted, use: - **Username:** visuallayer - **Password:** Contact support@visual-layer.com and we will share the password

### Step 2: Required Docker Images

The Visual Layer system consists of the following Docker images that need to be pulled from DockerHub:

Docker Image	Description
visuallayer/onprem-fe	Frontend web application interface
visuallayer/onprem-installer	Installation and setup utilities
visuallayer/onprem-healthcheck	System health monitoring service
visuallayer/onprem-pl-cpu	CPU-based processing pipeline
visuallayer/onprem-pl-gpu	GPU-accelerated processing pipeline
visuallayer/onprem-pg	PostgreSQL database server
visuallayer/onprem-be	Backend API services
visuallayer/onprem-cd	Continuous deployment service
visuallayer/onprem-model-pack	Pre-trained models and model management

Pull all required images (replace “ with the specific version provided by Visual Layer support):

```
Pull all required images
docker pull visuallayer/onprem-fe:
docker pull visuallayer/onprem-installer:
docker pull visuallayer/onprem-healthcheck:
docker pull visuallayer/onprem-pl-cpu:
docker pull visuallayer/onprem-pl-gpu:
docker pull visuallayer/onprem-pg:
docker pull visuallayer/onprem-be:
docker pull visuallayer/onprem-cd:
docker pull visuallayer/onprem-model-pack:
```

## Step 3: Get Installation Script

First, obtain the `install_helper.sh` script. You can either download it using `curl` or request it from Visual Layer support:

```
Download the installation script
curl -fsSL https://vl-public-bucket.s3.us-east-2.amazonaws.com/installer/new/install_h
```

Alternatively, contact [support@visual-layer.com](mailto:support@visual-layer.com) to get the script.

## Step 4: Run Installation

Replace “ with the specific version provided by Visual Layer support.

### For GPU-enabled Systems:

```
HARDWARE=gpu VL_VERSION="" bash install_helper.sh
```

### For CPU-only Systems:

```
HARDWARE=cpu VL_VERSION="" bash install_helper.sh
```

## Step 5: Verification

After installation, verify that all containers are running properly:

```
docker ps
```

You should see all the Visual Layer containers running.

## Troubleshooting

- If containers fail to start, check the logs: `docker logs <container_name>`
- Ensure you have sufficient disk space for all images
- Verify network connectivity to DockerHub
- Make sure Docker daemon is running

## Configure Settings

LOG\_LEVEL

**Description:** Controls how verbose the logging system is. Useful for debugging or reducing noise in production.

**Accepted Values:**

DEBUG

INFO (default)

WARNING

ERROR

CRITICAL

Example:

```
export LOG_LEVEL='10'
then run the profiler on-prem according to the run_profiler.sh instructions
```

At DEBUG, additional internal processing messages will be logged, including more detailed diagnostics (e.g., when the log level is set to 10, extra diagnostic info is shown).

---

SWAP\_RGB\_CHANNELS\_FOR\_TIFF\_HEIC

**Description:** Corrects color rendering issues that can occur in certain TIFF and HEIC image formats, where red and blue channels may appear reversed.

**Behavior:** When enabled, the red and blue color channels are swapped while green remains unchanged. This ensures that images are displayed using the expected RGB format across all systems.

Example:

```
c bash export SWAP_RGB_CHANNELS_FOR_TIFF_HEIC=true
```



## S3 Configuration

The following configuration should be added to the config.json file inside the .vl/ folder.

### □ Configuration Flags

```
{
 "SERVE_ORIGINAL_IMAGE": true,
 "CDN_FULLPATH": "/image",
 "SAVE_THUMBNAILS_METHOD": "crops",
 "PREPROCESS_IMAGE_TRANSFER": "keep_original_filename",
 "PREPROCESS_NORMALIZE_OUTPUT": false,
 "REMOTE_SAVE": "s3://visual-layer/tmp/remove-me-test-folder",
 "SOURCE_USE_FULLPATH": true,
 "USE_S5CMD_WHEN_AVAILABLE": true,
 "SKIP_IMAGE_THUMBNAIL_GENERATION": true,
 "EXPORT_ENTITIES_RESULT_BUCKET": "mybucket",
 "OBJECT_STORE_URL": "http://minio:9000"
}
```

#### SERVE\_ORIGINAL\_IMAGE: true

- **Purpose:** Forces the UX to serve the original image directly from the remote storage (e.g., S3).
- **Use case:** Ideal for high-speed networks or when thumbnails are not needed.
- **Note:** This must be combined with PREPROCESS\_IMAGE\_TRANSFER: "keep\_original\_filename" to ensure proper file linking.

#### PREPROCESS\_IMAGE\_TRANSFER: "keep\_original\_filename"

- **Purpose:** Ensures the original image filenames are preserved during transfer and storage.
- **Impact:** Required when using SERVE\_ORIGINAL\_IMAGE to allow consistent referencing of images in the frontend.

#### CDN\_FULLPATH: "/image"

- **Purpose:** Defines the root path where images are served through a CDN or proxy.
- **Required:** Yes – this path is mandatory for the UX to resolve asset URLs correctly.

#### SOURCE\_USE\_FULLPATH: true

- **Purpose:** Maintains the original folder structure (relative paths) of source files during pre-processing and storage.
- **Use case:** Helps track image provenance and supports hierarchical datasets.

**REMOTE\_SAVE: “s3://mybucket/visual-layer-intermediate/”**

- **Purpose:** Specifies the destination path (S3 or other remote store) for intermediate processed assets.
- **Required:** Yes – used by the UX to access thumbnails, metadata, or originals.

**USE\_S5CMD\_WHEN\_AVAILABLE: true**

- **Purpose:** Uses s5cmd for file transfers instead of AWS CLI or boto3.
- **Benefit:** Can achieve up to 10x faster S3 copy operations.
- **Optional:** Yes – recommended for large datasets or high-throughput pipelines.

**SKIP\_IMAGE\_THUMBNAIL\_GENERATION: true**

- **Purpose:** Disables thumbnail generation during preprocessing.
- **Use case:** For fast uploads when bandwidth is not a concern and SERVE\_ORIGINAL\_IMAGE is enabled.
- **Optional:** Yes – reduces storage and processing cost in some environments.

**SAVE\_THUMBNAILS\_METHOD: “crops”**

- **Purpose:** Defines the method used to save thumbnails. “crops” stores only relevant cropped regions (e.g., detections or ROIs).
- **Effect:** Reduces thumbnail size and focuses on semantically important regions.
- **Optional:** No

**EXPORT\_ENTITIES\_RESULT\_BUCKET: “mybucket”**

- **Purpose:** Specifies an optional S3 bucket name where export files will be saved.
- **Format:** Should contain just the bucket name without the s3:// prefix (e.g., “mybucket” instead of “s3://mybucket/”).
- **Use case:** When exporting entities or results, files will be stored in this bucket instead of the default location.
- **Optional:** Yes – if not specified, exports will use the default export location.

**OBJECT\_STORE\_URL: “http://minio:9000”**

- **Purpose:** Configures a custom S3-compatible object storage endpoint URL for the system.
- **Use case:** Allows using alternative object storage services (like MinIO or on-premise S3-compatible storage) instead of AWS S3. Enables on-premise or custom object storage deployments.
- **Behavior:** When set, it overrides the default AWS S3 endpoint for all storage operations including downloads and preprocessing artifact storage.
- **Technical details:** When URL starts with http://, SSL verification is disabled for local deployments. Used for both data downloads and preprocessing operations.
- **Optional:** Yes – if not specified, the system will use the default AWS S3 endpoint.

## ❑ AWS Credentials Setup

To enable remote saving and loading of assets from S3, you must provide AWS credentials. Visual Layer looks for credentials in the file:

`.vl/aws_credentials`

## ❑ File Format

The file must follow the standard AWS credentials format used by the AWS CLI and boto3:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY_ID
aws_secret_access_key = YOUR_SECRET_ACCESS_KEY
region = YOUR_AWS_REGION #optional
```

**Example:**

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
region = us-west-2
```

## ❑ File Location

Make sure the file is located at:

`<your_project_root>/.vl/aws_credentials`

❑ **Tip:** `.vl` is a hidden folder used by Visual Layer to store configuration and credentials.

## ❑ Security Best Practices

Use temporary credentials (e.g., via IAM roles or STS) in production if possible.

## ❑ Custom SSL Certificate (Optional)

If your S3 endpoint requires a custom SSL certificate (e.g., when using a private MinIO or S3-compatible object store behind a proxy), you can set the `AWS_CA_BUNDLE` environment variable.

## ❑ What it does:

This tells boto3, s5cmd, and other AWS SDK tools to use a specific `.pem` certificate file when making HTTPS requests.

## ❑ Usage

Copy your certificate `.pem` file into the local `.vl` folder. Set the environment variable before launching your Visual Layer job:

```
export AWS_CA_BUNDLE=./.vl/custom-ca.pem
```

## □ Folder Structure Summary

```
project-root/
├── .vl/
│ ├── aws_credentials
│ └── aws_ca_bundle.pem # Optional
├── config.json # Your Visual Layer configuration
└── run_profiler.sh # Run a pipeline
```

## □ Need Help?

For support, reach out to your Visual Layer technical contact or email: [support@visual-layer.com](mailto:support@visual-layer.com).

## Usage

## Basic Operations

### Connect to the Application

Once the system is running, connect to the application using your browser:

- **HTTP**: `http://<server name>:2080`
- **HTTPS**: `https://<server name>:2443`

### Start the System

```
./start.sh
```

### Stop the System

```
./stop.sh
```

### Creating Datasets

```
./run_profiler.sh -n "<desired dataset name>" -p "<dataset source dir>"
```

- This process **can take several hours**, depending on data size.
- If the dataset name includes spaces, use **quotation marks**(e.g., "My Dataset").

## Accessing Logs & Support

### Debugging Run

Use the following environment variables to enable debugging:

```
LOG_LEVEL=10 # Set log level to debug
REPORTING_LOG_PATH="/path/to/log.txt" # Write pipeline log to file
```

## The VL Profiler Script: run\_profiler.sh

The run\_profiler.sh script enables processing and profiling of image datasets for the Visual Layer platform. It supports multiple data sources, including local directories, S3 buckets, HTTP/HTTPS URLs, and file lists in .txt, .csv, or .parquet formats. With flexible execution modes and broad data compatibility, the script adapts to a wide range of environments and workflows.

The rest of this article outlines script usage syntax and provides detailed explanations for each command-line parameter.

## Usage

Following is the command-line syntax for running the script:

```
./run_profiler.sh [-h] [-p <path> -n <dataset_name>] [-e compose|local] [-r]
```

## Parameters

Following are the command-line parameters supported by the script:

### Dataset Path (-p, *required*)

Specifies the source location of your dataset. Multiple formats are supported:

Local directory	~/data/images
S3 bucket	s3://mybucket/images
HTTP/HTTPS URL	https://example.com/dataset
File list	Path to a .txt, .csv, or .parquet file containing image paths

### Dataset Name (-n, *required*)

A human-readable name for your dataset. This name will be used for identification in logs and results.

### Execution Mode (-e)

Determines how the processing pipeline is executed.

**default:** compose

compose	Uses Docker Compose and the Visual Layer API (recommended for most users, default mode)
---------	-----------------------------------------------------------------------------------------

---

local	Runs directly on your local machine using a Python virtual environment (advanced)
-------	-----------------------------------------------------------------------------------

---

## Reduce Disk Space (-r)

Enables reduced disk space consumption mode. Useful for large datasets or environments with limited storage.

**default:** false

When enabled, activates `serve_mode=reduce_disk_space` for optimized storage usage.

## Help (-h)

Displays usage information and example commands.

## Examples

```
bash Local Directory ./run_profiler.sh -p ~/data/ds1 -n 'dataset1'
bash S3 Directory ./run_profiler.sh -p s3://mybucket/images -n 'dataset1'
bash File List (Local) ./run_profiler.sh -p ~/filelist.txt -n 'dataset1'
bash File List (S3) ./run_profiler.sh -p s3://mybucket/images/filelist.parquet -n 'dataset1'
bash Reduced Disk Space ./run_profiler.sh -p ~/data/ds1 -n 'dataset1' -r
```

## Execution Modes

The script supports two execution modes for processing datasets:

### Compose Mode (Default)

**Info: Recommended for most users.** Uses Docker Compose to run the processing pipeline with the Visual Layer API.

Feature	Description
Processing Method	Processes datasets through HTTP API endpoint
Supported Data Sources	Local, S3, HTTP/HTTPS
Path Handling	Automatically handles path encoding for remote sources
Integration	Integrates with the full Visual Layer service stack

**API Endpoint:** POST `http://localhost:2080/api/v1/process`

**Parameters sent to API:**

Parameter	Description
path	Dataset source path (URL-encoded for remote sources)
name	Dataset name
serve_mode	Set to <code>reduce_disk_space</code> when <code>-r</code> flag is used

## Local Mode

**Warning: Advanced users only.** Runs the pipeline directly using Python virtual environment.

- Executes `pipeline.controller.controller` module directly
- Requires local Python virtual environment setup
- Uses manual flow configuration (`MANUAL_FLOW=yes`)
- Configures device settings based on hardware type (CPU/GPU)

**Environment Variables Set:**

- `MANUAL_FLOW=yes`
- `FASTDUP_PRODUCTION=1`
- `PREFECT_LOGGING_SETTINGS_PATH=./.vl/prefect-logging.yaml`
- Device-specific settings for CPU mode

## Data Source Support

### Local Paths

- Converts relative paths to absolute paths using `realpath`
- For file inputs (lists), copies to `.vl/` directory for container access
- Validates path existence before processing

### Remote Paths

Source Type	Example/Details
<b>S3</b>	<code>s3://bucket/path</code>
<b>HTTP/HTTPS</b>	<code>http://</code> or <code>https://</code> URLs
<b>File Detection</b>	Automatically detects file extensions ( <code>.txt</code> , <code>.csv</code> , <code>.parquet</code> )
<b>URL Encoding</b>	Applies proper encoding for API transmission

## Supported File Formats

- **Image Directories:** Any directory containing image files
- **File Lists:**
  - `.txt`: Plain text file with one file path per line



- .csv: CSV file with file paths
- .parquet: Parquet file containing file path data

## Error Handling

The script includes comprehensive error handling:

Error Type	Description
<b>Missing Arguments</b>	Displays usage information and exits
<b>Invalid Paths</b>	Validates local path existence
<b>Invalid Execution Mode</b>	Ensures mode is either compose or local
<b>API Failures</b>	Captures and displays API error responses
<b>Pipeline Failures</b>	Handles local pipeline execution errors

## Dependencies

Dependency	Purpose/Usage
<b>Bash</b>	Shell environment
<b>curl</b>	API communication (compose mode)
<b>Python 3</b>	Local execution and URL encoding
<b>Docker Compose</b>	Compose mode execution
<b>Virtual Environment</b> (./venv_local/)	Local mode Python environment

## Output

### Compose Mode

- Success: Displays dataset processing confirmation with response
- Failure: Shows API error message in red text

### Local Mode

- Runs pipeline with full logging output
- Returns to original shell environment on completion

## Performance Considerations

### Reduced Disk Space Mode (-r)

- Activates serve\_mode=reduce\_disk\_space parameter
- Optimizes storage usage during processing
- Recommended for large datasets or limited storage environments

## Hardware Configuration

- **CPU Mode:** Automatically configures all processing devices to use CPU
- **GPU Mode:** Uses default GPU acceleration when available

## Integration

This script integrates with the Visual Layer platform's core components:

Component	Description
Pipeline Controller	Orchestrates the complete processing workflow
Database	Stores processed dataset metadata and results
API Service	Provides RESTful interface for dataset operations
Storage Systems	Supports local filesystem, S3, and HTTP sources

## Troubleshooting

### Common Issues

#### Path not found errors

- Verify local paths exist and are accessible
- Check S3 credentials and permissions for S3 paths
- Ensure HTTP/HTTPS URLs are accessible

## Administration

## **User Management: ADFS (OIDC) Integration Guide**

This document provides all necessary instructions to integrate your Identity Provider (IdP) — such as ADFS — with Visual Layer's platform using the OpenID Connect (OIDC) protocol.

We will work together to ensure a smooth Single Sign-On (SSO) experience.

## **OIDC Authentication Flow**

### **Step: Login Redirect**

The user tries to log in to Visual Layer and is redirected to your IdP (OpenID Provider, OP) with the client ID.

### **Step: User Authentication**

The IdP authenticates the user.

### **Step: Authorization Code Return**

The user is redirected back to a predefined Redirect URI with an authorization code.

### **Step: Token Exchange**

Visual Layer exchanges this code, along with the client ID and secret, for Access and ID tokens.

### **Step: User Info Retrieval**

Optionally, additional user details can be retrieved using the Access Token via the UserInfo endpoint.

## **Information Required from Your Side**

### **1. Well-Known Configuration URL**

- Provide your IdP's OpenID configuration URL (For example: `https://<your-adfs-domain>/adfs/.well-known/openid-configuration`).
- This allows us to automatically fetch metadata like authorization endpoints.

### **2. Client ID and Client Secret**

- Generate these credentials on your IdP (ADFS or any OIDC-compliant provider).
- We will use these securely to authenticate against your IdP.

### **3. Allowed Scopes**

- Confirm which scopes are supported.
- Typical scopes include: `openid`, `profile`, `email`.

## 4. Example Tokens

- No need to send real user tokens.
- Provide an example ID token and, if possible, a UserInfo response showing the available user claims.

```
{
 "sub": "abc123",
 "email": "user@company.com",
 "given_name": "Example",
 "family_name": "User",
 "preferred_username": "example.user"
}
```

## Information We Will Provide

### 1. Redirect URI

- Please configure this URI in your IdP:  
`https:///api/v1/oidc/callback`
- This is where users will be redirected after authentication.

## Required Changes in Your OpenShift Settings

### Update or Add the Following Settings:

```
Existing settings
DISABLE_AUTH: 'false'
USAGE_REPORT_ENABLED: 'false'
STORAGE_KEY: <randomly generated key>

New settings
OIDC_ISSUER: <value from well-known URL>
OIDC_CLIENT_ID:
OIDC_CLIENT_SECRET:
OIDC_REDIRECT_URI: 'https:///api/v1/oidc/callback'
```

### Frontend (FE) Settings:

```
REACT_APP_IS_AUTHENTICATION_DISABLED: false
```

**Tip:** To generate a random storage key:

```
openssl rand -base64 32
```

## Permissions Management

- User access will be managed via the **Dataset Manager Script**.
- The customer-facing script is available here:  
Dataset Manager Script on GitHub

## Additional Notes

- If needed, we can assist you with ADFS configuration, token structure validation, or claim mapping.
- We support both cloud and on-premises deployments.
- On-prem users: if you're interested in an evaluation, we can provide a trial version to test before committing.

## Troubleshooting

### Resolving Common Errors

#### Problem: Image Not Found

```
docker: Error response from daemon: pull access denied for visuallayer/onprem-installer
```

**Solution:** Ensure you are logged into Docker.

#### Problem: Docker Daemon Not Running

```
docker: Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
```

**Solution:** Start the Docker daemon.

#### Problem: Image Pulling Fails

```
manifest for visuallayer/onprem-cd:2.25.6-rc6 not found: manifest unknown
```

**Solution :** Verify the\*\* software version\*\*and that you are logged into Docker with the correct user.

#### Problem: Platform Mismatch Warning

```
WARNING: The requested image's platform (linux/arm64) does not match the detected host platform (linux/amd64)
```

**Solution :** This is a\*\* warning only\*\*—can be ignored.

#### Problem: docker compose Not Found

```
./run_profiler.sh: line 159: docker compose: command not found
```

**Solution:** Install or reinstall Docker following the official installation instructions.

#### Problem: PostgreSQL Operational Error

```
sqlalchemy.exc.OperationalError: (psycopg.OperationalError) [Errno -3] Temporary failure in name resolution
```

**Solution:** Ensure the system was brought up using the `docker up` command.